

数据结构

Ch6 树和二叉树

北京邮电大学 计算机学院

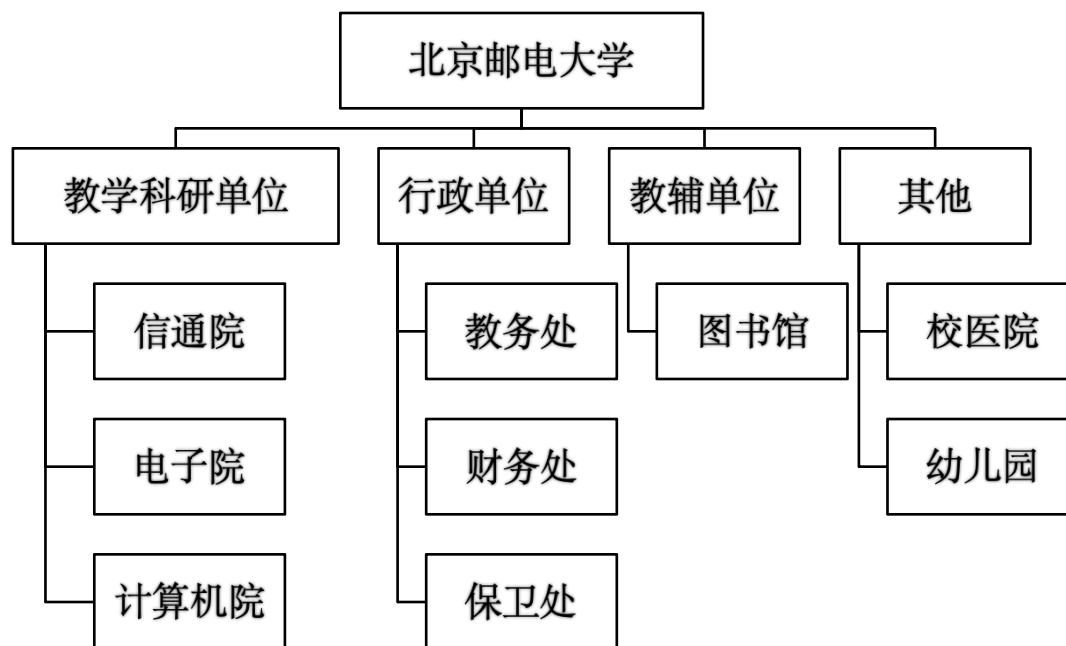
第6章 树和二叉树

- 6.1 树的定义和基本术语
- 6.2 二叉树（定义、性质与存储结构）
- 6.3.1 遍历二叉树
- 6.3.2 线索二叉树
- 6.6 哈夫曼树及其应用
- 6.4 树和森林
- 6.7 回溯法与树的遍历

6.1 树的定义和基本术语

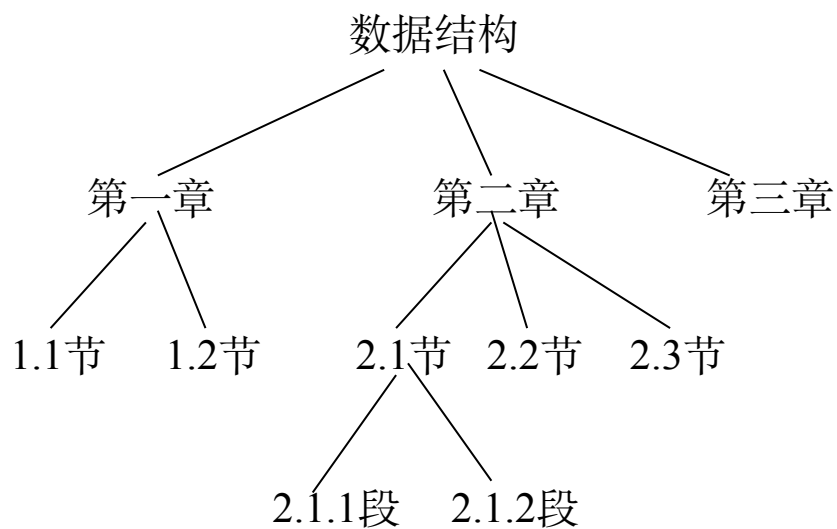
6.1.1 问题提出

线性结构是指元素之间至多有一个前驱元素或一个后继元素的情况，然而在现实生活中或数学抽象中还有一种情况是元素至多有一个前驱元素，而可有多个后继元素的情况，称之为**树结构**。



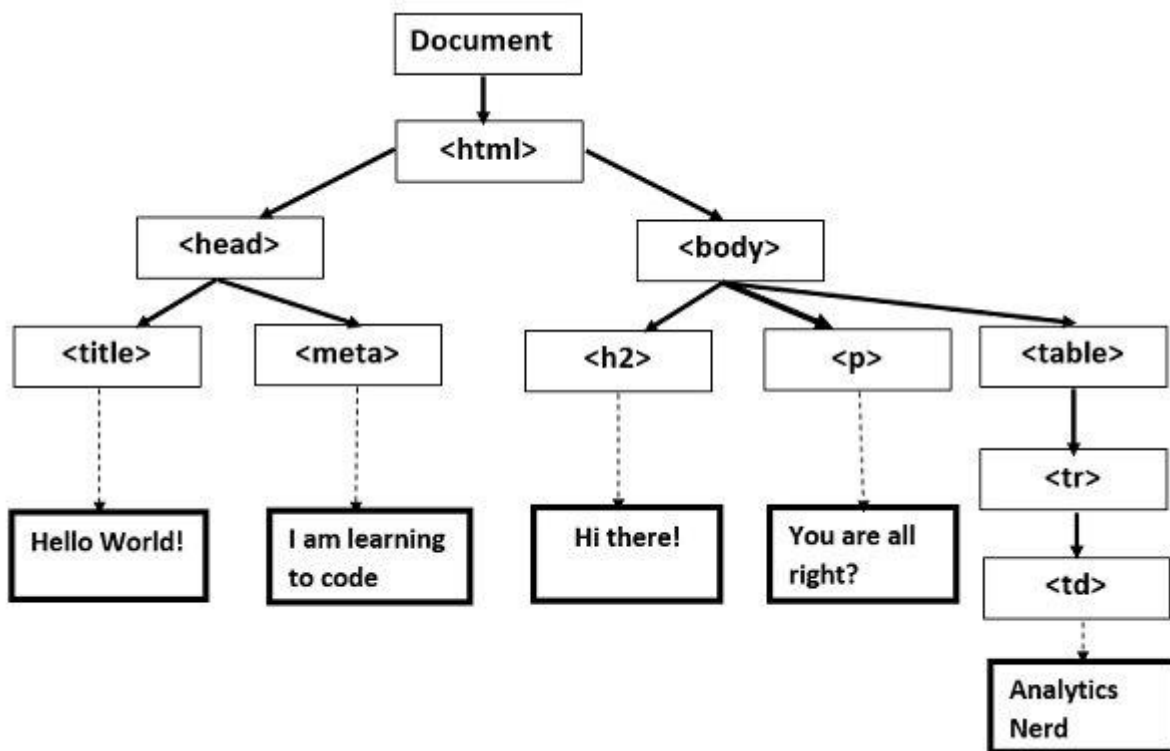
6.1 树的定义和基本术语

6.1.1 问题提出



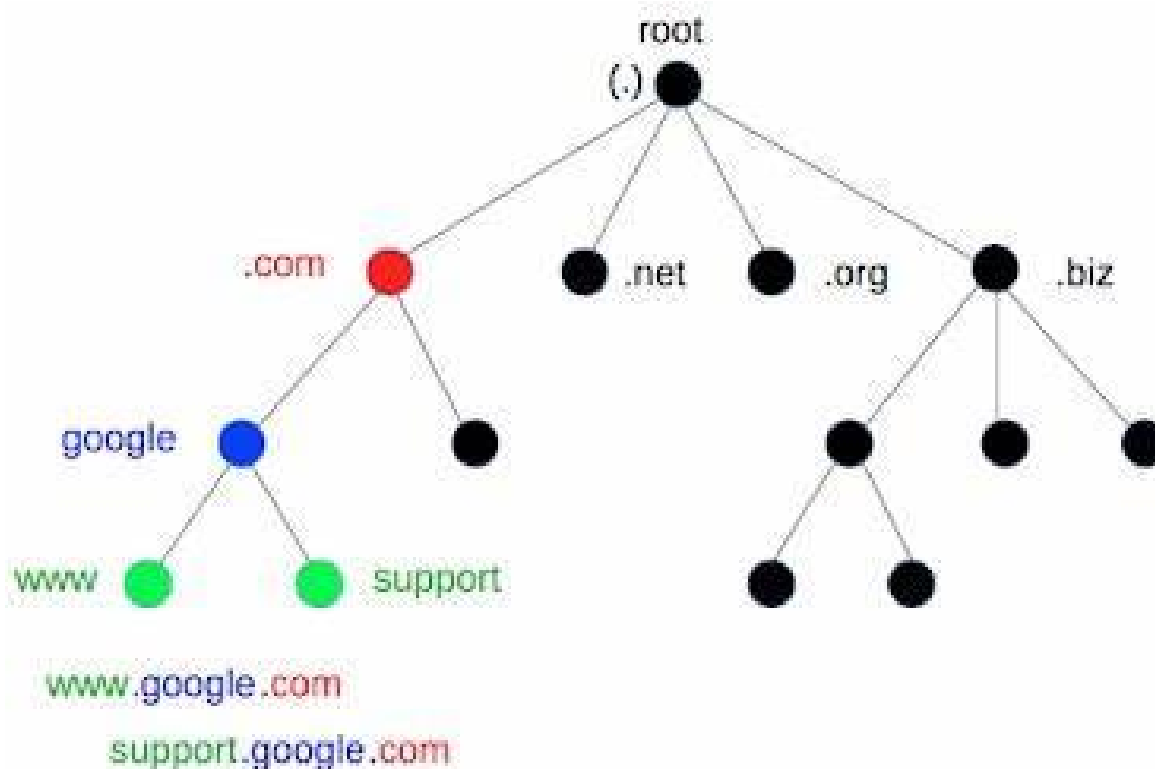
6.1 树的定义和基本术语

6.1.1 问题提出



6.1 树的定义和基本术语

6.1.1 问题提出



6.1 树的定义和基本术语

6.1.2 树的定义

树是一类重要的非线性数据结构，是以分支关系定义的层次结构。

树是由 $n(n \geq 0)$ 个结点组成的有限集合 T ，非空树满足：

1) 有一个称之为根(root)的结点，根结点没有前驱结点。

2) 除根以外的其余结点被分成 $m(0 \leq m < n)$ 个互不相交

的集合 $T_1, T_2, \dots, T_m$ ，其中每一个集合本身又

是一棵树，且称为根的子树。

- 树的递归定义刻画了树的固有特性，即一棵非空树是由若干棵子树构成的，而子树又可由若干棵更小的子树构成。

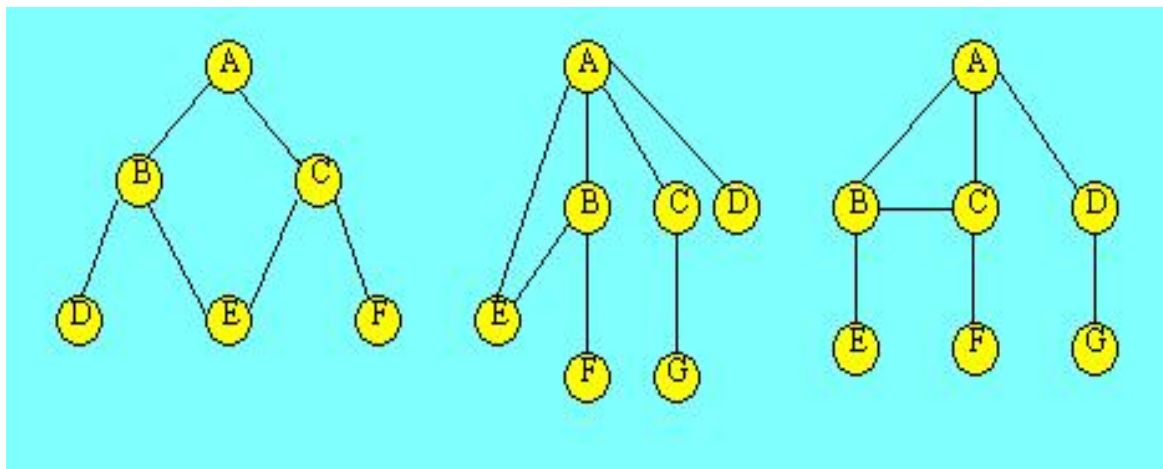
6.1 树的定义和基本术语

6.1.2 树的定义

树具有下面两个特点：

- (1) 树的根结点没有前驱结点，除根结点之外的所有结点有且只有一个前驱结点。
- (2) 树中所有结点可以有零个或多个后继结点。

由此特点可知，下图所示的都不是树结构。



基本术语

结点的度: 结点拥有的子树数目。

树的度: 树的各结点度的最大值。

叶子(终端)结点: 度为0的结点。

分支(非终端)结点: 度不为0的结点。

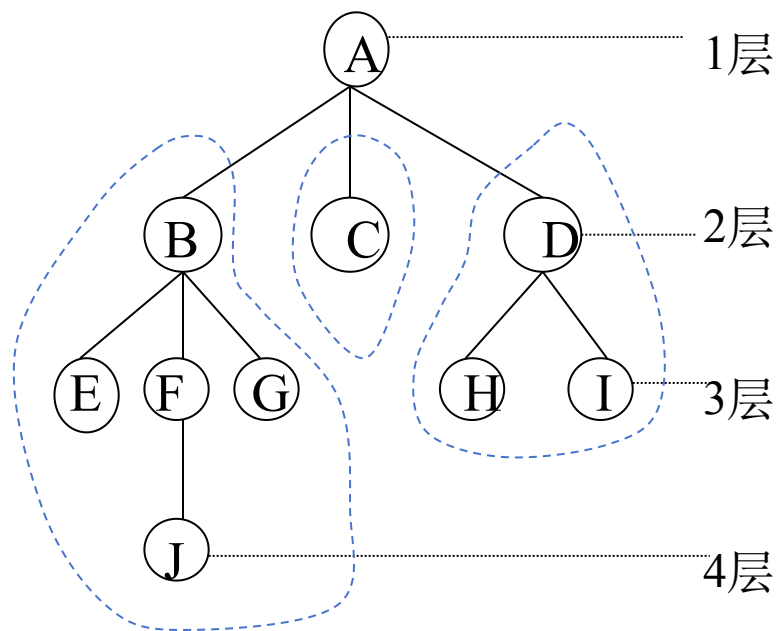
内部结点: 除根结点之外的分支结点。

双亲与孩子(父与子)结点: 结点的子树的根称为该结点的孩子; 该结点称为孩子的双亲。

兄弟: 属于同一双亲的孩子。

结点的祖先: 从根到该结点所经分支上的所有结点。

结点的子孙: 该结点为根的子树中的任一结点。



基本术语

结点的层次: 表示该结点在树中的相对位置。根为第一层, 其它的结点依次下推; 若某结点在第 L 层上, 则其孩子在第 $L+1$ 层上。

树的深(高)度: 树中结点的最大层次。

有序树: 树中各结点的子树从左至右是有次序的, 不能互换。否则, 称为**无序树**。

路径长度: 从树中某结点 N_i 出发, 能够通过树中结点到达结点 N_j , 则称 N_i 到 N_j 存在一条路径, 路径长度等于这两个结点之间的分支/边的个数。

森林: 是 $m(m \geq 0)$ 棵互不相交的树的集合。

树的基本操作

1)初始化操作

InitTree(&T)

2)销毁树操作

DestroyTree(&T)

3)建树函数

CreateTree(&T,definition)

4)将树清为空树操作

ClearTree(&T)

5)判断是否为空树

TreeEmpty(T)

6)返回树的深度

TreeDepth(T)

7)求根函数

Root(T) / Root(cur_e)

8)返回结点值

Value(T, cur_e)

9)给结点赋值

Assign(T, cur_e, value)

树的基本操作

10)求双亲函数

Parent(T,cur_e)

11)求孩子结点函数

Child(T,cur_e,i)

12)求右兄弟函数

RightSibling(T,cur_e)

13)插入子树操作

InsertChild(&T, &p, i, x)

14)删除子树操作

DelChild(&T,&p,i)

15)遍历操作

TraverseTree(T, Visit())

6.2 二叉树

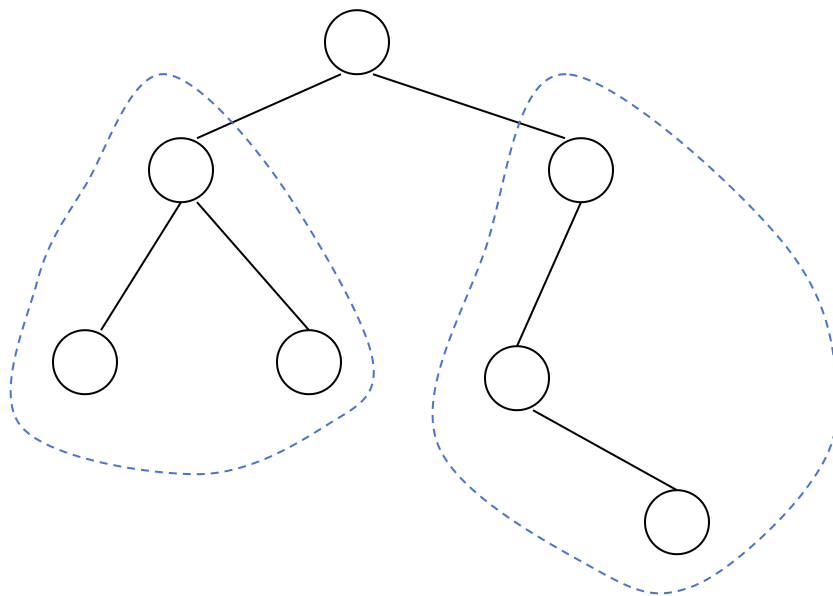
6.2 二叉树

6.2.1 二叉树的概念

二叉树的定义 二叉树是 $n(n \geq 0)$ 个结点的有限集合，它或为**空树**($n=0$)，或由一个**根**结点和两棵互不相交的被分别称为根的**左子树**和根的**右子树**的二叉树组成。

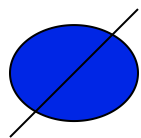
二叉树的特点

- 定义是递归的
- $0 \leq \text{结点的度} \leq 2$
- 是有序树

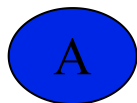


二叉树的每个结点至多只有两棵子树，且子树有左右之分，其次序不能任意颠倒。

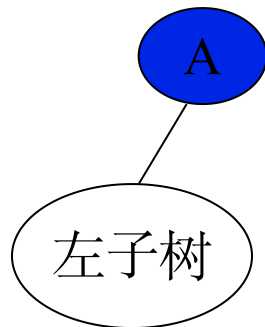
二叉树结点的子树要区分左子树和右子树，即使只有一棵子树也要进行区分，说明它是左子树，还是右子树。这是二叉树与树的最主要的差别。下图列出二叉树的5种基本形态，图(C) 和图 (d) 是不同的两棵二叉树。



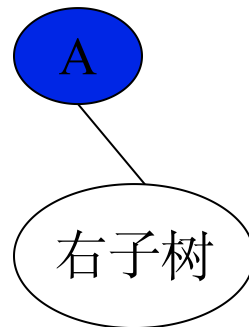
(a)
空二叉树



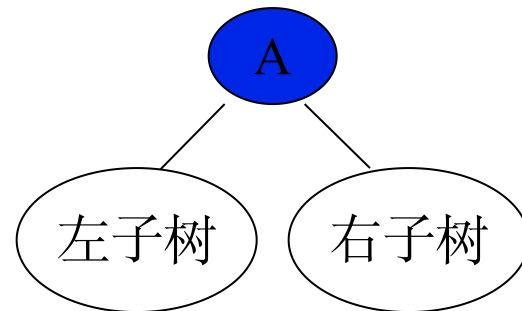
(b)
根和空的左
右子树



(c)
根和左子树



(d)
根和右子树

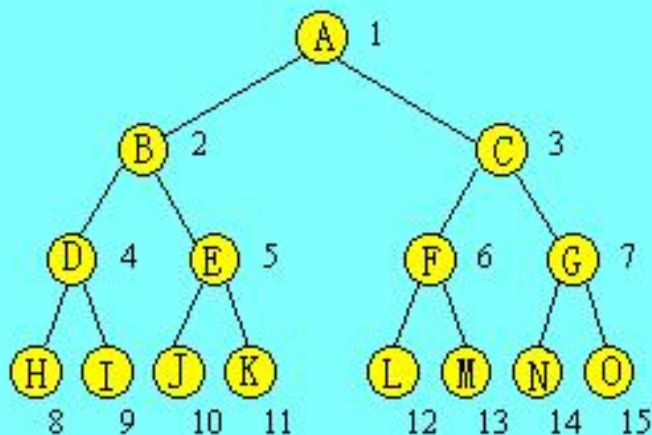


(e)
根和左右子树

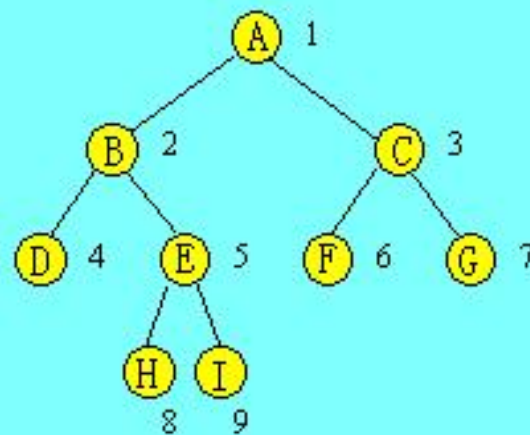
二叉树的5种形式

满二叉树:

如果一棵二叉树每一层的结点个数都达到了最大，这棵二叉树称为满二叉树。对于满二叉树，所有的分支结点都存在左子树和右子树，所有叶子都在最下面一层。一棵深度为 k 的满二叉树有 2^k-1 个结点。



(a) 一棵满二叉树



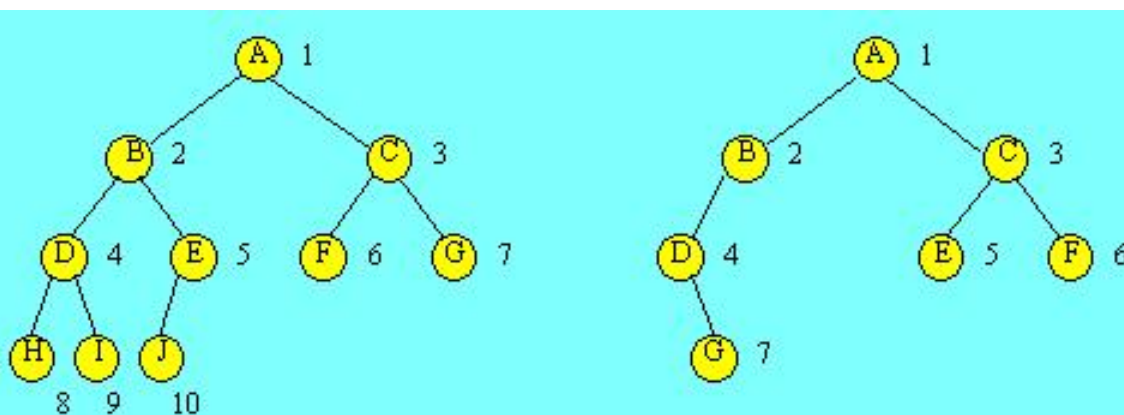
(b) 一棵非满二叉树

满二叉树和非满二叉树示意图

完全二叉树:

一棵深度为 k 的有 n 个结点的二叉树，对其结点按从上至下，从左到右的顺序进行编号，如果编号为 i 的结点与满二叉树中编号为 i 的结点在二叉树中的位置相同，则这棵二叉树称为完全二叉树。

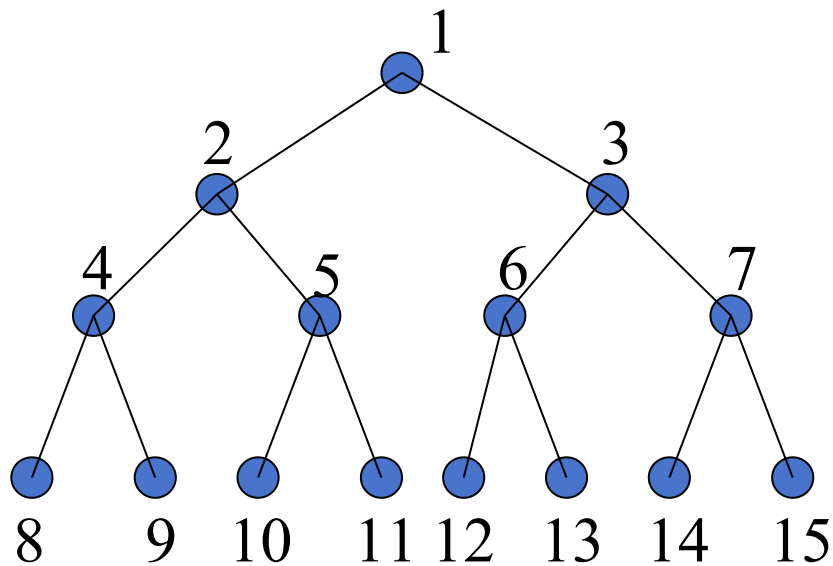
完全二叉树的特点是叶子结点只能出现在最下层和次最下层，并且最下面一层上的结点都集中在该层最左边的若干位置上，至多只有最下面的两层上结点的度数可以小于2，满二叉树肯定是完全二叉树，而完全二叉树未必是满二叉树。



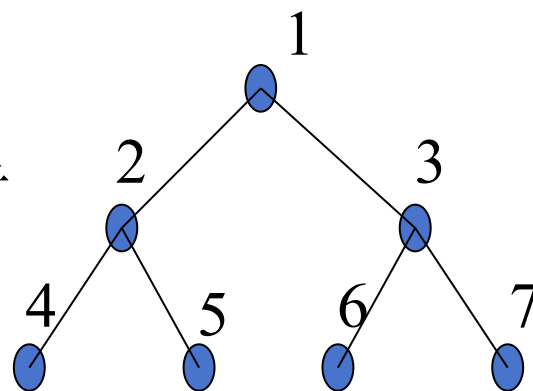
(a) 一棵完全二叉树

(b) 一棵非完全二叉树

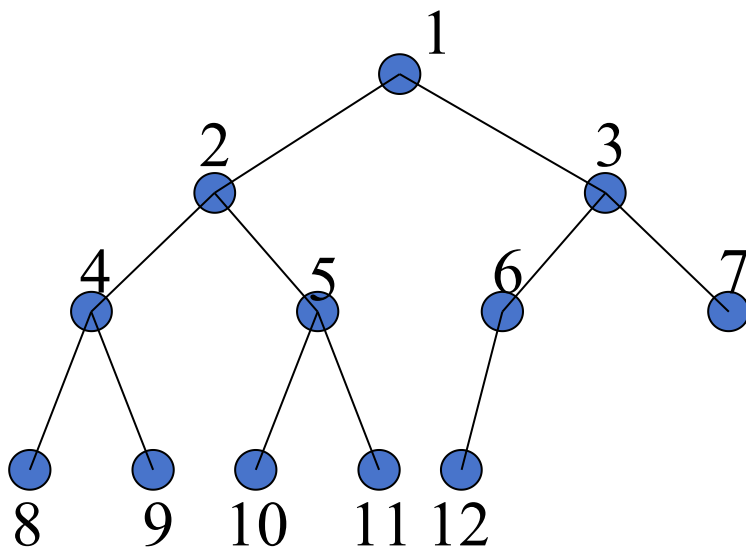
完全二叉树和非完全二叉树示意图

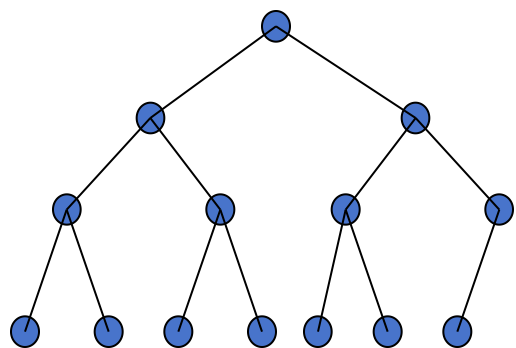


满二叉树

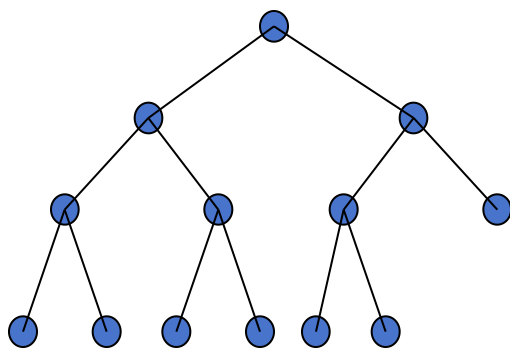


完全二叉树

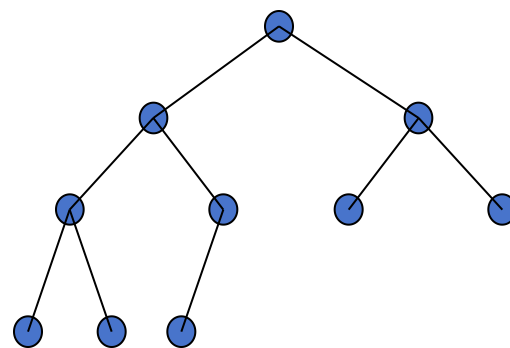




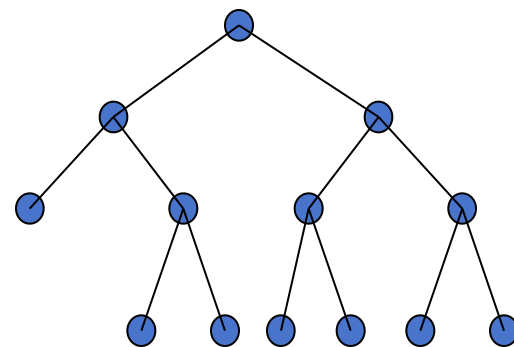
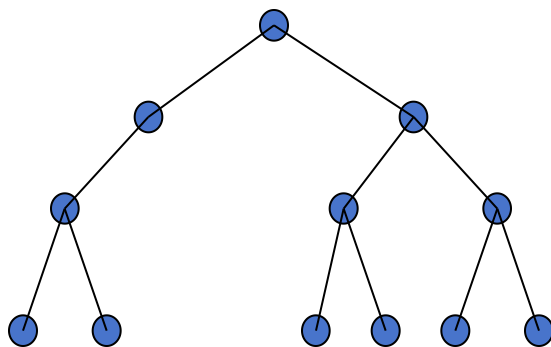
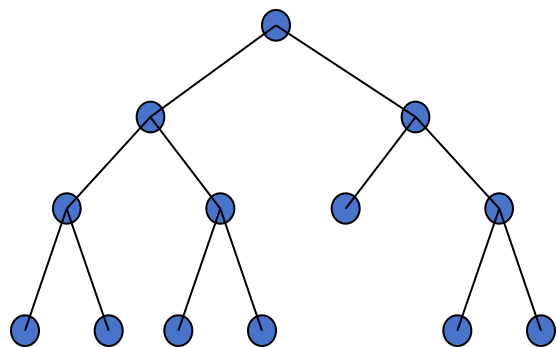
完全二叉树



完全二叉树



完全二叉树



6.2.2 二叉树的性质

性质1 二叉树的第 i 层上至多有 2^{i-1} ($i \geq 1$)个结点。

性质2 深度为 k 的二叉树至多有 $2^k - 1$ 个结点 ($k \geq 1$)。

性质3 对任何一棵二叉树 T ，如果其终端结点数为 n_0 ，度为2的结点数为 n_2 ，则 $n_0 = n_2 + 1$ 。

性质4 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$ 。

性质5 一棵具有 n 个结点的完全二叉树(又称顺序二叉树)，对其结点按层从上至下(每层从左至右)进行1至 n 的编号，则对任一结点 i ($1 \leq i \leq n$)有：

(1)若 $i > 1$ ，则 i 的双亲是 $\lfloor i/2 \rfloor$ ；若 $i = 1$ ，则 i 是根，无双亲。

(2)若 $2i \leq n$ ，则 i 的左孩子是 $2i$ ；否则， i 无左孩子。

(3)若 $2i + 1 \leq n$ ，则 i 的右孩子是 $2i + 1$ ；否则， i 无右孩子。

性质1证明

数学归纳法，假设 $i=k$ 时命题成立，即第 k 层最多有 2^{k-1} 个结点，由于二叉树每个结点最多有两个孩子结点，因此第 $k+1$ 层的结点数最多为第 k 层的两倍，即 $2 \times 2^{k-1} = 2^{(k+1)-1}$

性质2证明

$$1 + 2^1 + 2^2 + 2^3 + \dots + 2^{k-1} = 2^k - 1$$

性质3证明

设叶子结点数为 n_0 , n_1 为二叉树中度为1的结点数, 度为2的结点数为 n_2

- 进入节点分支数= $n_0+n_1+n_2-1$ (除根结点外每个结点都有唯一的进入分支)
- 离开节点的分支数= n_1+2n_2
- $n_1+2n_2 = n_0+n_1+n_2-1$
- $n_2=n_0-1$

性质4证明

- 假设深度为 k ，根据完全二叉树的定义和性质2可知：

有 $2^{k-1} - 1 < n \leq 2^k - 1$ 且 k 是整数

则： $2^{k-1} \leq n < 2^k$

$$K-1 \leq \log_2 n < K$$

$$K-1 = \lfloor \log_2 n \rfloor$$

二叉树的基本操作

1)初始化操作	InitBiTree(&BT)
2)销毁操作	DestroyBiTree(&BT)
3)建树函数	CreateBiTree(&BT, definition)
4)清除结构操作	ClearBiTree(&BT)
5)判断是否为空	BiTreeEmpty(BT)
6)返回树的深度	BiTreeDepth(BT)
7)求根函数	Root(BT)
8)返回结点值	Value(BT, e)
9)给结点赋值	Assign(BT, &e, value)

二叉树的基本操作

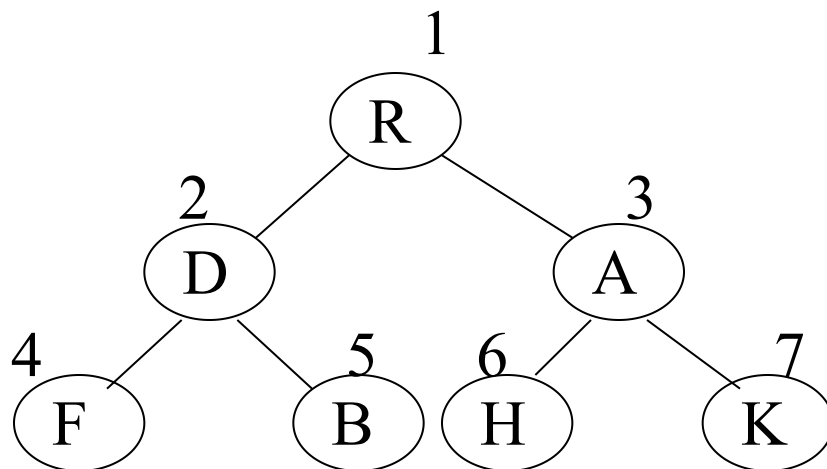
- 10)求孩子结点函数 LelfChild(BT,x) / RightChild(BT,x)
- 11)求兄弟函数 LelfSibling(BT,x) / RightSibling(BT,x)
- 12)插入子树操作 InsertChild(&BT,y,x) /
 InsertRChild(&BT,y,x)
- 13)删除子树操作 DeleteLChild(&BT,x) /
 DeleteRChild(&BT,x)
- 14)先序遍历操作 PreOrderTraverseTree(BT, Visit())
- 15)中序遍历操作 InOrderTraverseTree(BT, Visit())
- 16)后序遍历操作 PostOrderTraverseTree(BT, Visit())
- 17)层次遍历操作 LevelOrderTraverseTree(BT, Visit())

二叉树的存储结构:

- 顺序存储
- 链式存储

二叉树的顺序存储

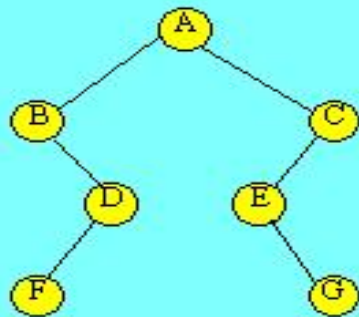
所谓二叉树的顺序存储，是用一组连续的存储单元存放二叉树中的结点。一般是按照二叉树结点从上至下、从左到右的顺序存储。



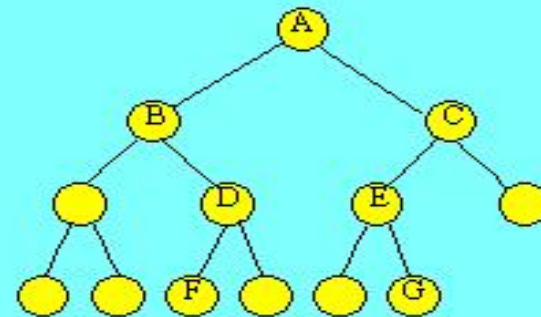
顺序存储

1	2	3	4	5	6	7				
R	D	A	F	B	H	K				

- 对于一般的二叉树，如果仍按从上至下和从左到右的顺序将树中的结点顺序存储在一维数组中，则数组元素下标之间的关系不能够反映二叉树中结点之间的逻辑关系，只有增添一些并不存在的空结点，使之成为一棵完全二叉树的形式，然后再用一维数组顺序存储。



(a) 一棵二叉树



(b) 改造后的完全二叉树

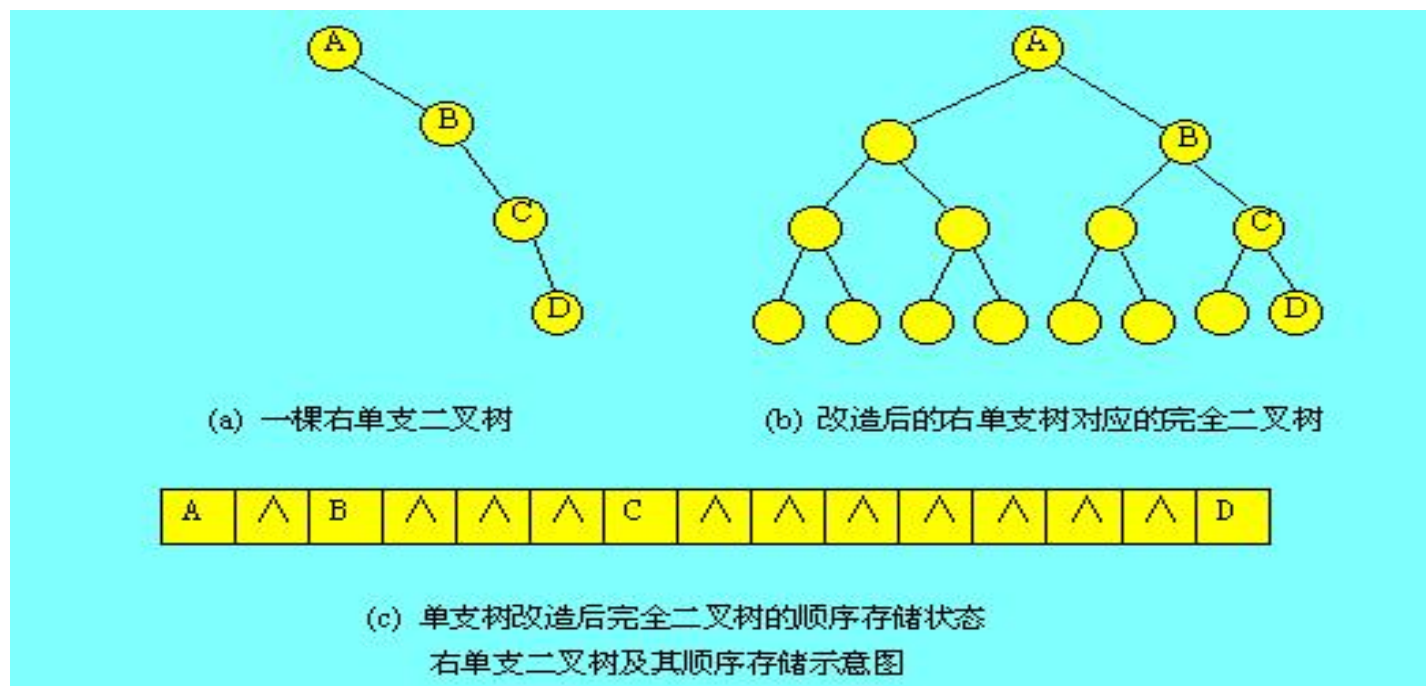
A	B	C	^	D	E	^	^	^	F	^	^	G
---	---	---	---	---	---	---	---	---	---	---	---	---

(c) 改造后完全二叉树顺序存储状态

一般二叉树及其顺序存储示意图

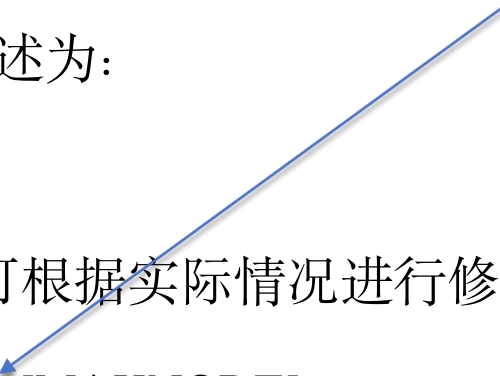
显然，**完全二叉树和满二叉树采用顺序存储比较合适**。对于需增加许多空结点才能将一棵二叉树改造成为一棵完全二叉树的存储时，会造成空间的大量浪费，不宜用顺序存储结构。

极端情况是单支树，如一棵深度为 k 的右单支树，只有 k 个结点，却需分配 $2^k - 1$ 个存储单元。



二叉树的顺序存储结构可描述为：

定义了一个数组类型



```
#define MAXNODE 1024
```

```
//二叉树的最大结点数，可根据实际情况进行修改
```

```
typedef TElemType SqBiTree[MAXNODE];
```

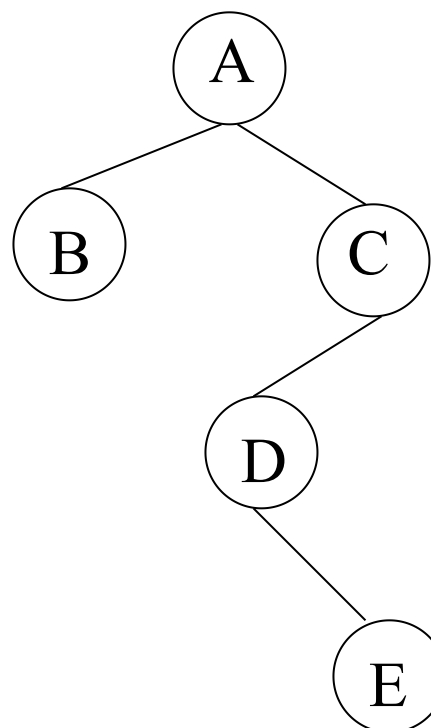
```
//0号单元存放根结点
```

定义一个顺序存储的二叉树变量： SqBiTree bt;

即将bt为含有MAXNODE个TElemType类型元素的一维数组。

[按完全二叉树编号存放]

A	B	C	...	D	...	E
1	2	3		6		13



其它顺序存储方案

	1	2	3	4	5
data	A	B	C	D	E
lc	2	0	4	0	0
rc	3	0	0	0	5

[存储结点数据和左、右孩子在向量中的序号]

	1	2	3	4	5
data	A	B	C	D	E
parent	0	1	-1	3	-4

[存储结点数据和其父结点的序号]

6.2.3.2 链式存储结构

用链来指示元素之间的逻辑关系，通常有**二叉链表**存储和**三叉链表**存储

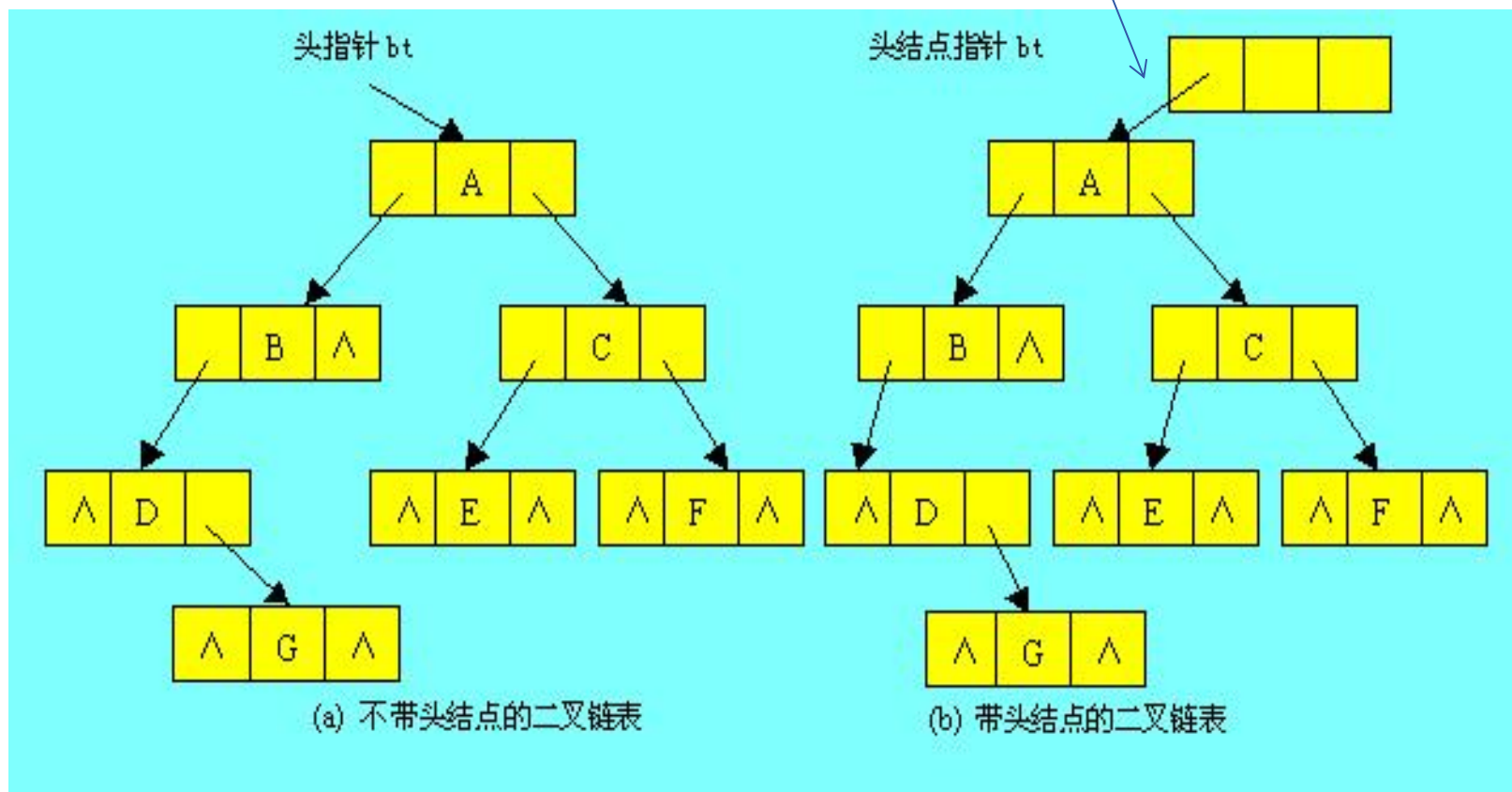
(1) 二叉链表存储

链表中每个结点有3个域组成，除了数据域外，还有两个指针域，分别给出该结点左孩子和右孩子所在的链结点的存储地址。



根结点的地址存放在头结点的左孩子指针域

下图(a)给出一棵二叉树的二叉链表存储表示。二叉链表也可以带头结点的方式存放，如图(b)所示。



二叉树的二叉链表存储结构可描述为:

```
typedef struct BiTNode
```

```
{TElemType data;
```

```
    struct BiTNode *lchild;*rchild;    //左右孩子指针
```

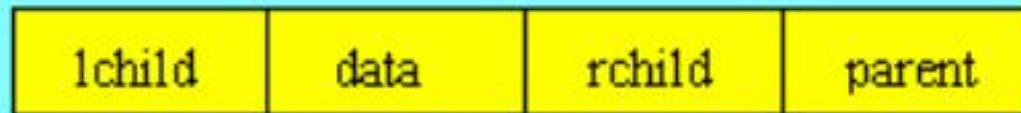
```
} BiTNode, *BiTree;
```

定义一个指向二叉树的指针变量: BiTree t;

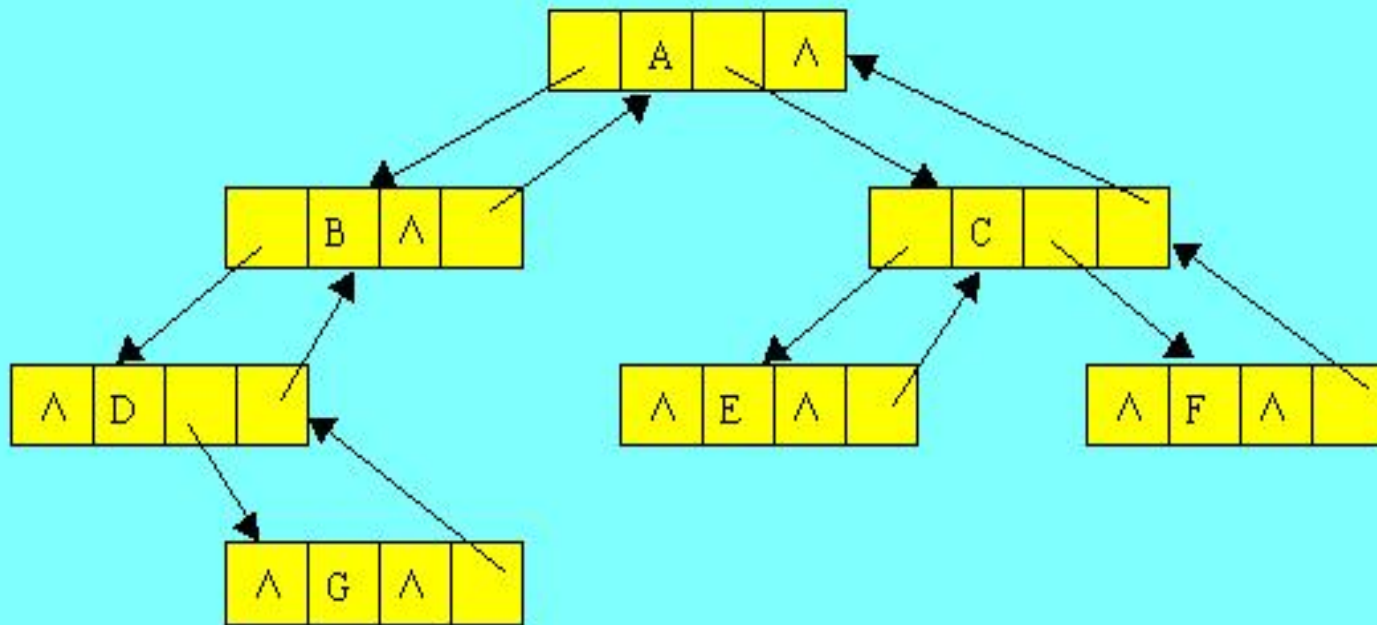
(2) 三叉链表存储

二叉链表存储可以直接找到结点的孩子结点，但不能直接找到其双亲结点，用三叉链表存储可以解决这一问题，给运算带来方便

每个结点由四个域组成，具体结构为：



下图给出一棵二叉树的三叉链表存储示意图。



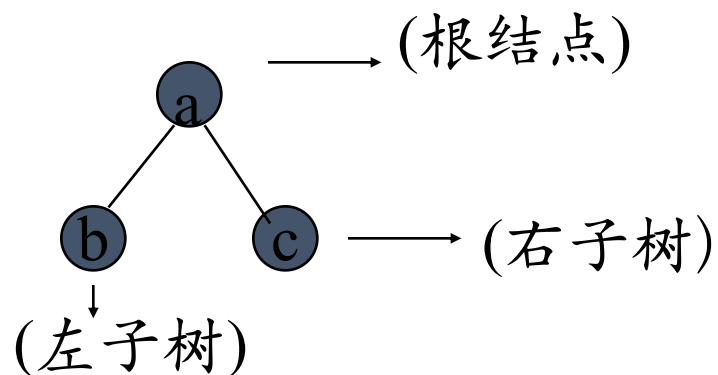
一棵二叉树的三叉链表表示示意图

6.3.1 遍历二叉树

6.3.1 遍历二叉树

在二叉树的一些应用中，常常要求在树中查找具有某种特征的结点，或者对树中全部结点逐一进行某种处理。这就引入了遍历二叉树的问题，即如何按某条搜索路径巡访树中的每一个结点，使得每一个结点均被访问一次，而且仅被访问一次。

遍历对线性结构是容易解决的，而二叉树是非线性的，因而需要寻找一种规律，以便使二叉树上的结点能排列在一个线性队列上，从而便于遍历。



由二叉树的递归定义，二叉树的三个基本组成单元是：根结点、左子树和右子树。

假如以L、D、R分别表示遍历左子树、遍历根结点和遍历右子树，遍历整个二叉树则有DLR、LDR、LRD、DRL、RDL、RLD六种遍历方案。若规定先左后右，则只有前三种情况，分别规定为：

DLR——先（根）序遍历，

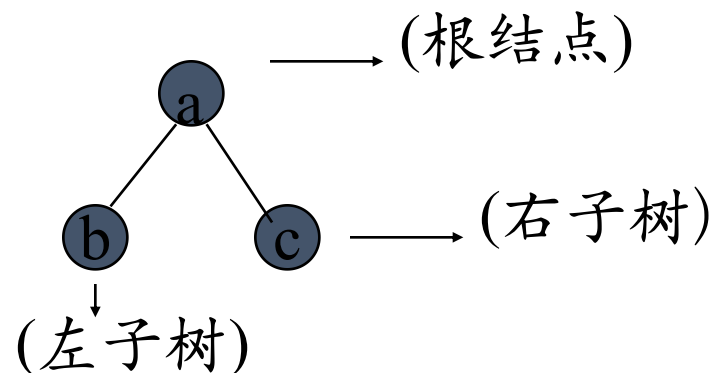
LDR——中（根）序遍历，

LRD——后（根）序遍历。

1、先序遍历二叉树的操作定义为：

若二叉树为空，则空操作；否则

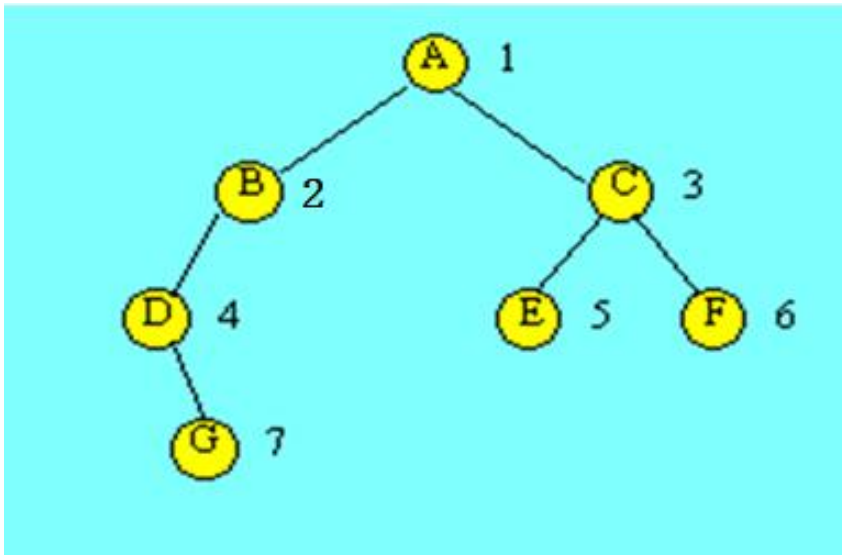
- (1) 访问根结点；
- (2) 先序遍历左子树；
- (3) 先序遍历右子树。



```

void PreOrderTraverse(BiTree T, Status(*Visit)(datatype))
{
    /* 初始条件：二叉树T存在, Visit是对结点操作的应用函数。 */
    /* 操作结果：先序递归遍历T, 对每个结点调用函数Visit一次且仅一次 */
    if(T) //T为空时递归调用结束
    {
        Visit(T->data); //访问根结点
        PreOrderTraverse(T->lchild, Visit); // 再先序遍历左子树
        PreOrderTraverse(T->rchild, Visit); //最后先序遍历右子树
    }
}

```



先序遍历所得到的结点序列为:

A B D G C E F

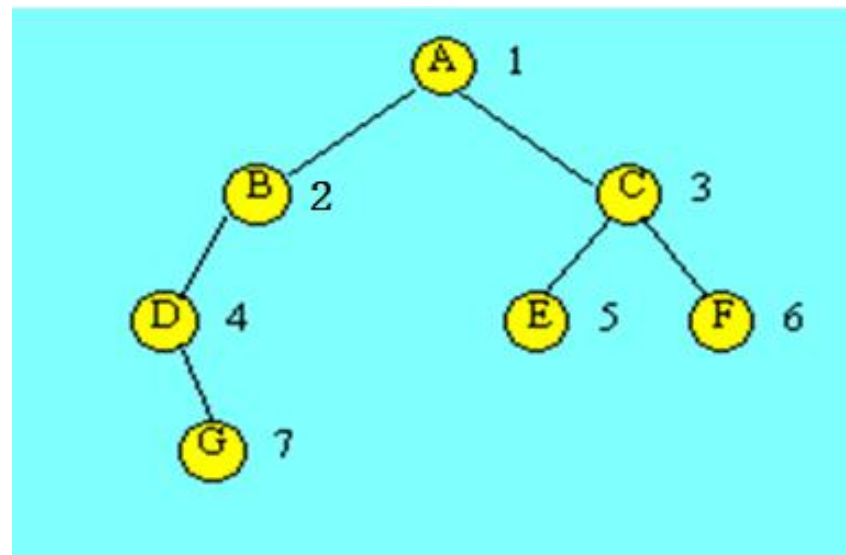
2、中序遍历二叉树的操作定义为：

若二叉树为空，则空操作；否则

(1) 中序遍历左子树；

(2) 访问根结点；

(3) 中序遍历右子树。



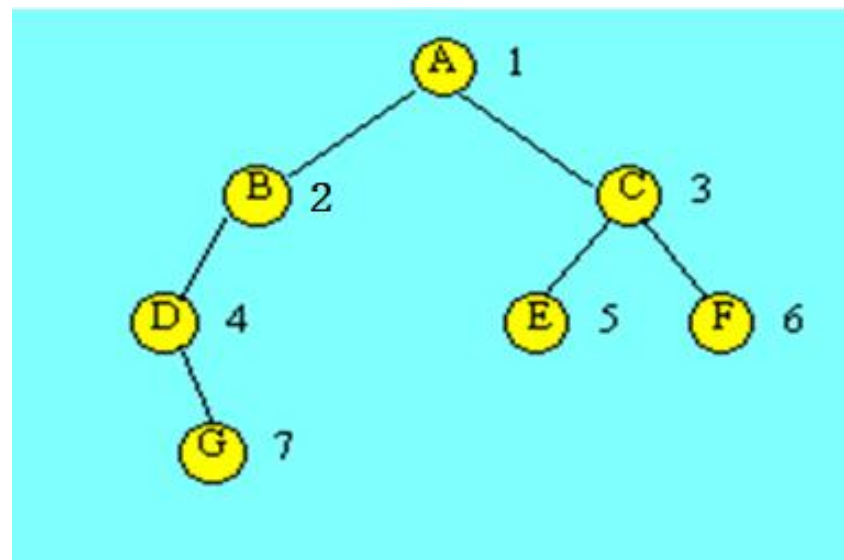
中序： D G B A E C F

```
void InOrderTraverse(BiTree T, Status(*Visit)(datatype))
{
    /* 初始条件：二叉树T存在, Visit是对结点操作的应用函数。 */
    /* 操作结果：中序递归遍历T, 对每个结点调用函数Visit一次且仅一次 */
    if(T) //T为空时递归调用结束
    {
        InOrderTraverse(T->lchild, Visit); // 先中序遍历左子树
        Visit(T->data); //访问根结点
        InOrderTraverse(T->rchild, Visit); //最后中序遍历右子树
    }
}
```

3、后序遍历二叉树的操作定义为:

若二叉树为空, 则空操作; 否则

- (1) 后序遍历左子树;
- (2) 后序遍历右子树;
- (3) 访问根结点。



后序: G D B E F C A

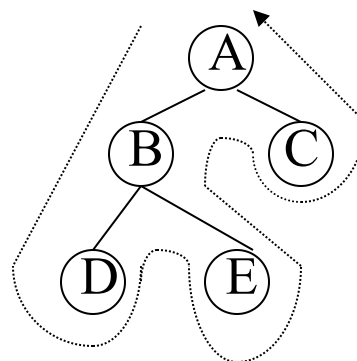
```
void PostOrderTraverse(BiTree T, Status(*Visit)(datatype))
{
    /* 初始条件: 二叉树T存在, Visit是对结点操作的应用函数。 */
    /* 操作结果: 后序递归遍历T, 对每个结点调用函数Visit一次且仅一次 */
    if(T) //T为空时递归调用结束
    {
        PostOrderTraverse(T->lchild, Visit); // 先后序遍历左子树
        PostOrderTraverse(T->rchild, Visit); // 然后后序遍历右子树
        Visit(T->data); // 最后访问根结点
    }
}
```

[示例]

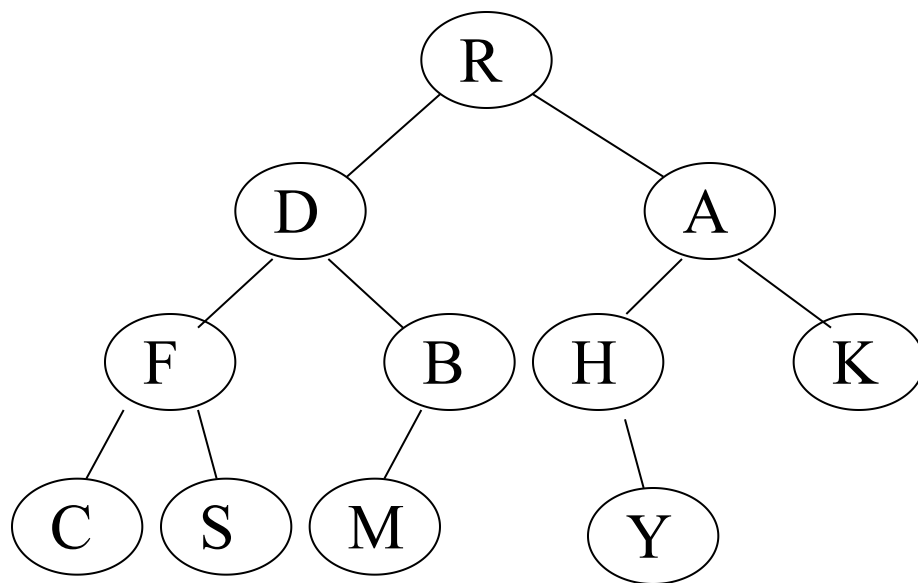
先序遍历: ABDEC

中序遍历: DBEAC

后序遍历: DEBCA

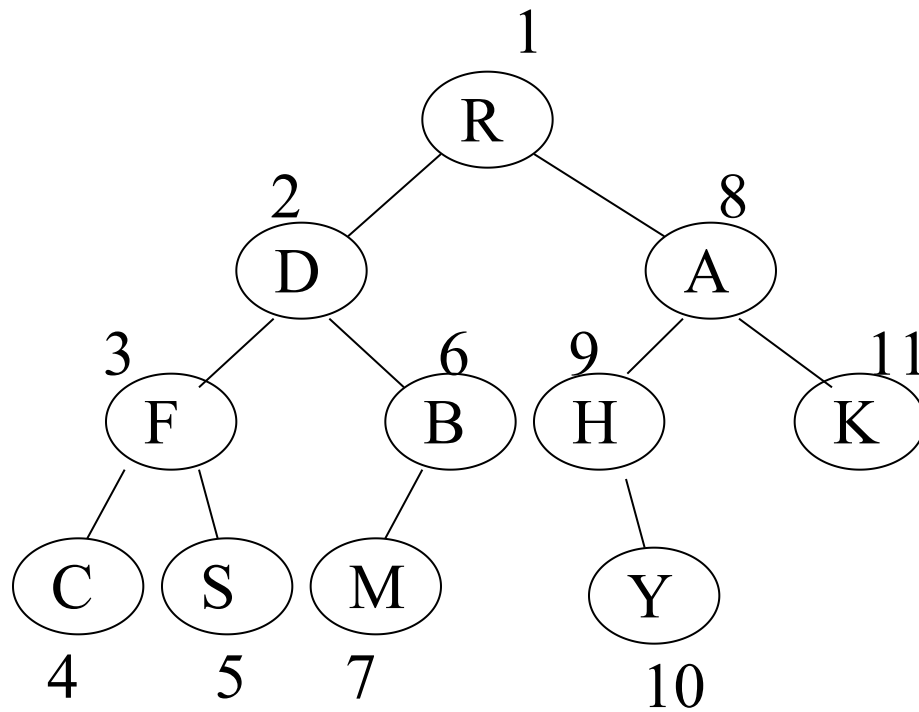


二叉树的遍历示例：



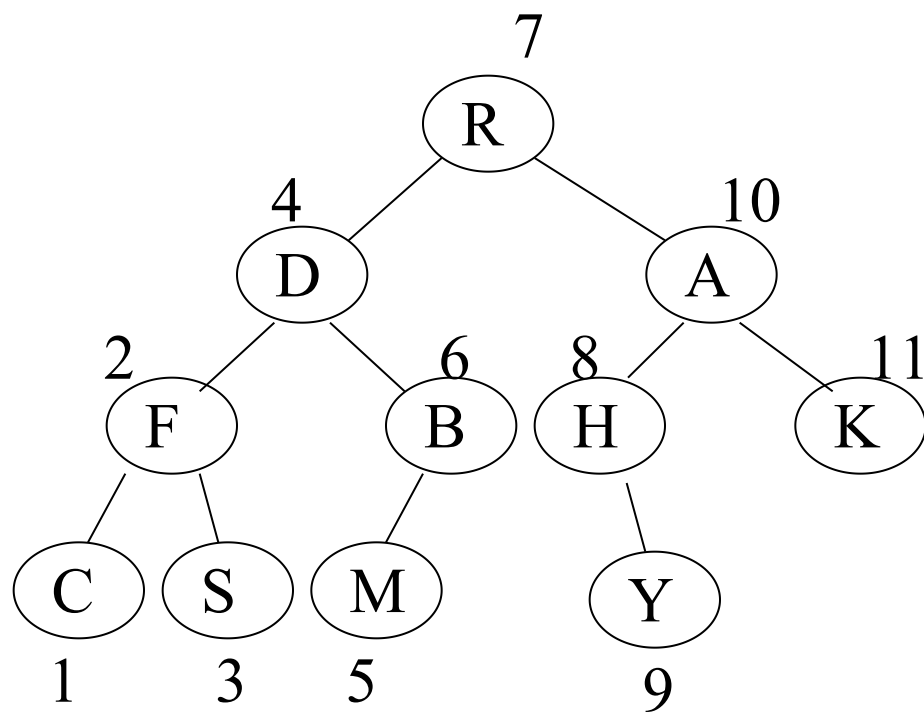
二叉树的先序遍历

DLR:



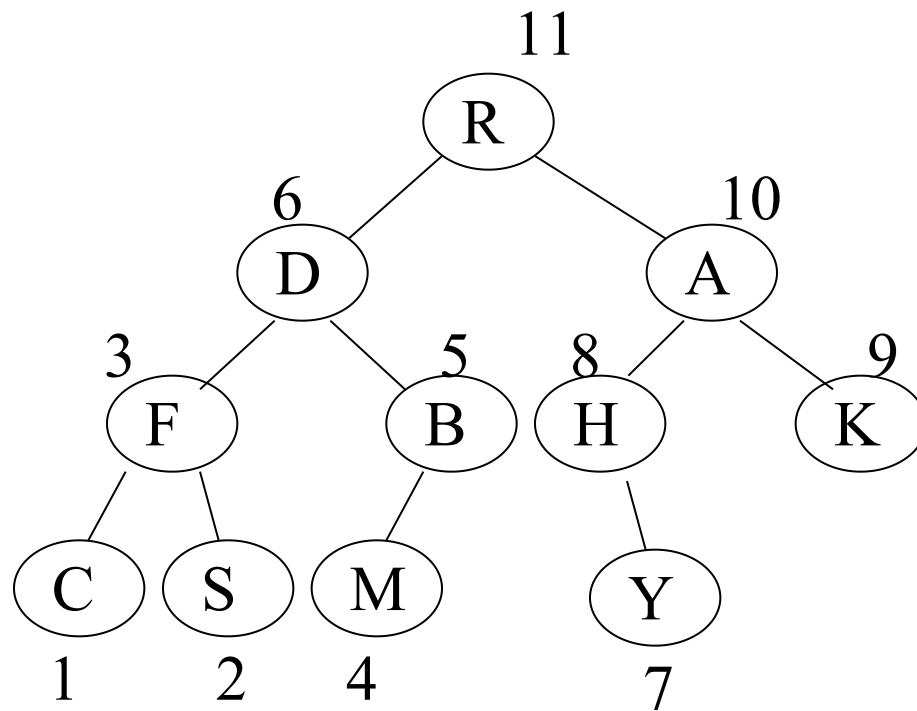
二叉树的中序遍历

LDR:

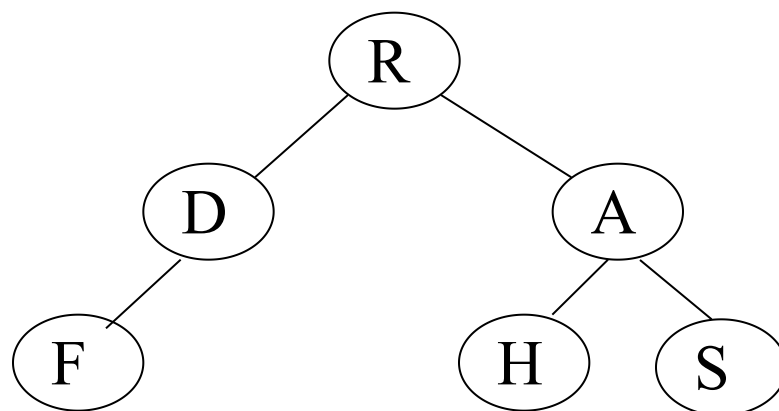


二叉树的后序遍历

LRD:



二叉树的遍历



DLR: R D F A H S

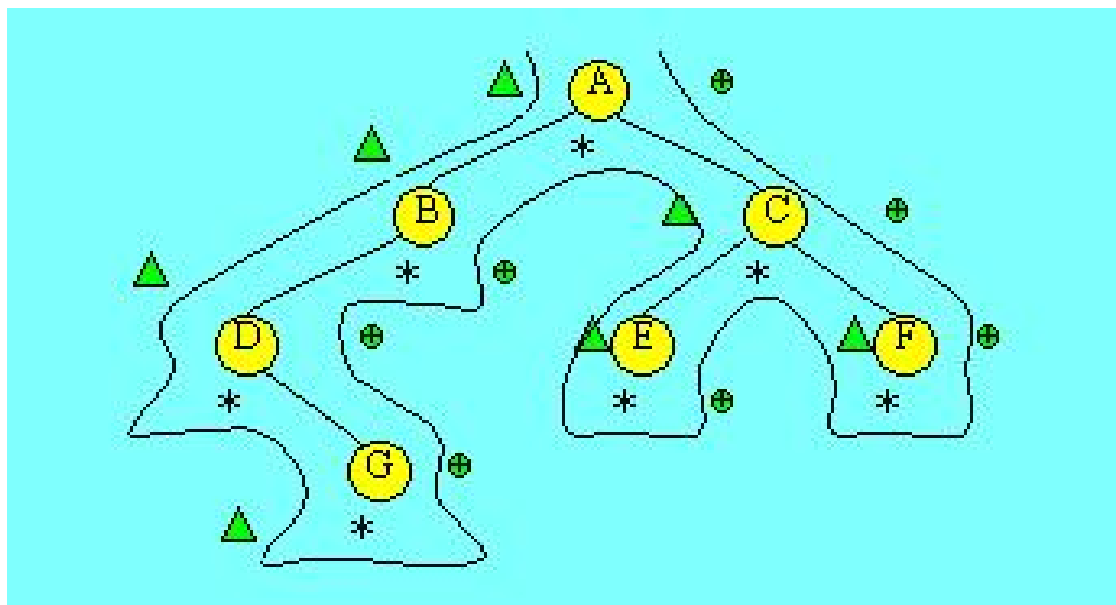
L**D**R: F D R H A S

LR**D**: F D H S A R

非递归方法实现二叉树的遍历

1、包络线

下图中所示的从根结点左外侧开始，由根结点右外侧结束的曲线，称其为**该二叉树的包络线**，二叉树的遍历是沿着该线路进行的。沿着该路线按 $\triangle$ 标记的结点读得的序列为先序序列，按 $*$ 标记读得的序列为中序序列，按 $\oplus$ 标记读得的序列为后序序列。

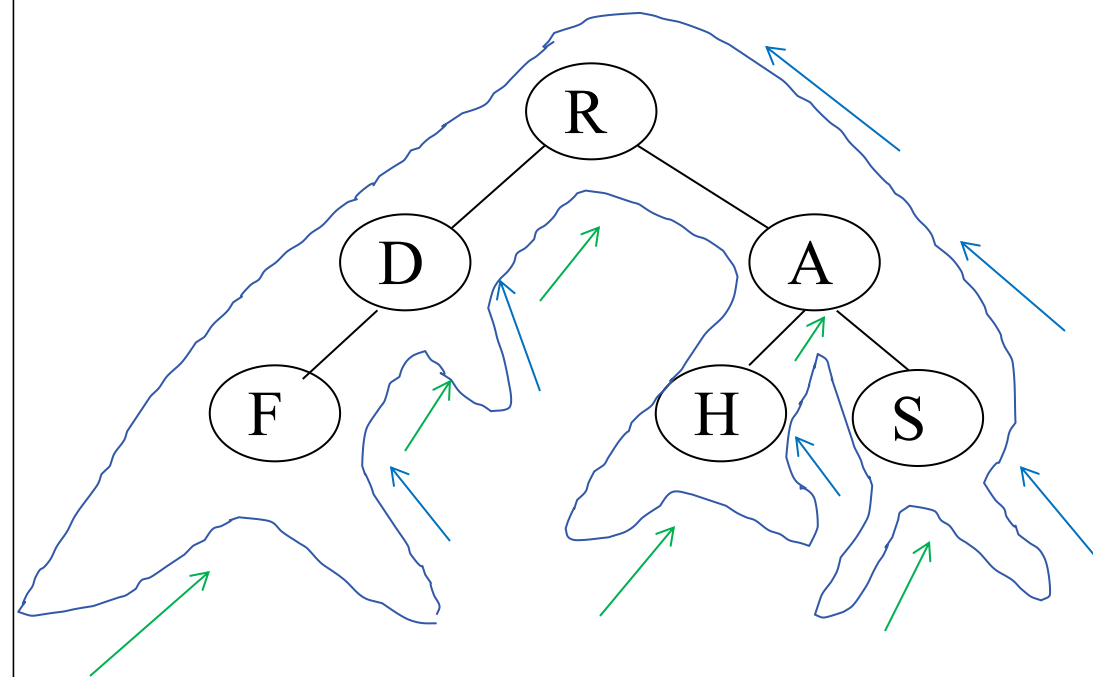


2、栈的使用

先序遍历是沿着包络线深入时遇到结点就访问；

中序遍历是在从左子树返回时遇到结点访问；

后序遍历是在从右子树返回时遇到结点访问。



DLR: R D F A H S

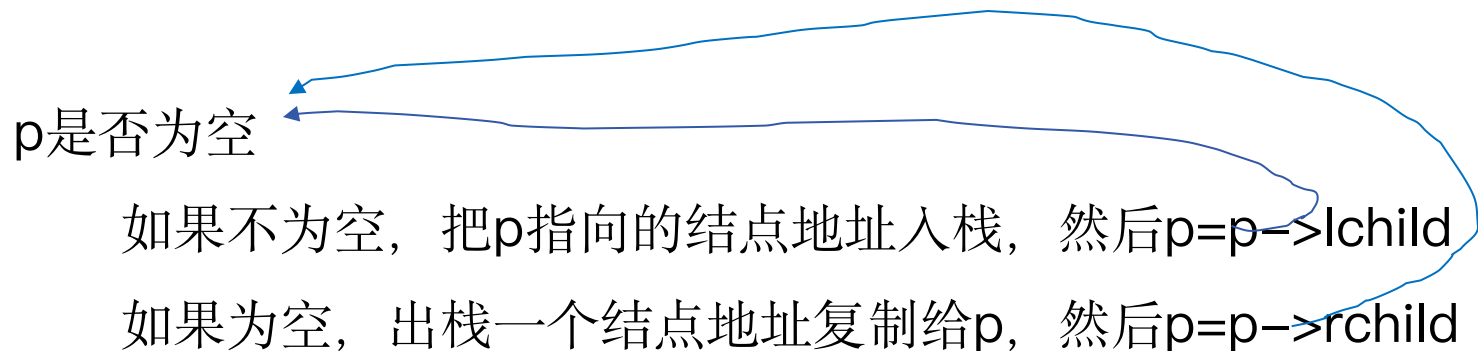
L**D**R: F D R H A S

LR**D**: F D H S A R

按包络线行走的核心思路:

从根结点开始沿左子树深入下去, 当深入到最左端, 无法再深入下去时, 则返回, 再逐一进入刚才深入时遇到结点的右子树, 再进行如此的深入和返回, 直到最后从根结点的右子树返回到根结点为止。

在这一过程中, 返回结点的顺序如深入结点的顺序想法, 即后深入先返回。符合“**栈**”结构的特点。



如果结点地址**入栈前访问**该结点, 则为**先序**遍历

如果结点地址**出栈后访问**该结点, 则为**中序**遍历

//中序遍历的非递归算法

Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){

InitStack(S); p=T; //p指向根结点

while(p || !StackEmpty(S)){ //p不为空或者栈不为空

if(p){

Push(S,p); // p不为空入栈p的值

p=p->lchild; // p指向之前指向结点的左孩子

}else{ //p值为null, 已经到了左下角

Pop(S,p); //出栈, 将值赋给p

if(!Visit(p->data)) return ERROR; //访问p指向的结点

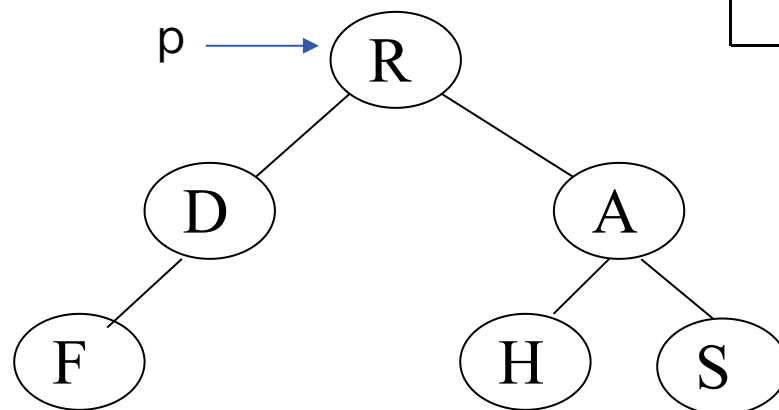
p=p->rchild;

}//else

}//while

return OK;

}



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

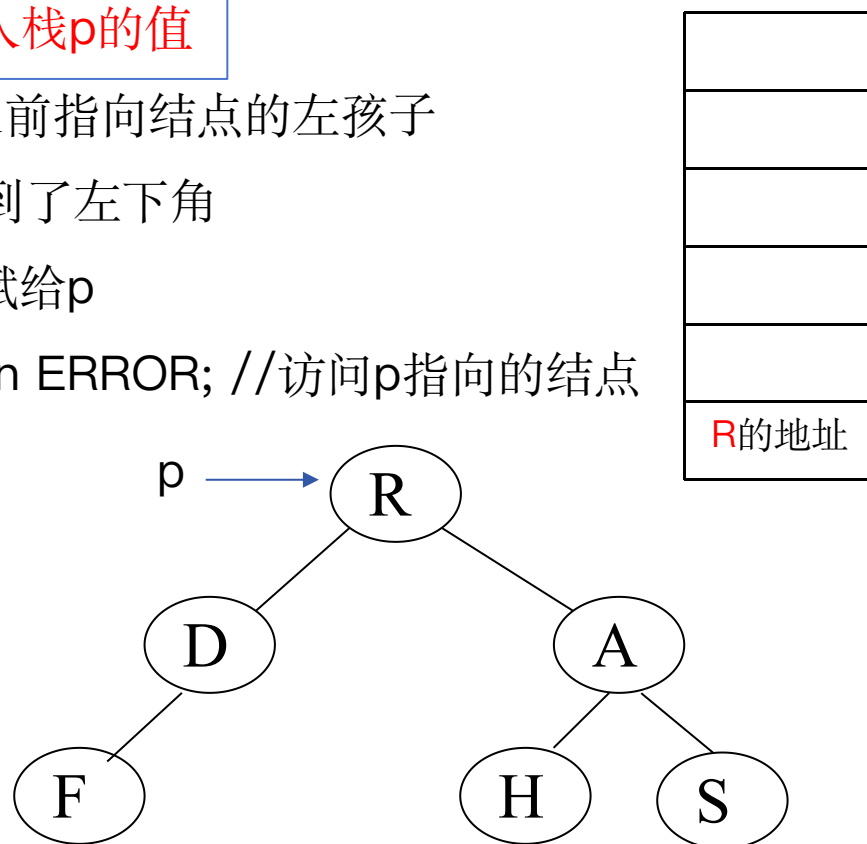
```
            p=p->rchild;
```

```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

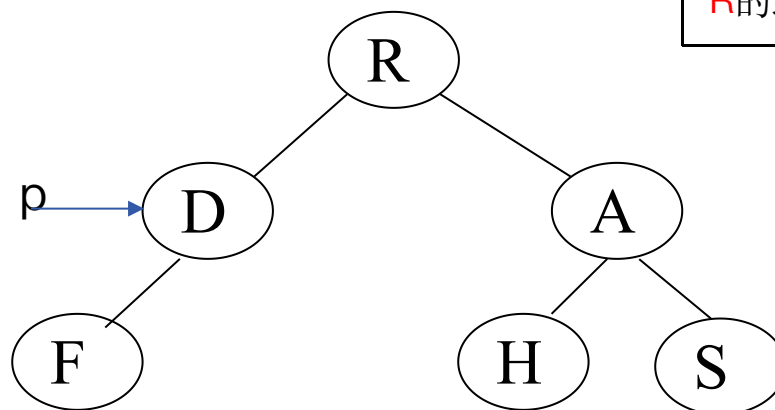
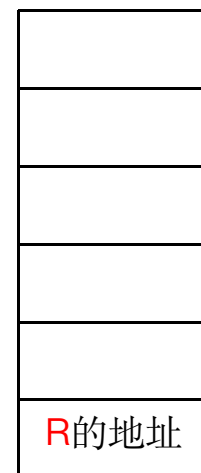
```
            p=p->rchild;
```

```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

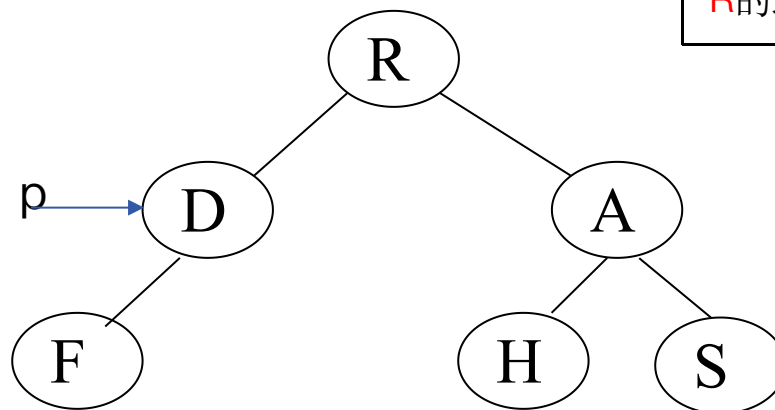
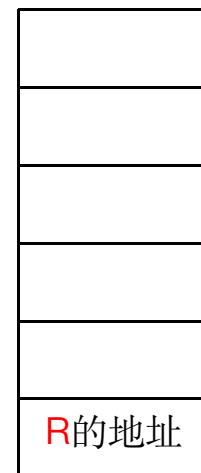
```
            p=p->rchild;
```

```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

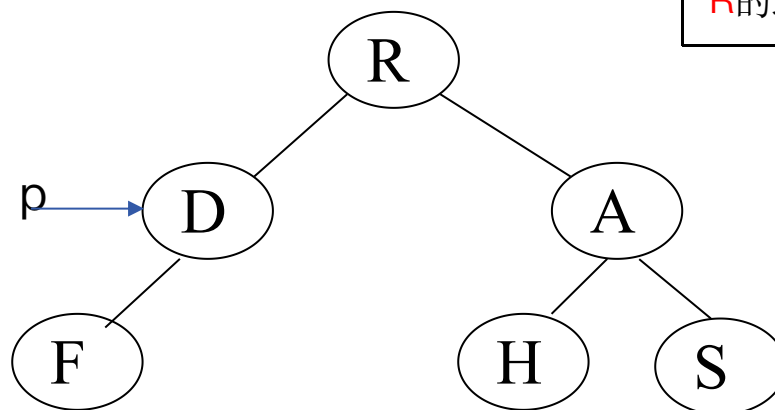
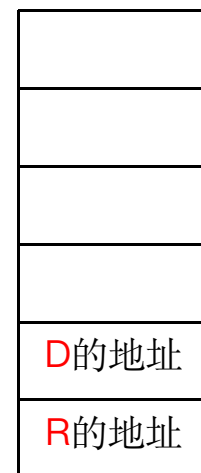
```
            p=p->rchild;
```

```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

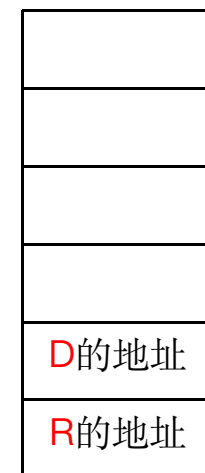
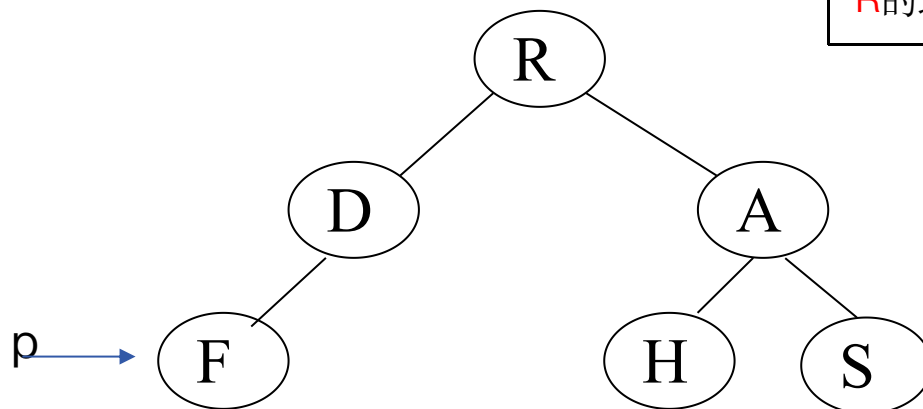
```
            p=p->rchild;
```

```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

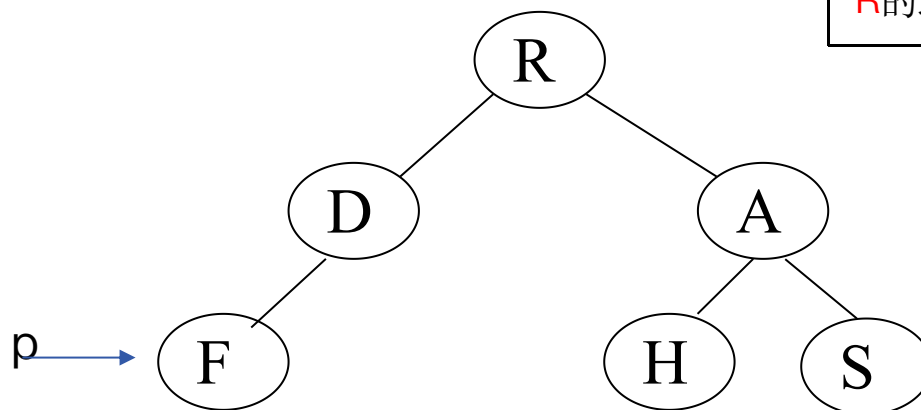
```
            p=p->rchild;
```

```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

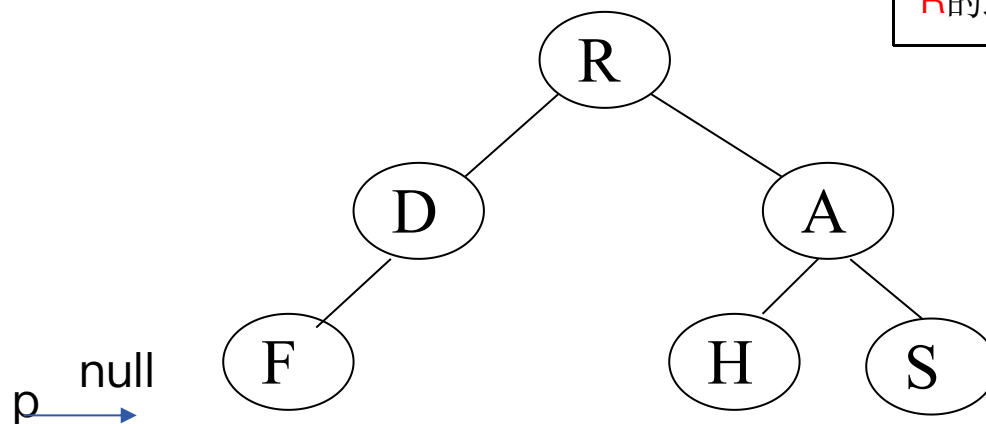
```
            p=p->rchild;
```

```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

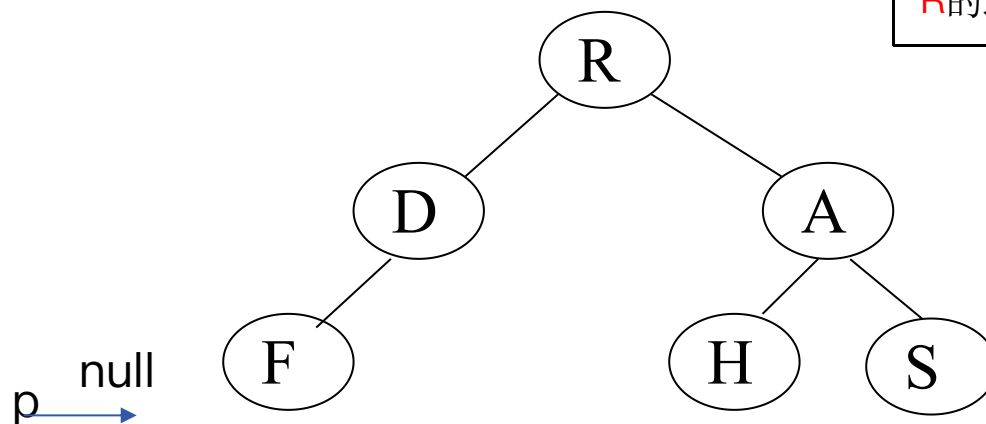
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

F的地址
D的地址
R的地址



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

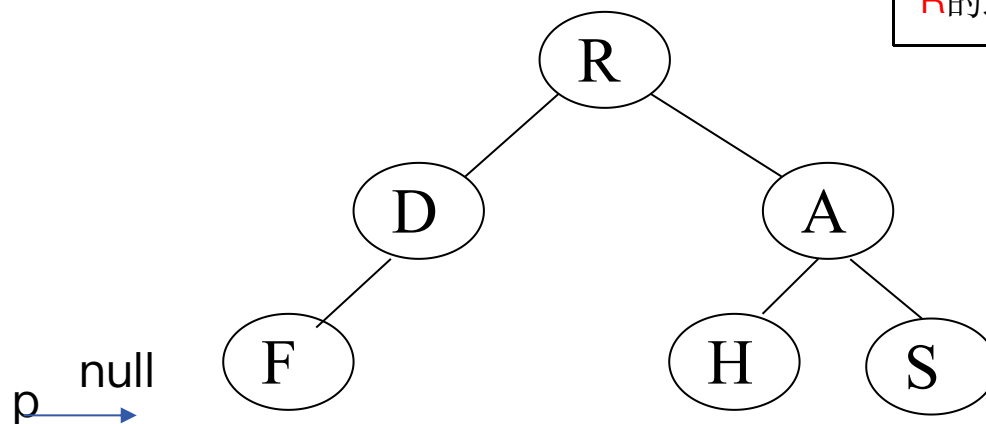
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

F的地址
D的地址
R的地址



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

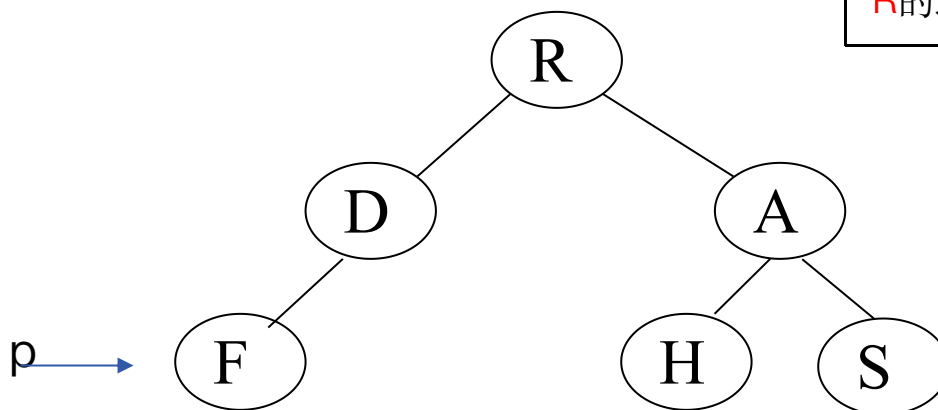
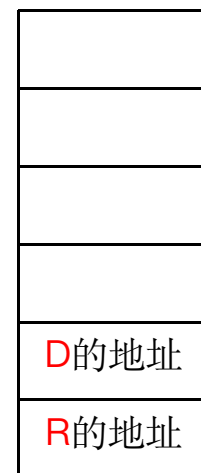
```
            p=p->rchild;
```

```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

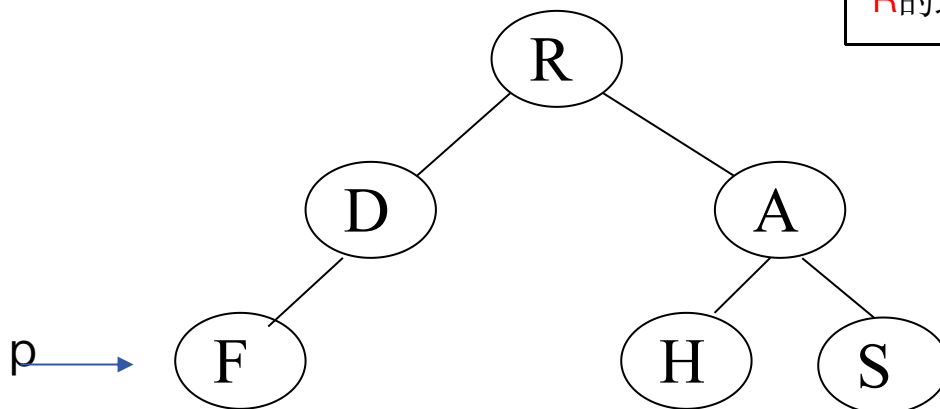
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F



D的地址
R的地址

//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

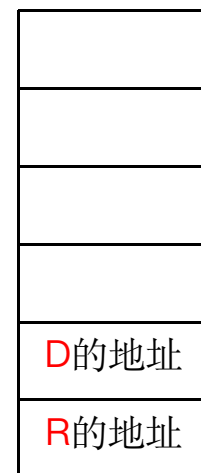
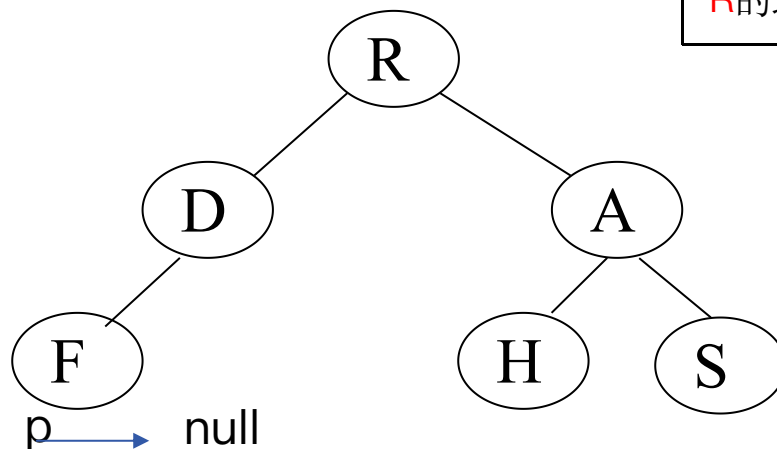
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

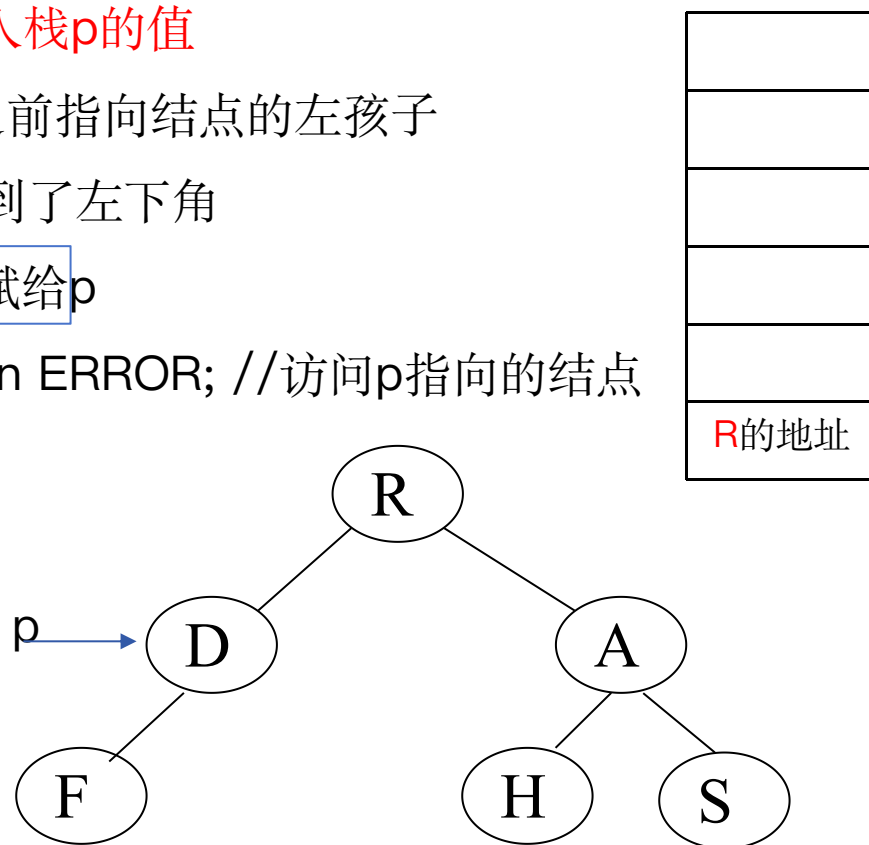
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

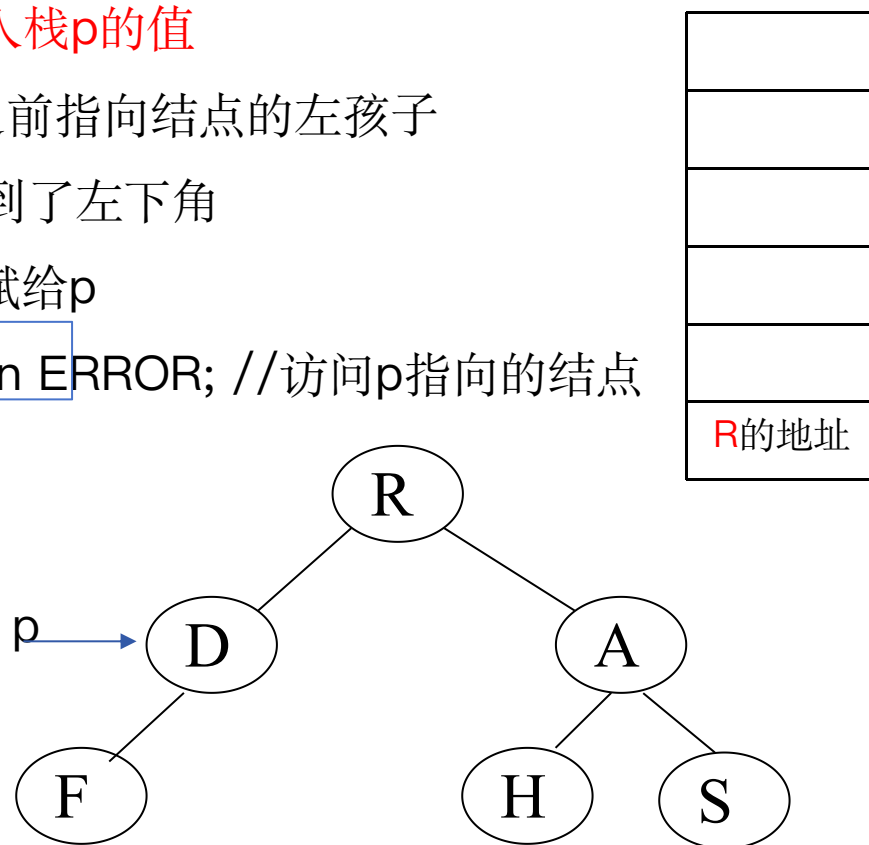
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

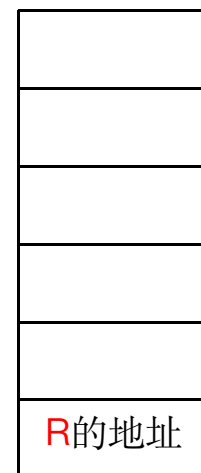
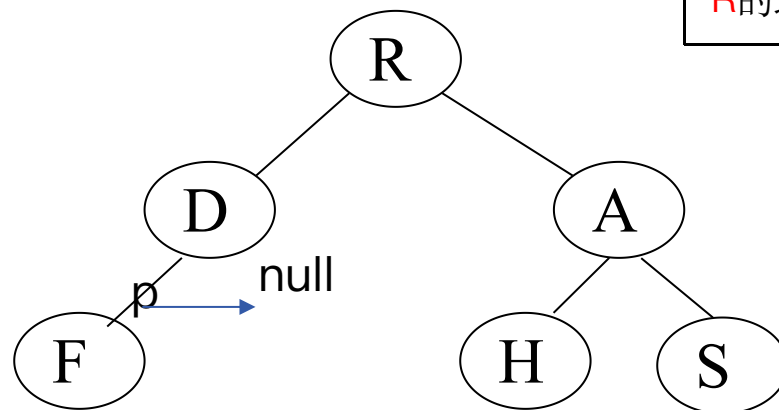
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

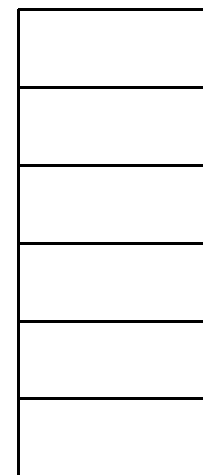
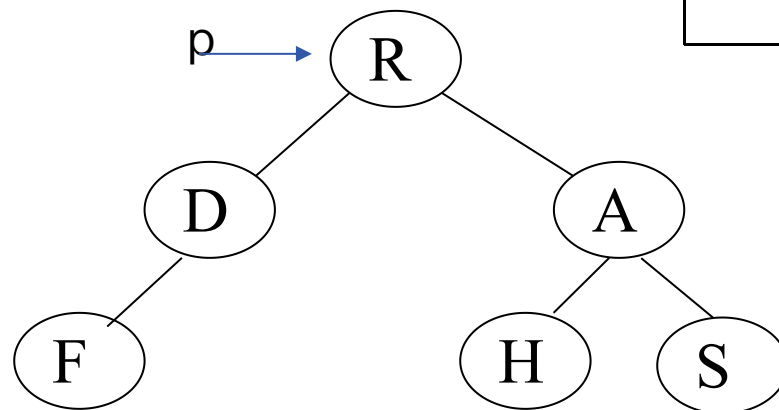
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

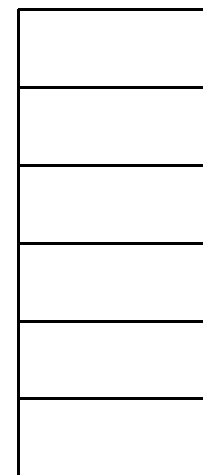
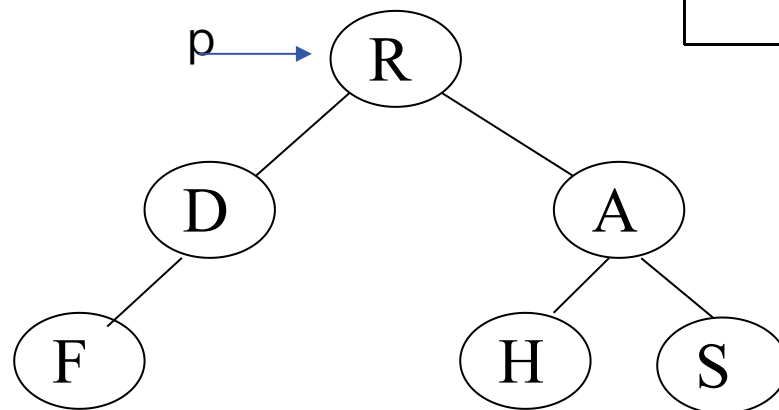
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D、R



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

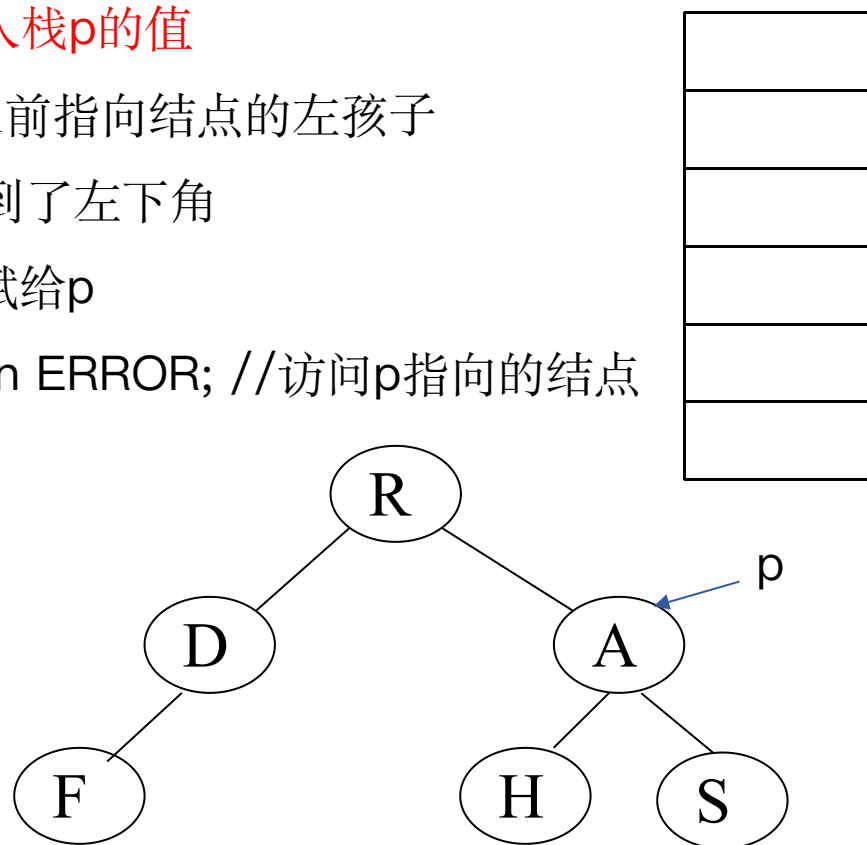
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D、R



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

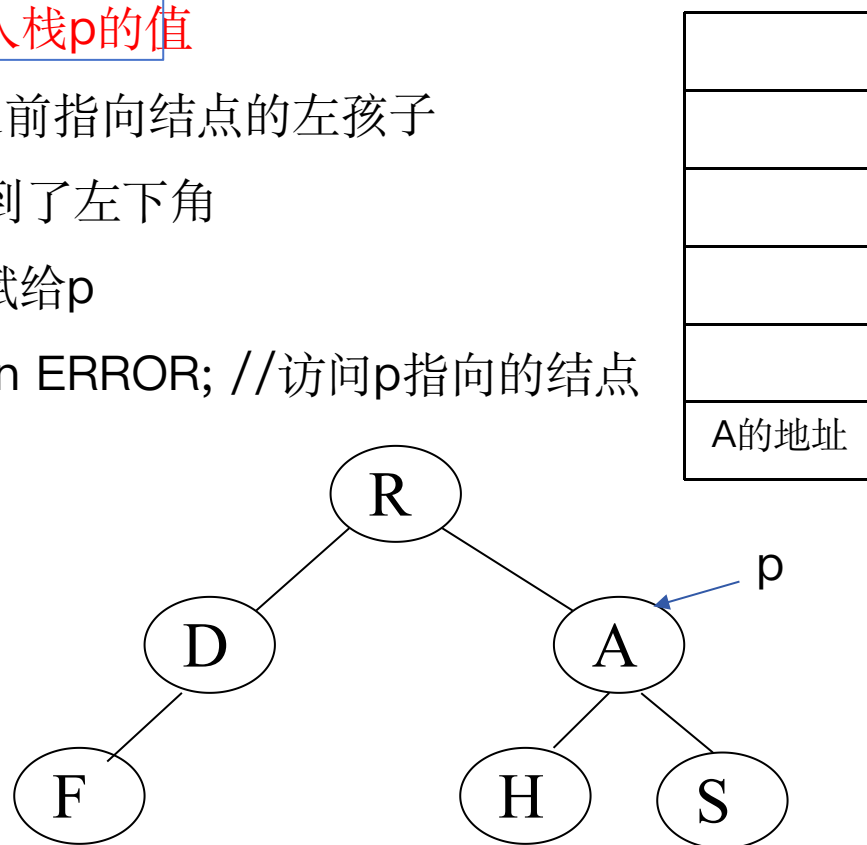
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D、R



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

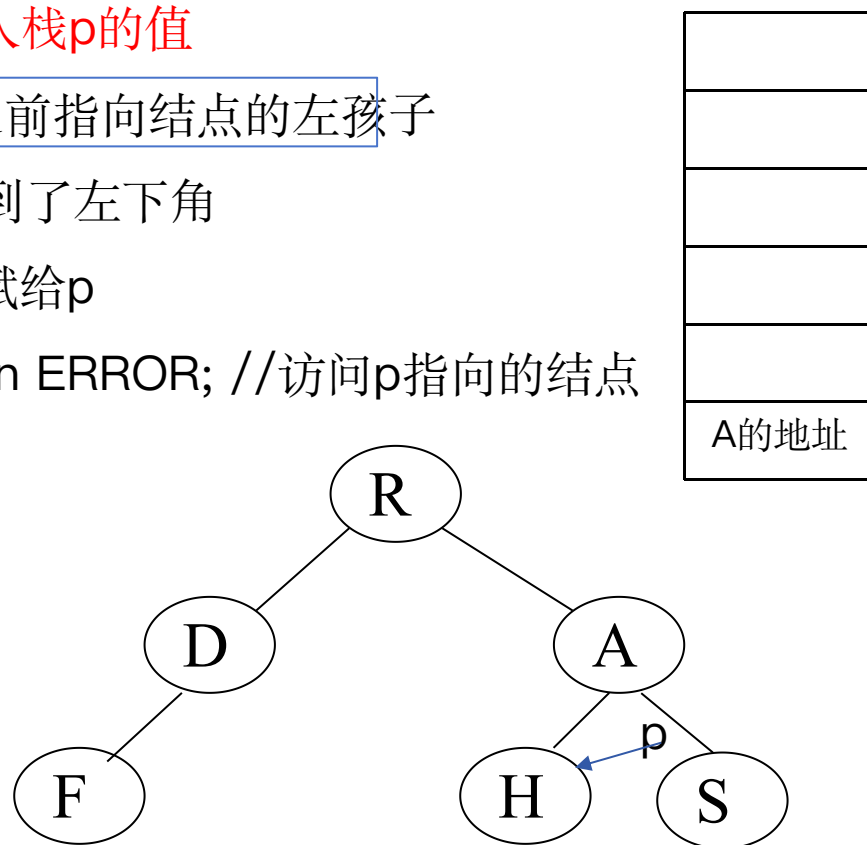
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D、R



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

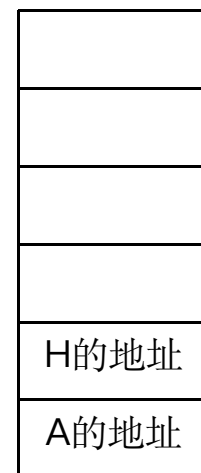
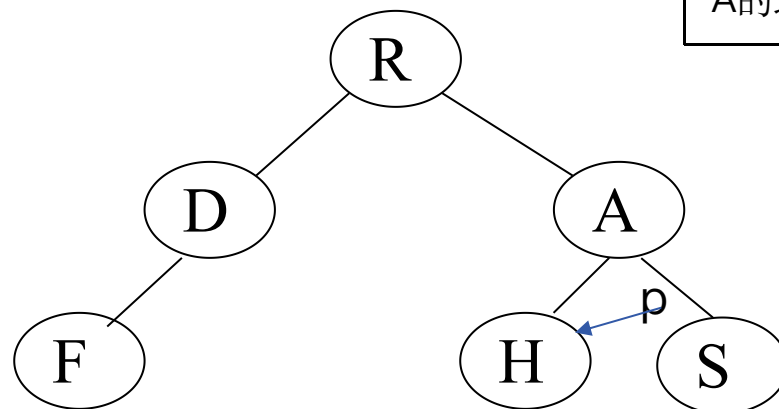
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D、R



//中序遍历的非递归算法

```
Status InOrderTraverse (BiTree T, Status(* Visit)(TElemType e)){
```

```
    InitStack(S); p=T; //p指向根结点
```

```
    while(p || !StackEmpty(S)){ //p不为空或者栈不为空
```

```
        if(p){
```

```
            Push(S,p); // p不为空入栈p的值
```

```
            p=p->lchild; // p指向之前指向结点的左孩子
```

```
        }else{ //p值为null, 已经到了左下角
```

```
            Pop(S,p); //出栈, 将值赋给p
```

```
            if(!Visit(p->data)) return ERROR; //访问p指向的结点
```

```
            p=p->rchild;
```

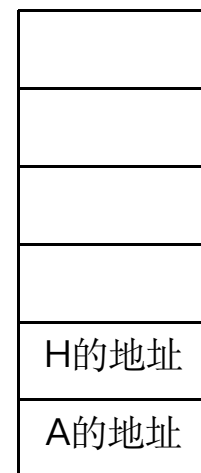
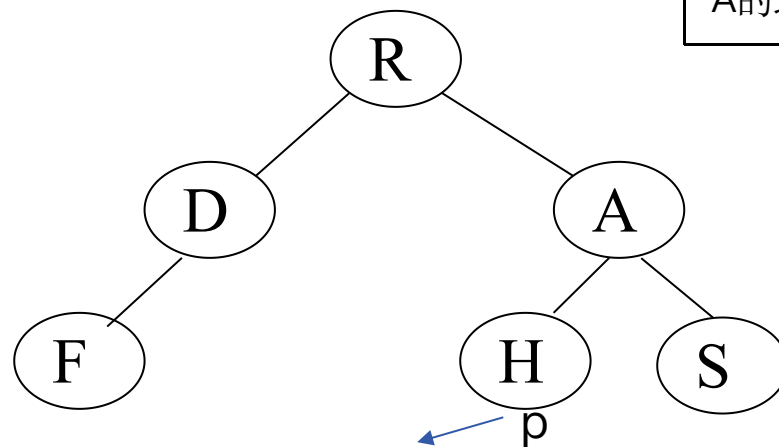
```
        } //else
```

```
    } //while
```

```
    return OK;
```

```
}
```

访问完成: F、D、R

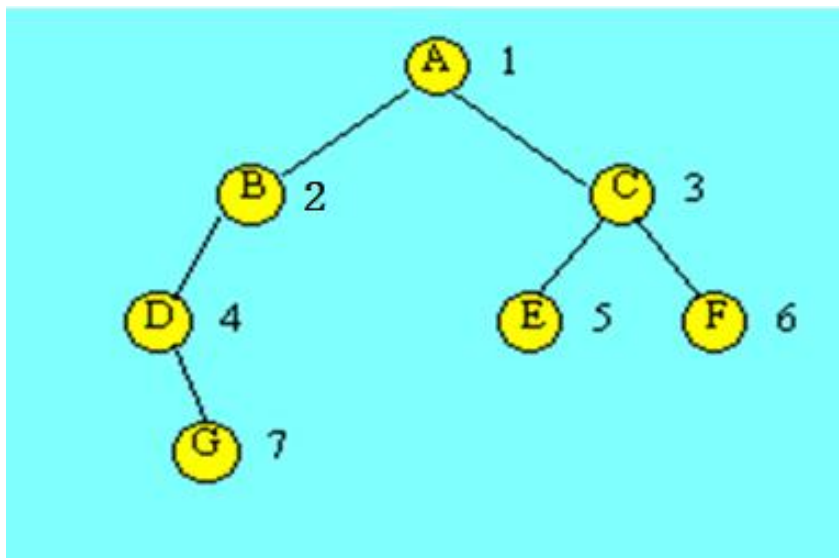


队列方法实现二叉树的层次遍历

所谓二叉树的层次遍历，是指从二叉树的第一层（根结点）开始，从上至下逐层遍历，在同一层中，则按从左到右的顺序对结点逐个访问。

对于下图所示的二叉树，按层次遍历所得到的结果序列为：

A B C D E F G



队列的使用

在进行层次遍历时，可设置一个队列结构，遍历从二叉树的根结点开始，首先将根结点指针入队列，然后从队头取出一个元素，每取一个元素，执行下面两个操作：

(1)访问该元素所指结点；

(2)若该元素所指结点的左、右孩子结点非空，则将该元素所指结点的左孩子指针和右孩子指针顺序入队。

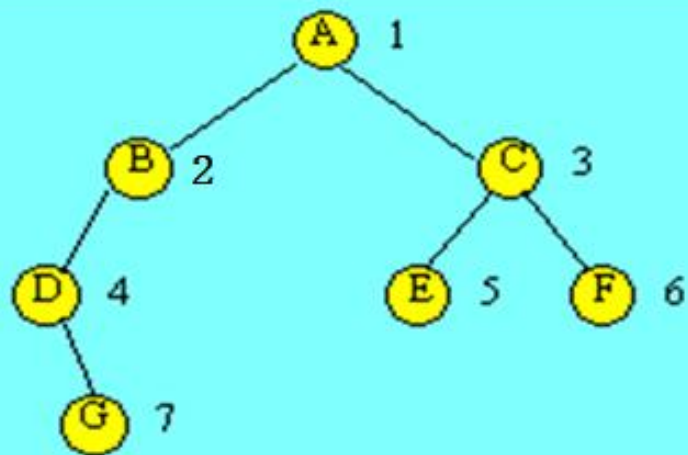
此过程不断进行，当队列为空时，二叉树的层次遍历结束。

层次遍历二叉树

```
Status LevelOrderTraverse(BiTree T,  
                           Status(* Visit)(TElemType e))  
{   if (T) {  
        InitQueue(Q);  
        EnQueue(Q, T);      //根结点的指针T入队列  
        while ( ! QueueEmpty (Q) )  
        {   DeQueue(Q, p); //从队头取一个指针  
            if ( !Visit(p->data) ) return ERROR;  
            if (p->lchild) EnQueue(Q, p->lchild);  
            if (p->rchild) EnQueue(Q, p->rchild);  
        }  
    }  
    return OK;  
} //LevelOrderTraverse
```

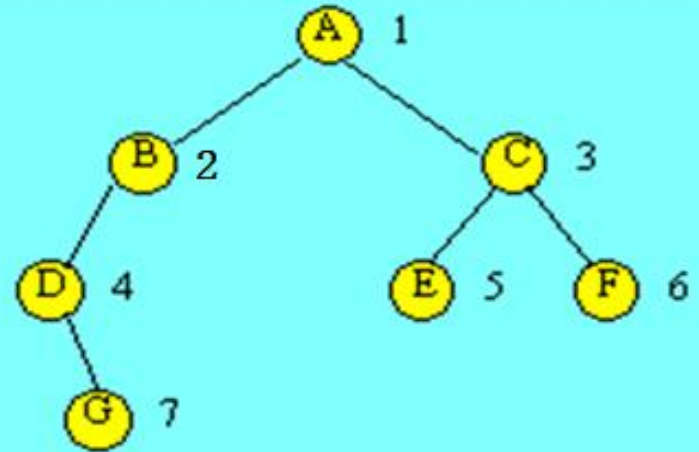
在二叉树中查找值为 x 的数据元素

```
BiTree Search(BiTree bt, datatype x){  
    BiTNode *p;  
    if(bt){  
        if(bt->data == x) return bt; //查找成功返回  
        p = Search(bt->lchild, x); //在左子树中查找  
        if(p) return p; //如果找到, 则返回  
        p = Search(bt->rchild, x); //在右子树中查找  
        return p;  
    }  
    return NULL; //查找失败返回  
}
```



统计给定二叉树中叶子结点的数目

```
int CountLeaf(BiTree bt){  
    if(bt == NULL) return 0; //若bt为空, 返回0  
    if(bt->lchild == NULL && bt->rchild == NULL)  
        return 1; //若bt为叶子结点, 返回1  
    return(CountLeaf(bt->lchild)+CountLeaf(bt->rchild))  
    //若bt为分支结点, 叶子数目等于左右子树中的叶子之和  
}
```



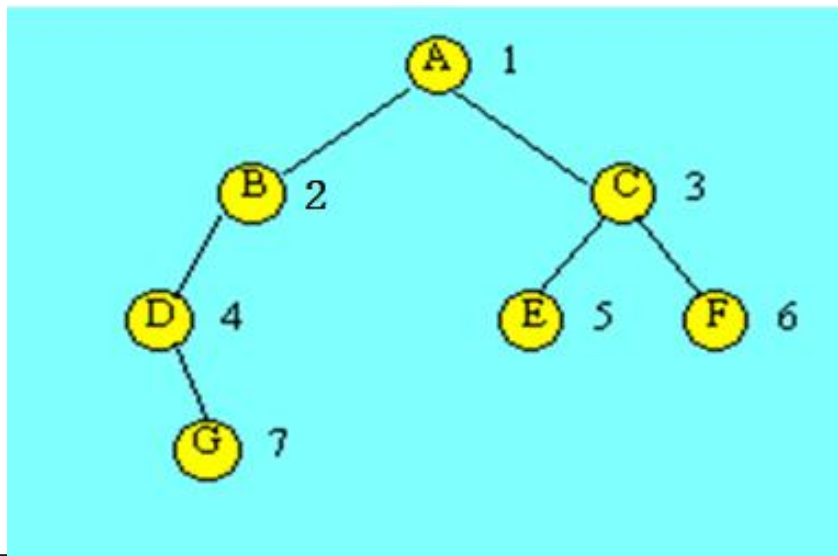
二叉树遍历的应用

构造二叉树的二叉链表存储

构建一棵二叉树的二叉链表也是基于遍历的过程进行的。这里按照先序遍历的过程构建。

首先建立二叉树**带空指针的先序序列**，依此作为构建时结点的输入顺序，如对于下图所示的二叉树，输入序列为：

ABD0G000CE00F00（**0表示空**，为了简化问题，设数据元素的类型为字符型）。



读入带空指针的先序序列构造二叉树，写法1，通过return返回二叉树指针

```
BiTree CreateBinTree(){
    //以带空指针的先序序列构造二叉链表存储的二叉树
    BiTree T;
    scanf("%c", &ch);
    if(ch == '0'){
        T = NULL; //读入0时，将相应指针置空
    }else{
        T = (BinTNode *)malloc(sizeof(BinTNode)); //生成结点空间
        T->data = ch;
        T->lchild = CreateBinTree(); //递归构造二叉树的左子树
        T->rchild = CreateBinTree(); //递归构造二叉树的右子树
    }
    return T;
}

main(){
    BiTree bt;
    bt = CreateBinTree();
}
```

读入带空指针的先序序列构造二叉树，写法2，通过引用返回二叉树指针

```
void CreateBinTree(BiTree &T){
    //以带空指针的先序序列构造二叉链表存储的二叉树
    char ch;
    scanf("%c", &ch);
    if(ch == '0'){
        T = NULL; //读入0时，将相应指针置空
    }else{
        T = (BinTNode *)malloc(sizeof(BinTNode)); //生成结点空间
        T->data = ch;
        CreateBinTree (T->lchild); //递归构造二叉树的左子树
        CreateBinTree (T->rchild); //递归构造二叉树的右子树
    }
}

main(){
    BiTree bt;
    CreateBinTree(bt);
    .....
}
```

读入带空指针的先序序列构造二叉树，写法3，传递指向指针的指针

```
void CreateBinTree(BiTree *p){  
    //以带空指针的先序序列构造二叉链表存储的二叉树  
    char ch;  
    scanf("%c", &ch);  
    if(ch == '0'){  
        *p = NULL; //读入0时，将相应指针置空  
    }else{  
        *p = (BinTNode *)malloc(sizeof(BinTNode)); //生成结点空间  
        *p->data = ch;  
        CreateBinTree (&(*p->lchild)); //递归构造二叉树的左子树  
        CreateBinTree (&(*p->rchild)); //递归构造二叉树的右子树  
    }  
}
```

由遍历序列恢复二叉树

从前面讨论的二叉树的遍历知道，任意一棵二叉树结点的先序序列和中序序列都是唯一的。

反过来，若已知结点的先序序列和中序序列，能否确定这棵二叉树呢？这样确定的二叉树是否是唯一的呢？

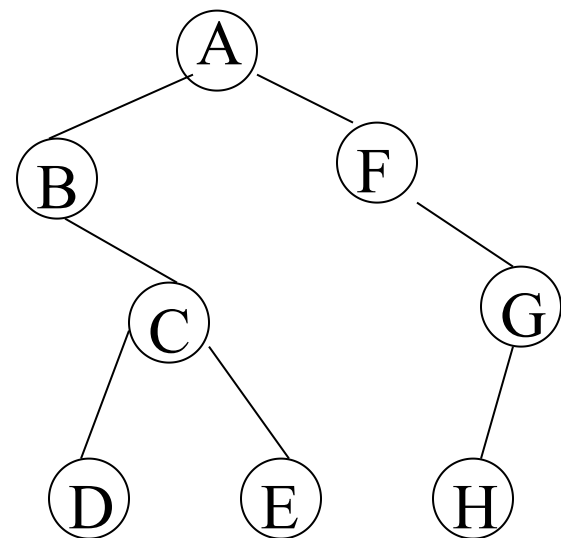
先序序列+中序序列

中序序列+后序序列

先序序列+后序序列 ✕

先序序列: **A**BCDEFGH

中序序列: BDC**E**AFHG



1、依据遍历定义：

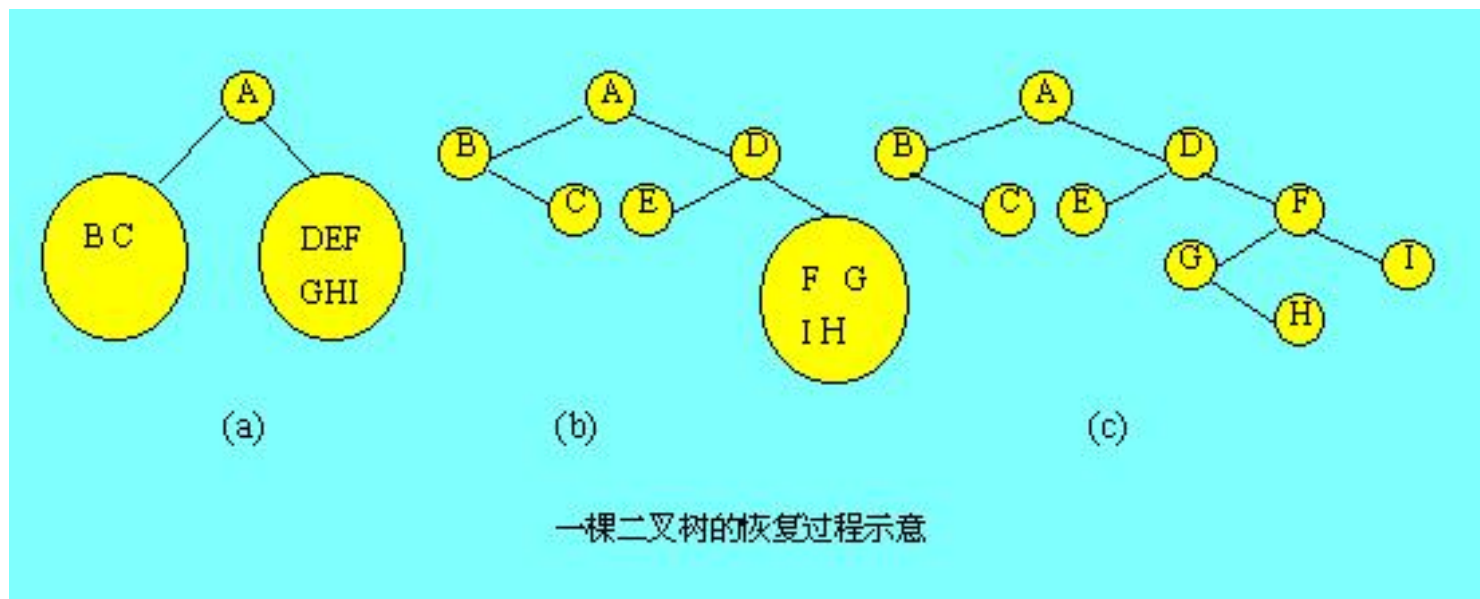
由二叉树的先序序列和中序序列可唯一地确定该二叉树。

由二叉树的后序序列和中序序列也可唯一地确定该二叉树。

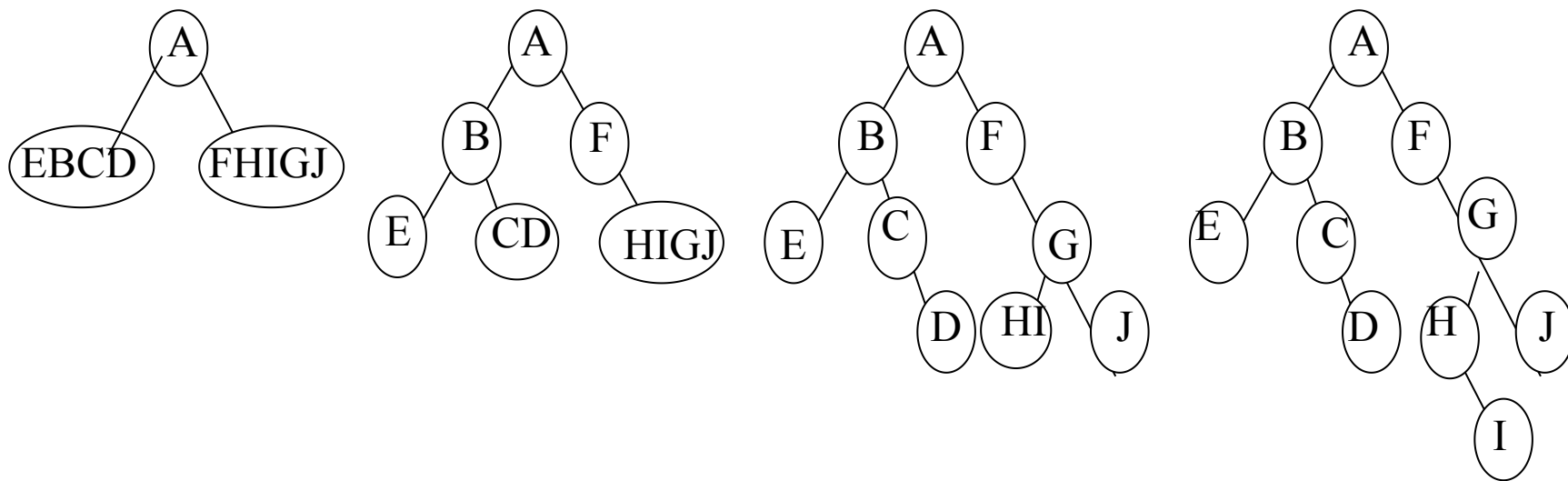
由二叉树的先序序列和后序序列不能唯一地确定该二叉树。

2、分隔过程：

已知一棵二叉树的先序序列与中序序列分别为： A B C D E F G H I , B C A E D G H F I , 试恢复该二叉树。



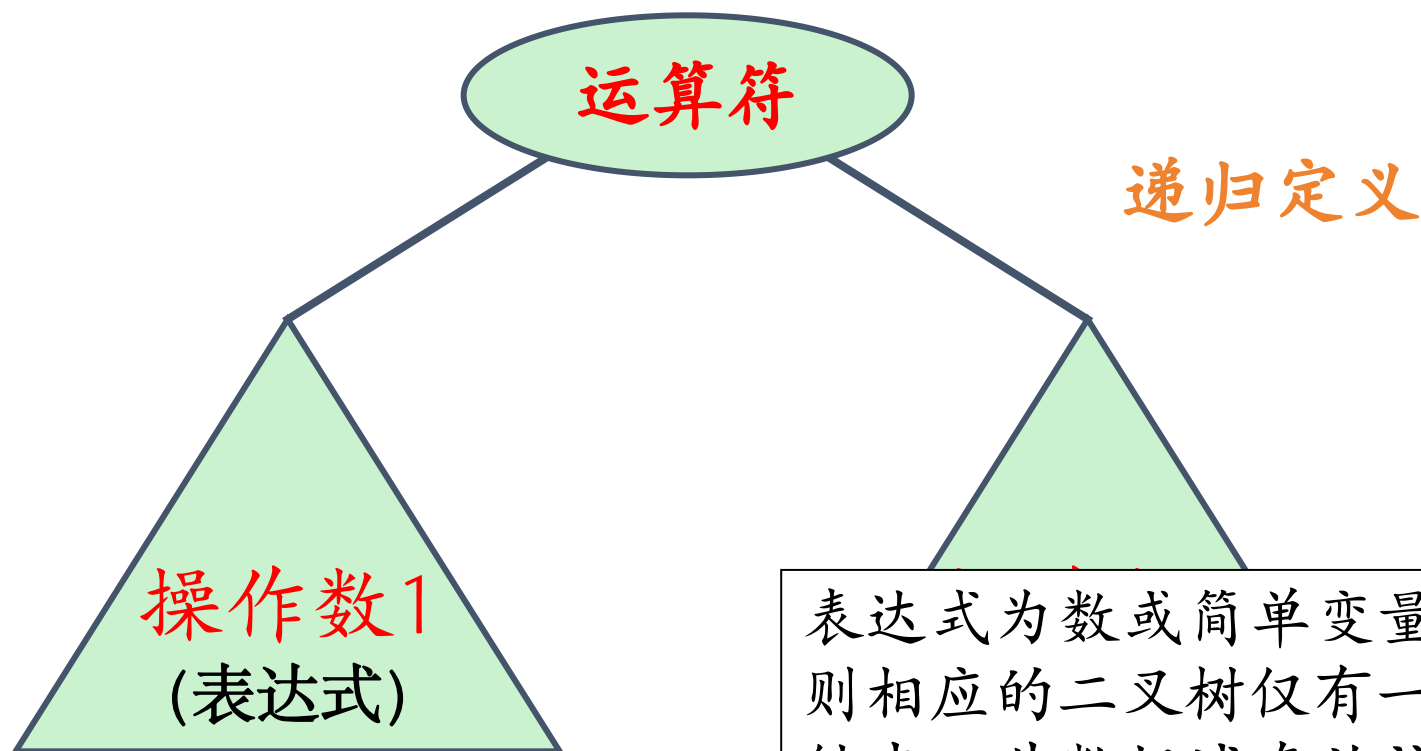
当先序序列为ABECDFFGHIJ，中序序列为EBCDAFHIGJ时，逐步形成二叉树的过程如下图所示：



核心思路：先根据先序序列的第一个元素建立根结点，然后在中序序列中找到该元素，确定根结点的左、右子树的中序序列，再在先序序列中确定左、右子树的先序序列；最后递归调用分别恢复左子树和右子树。

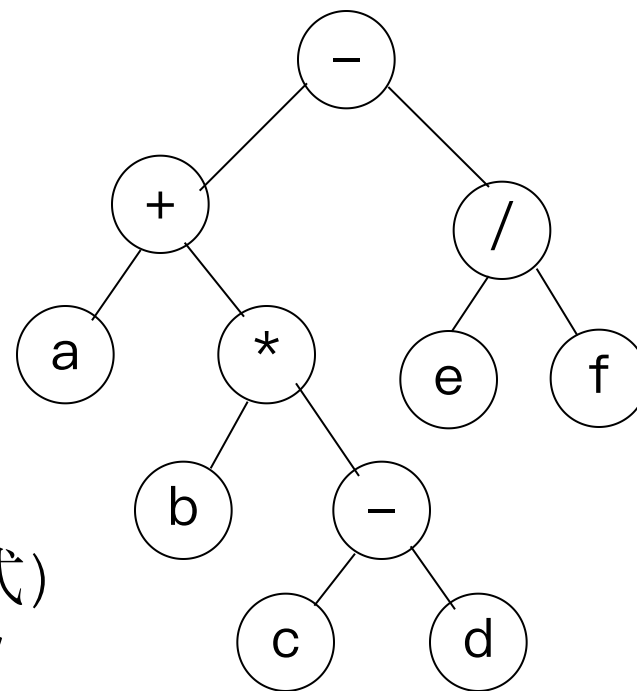
二叉树表示的表达式

表达式 = (操作数1)(运算符)(操作数2)



表达式为数或简单变量时，
则相应的二叉树仅有一个根
结点，其数据域存放该表达
式信息。

[例] $a+b*(c-d)-e/f$



先序序列

前缀表示 (波兰式)

$-+a*b-cd/ef$

中序序列

中缀表示

$a+b*(c-d)-e/f$

后序序列

后缀表示 (逆波兰式)

$abcd-*+ef/-$

思考：中序遍历如何得到正确的中缀表达式？

6.3.2 线索二叉树

6.3.2 线索二叉树

遍历二叉树实际上就是将树中所有结点排成一个**线性序列**（即非线性结构线性化）。在这样的线性序列中，除第一个结点外，每个结点有且仅有一个直接前驱结点；除最后一个结点外，每个结点有且仅有一个直接后继结点。

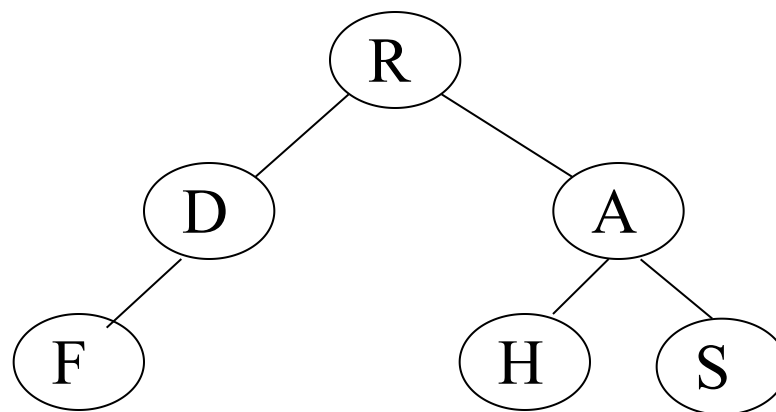
然而，有时我们希望**不进行遍历**，就能很快找到某个结点在某种遍历下的**直接前趋和后继**。为此，就需要把每个结点的**直接前趋和直接后继**记录下来。

线索二叉树分为：

先序线索二叉树

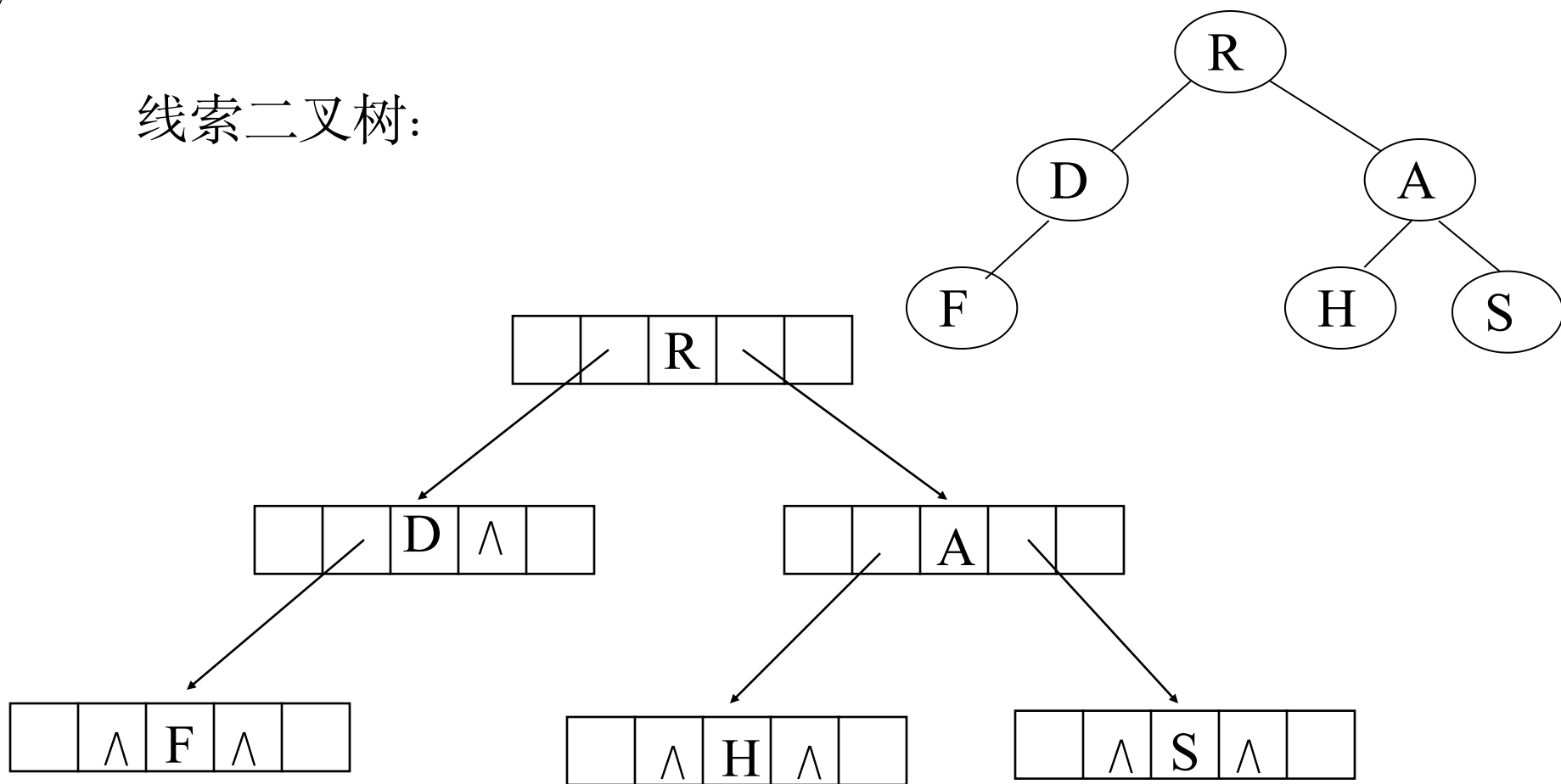
中序线索二叉树

后序线索二叉树



DLR: R D F A H S

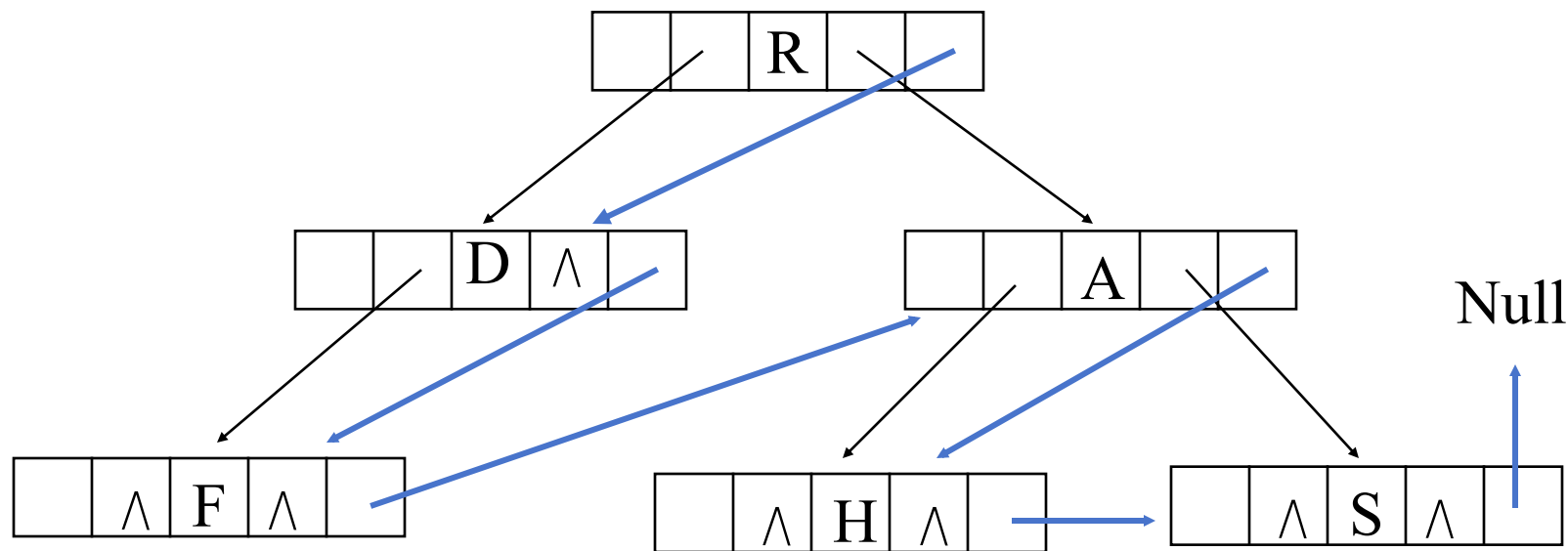
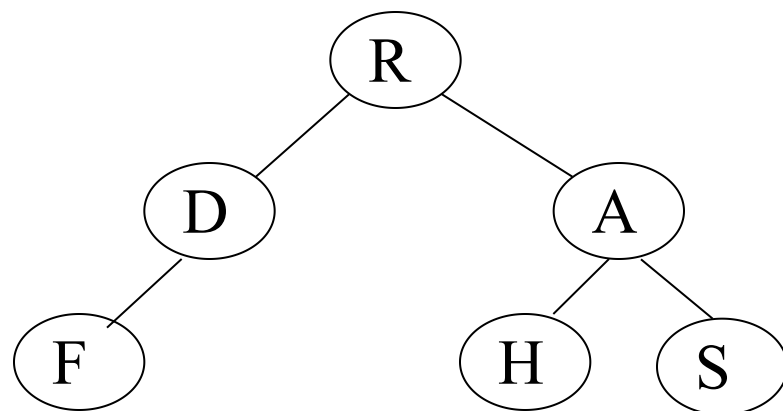
线索二叉树:



但二叉树的存储结构中并没有反映出前趋和后继信息。一个最简单的办法是在原来的二叉链表的每个结点中，再增加两个指针域**fwd**和**bkwd**，分别指向前趋和后继。这些指针被称为**线索 (thread)**，加了线索的二叉树叫做**线索二叉树**

线索二叉树（先序）：

红箭头指向后继

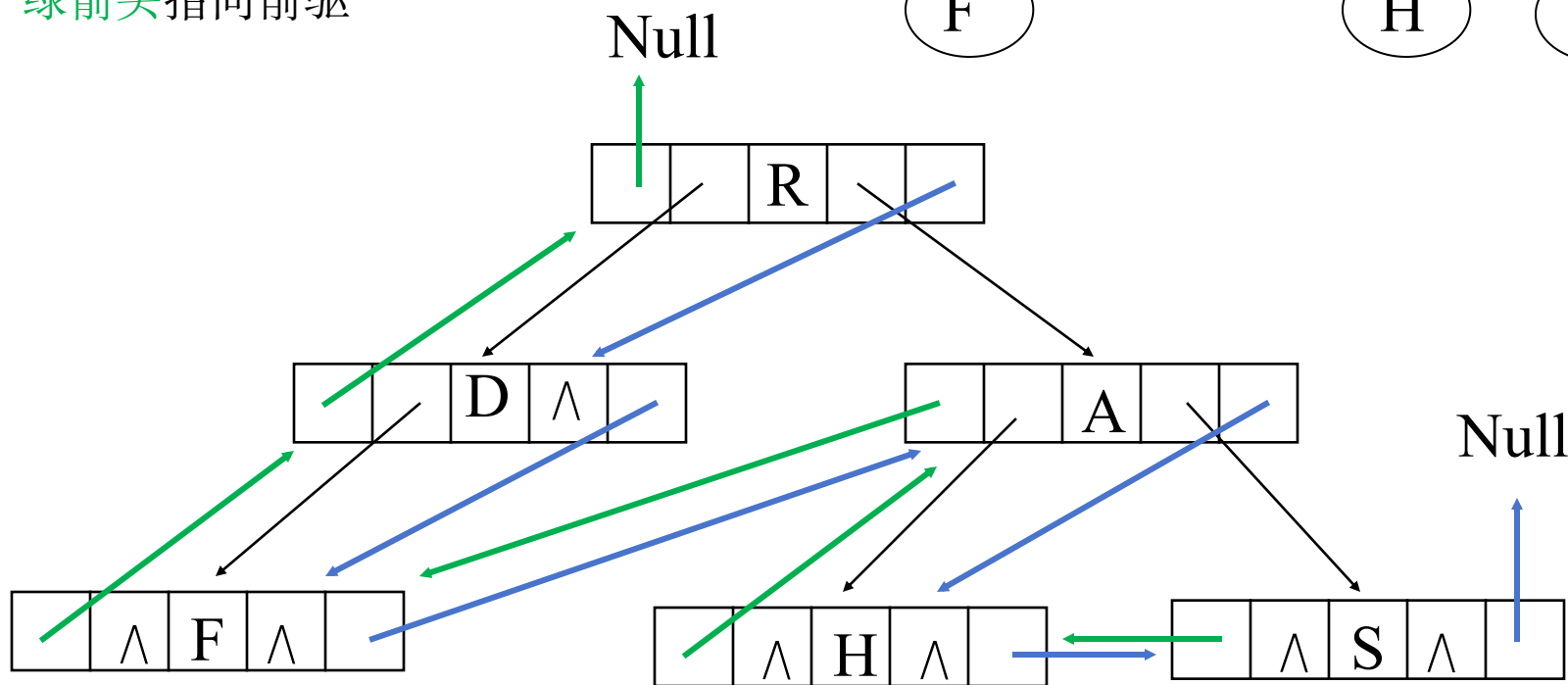
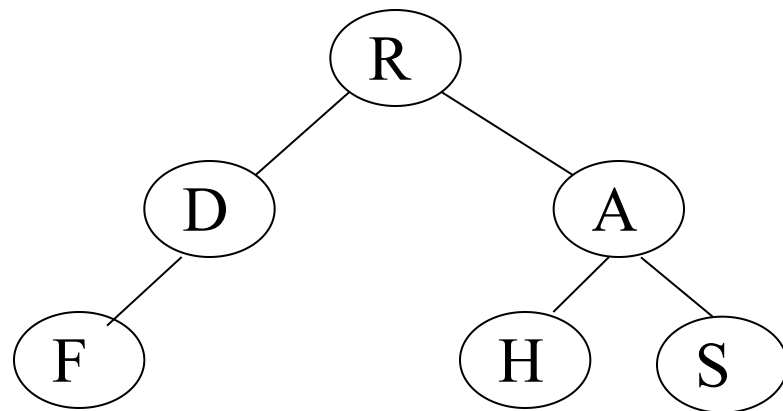


DLR: R D F A H S

线索二叉树（先序）：

红箭头指向后继

绿箭头指向前驱

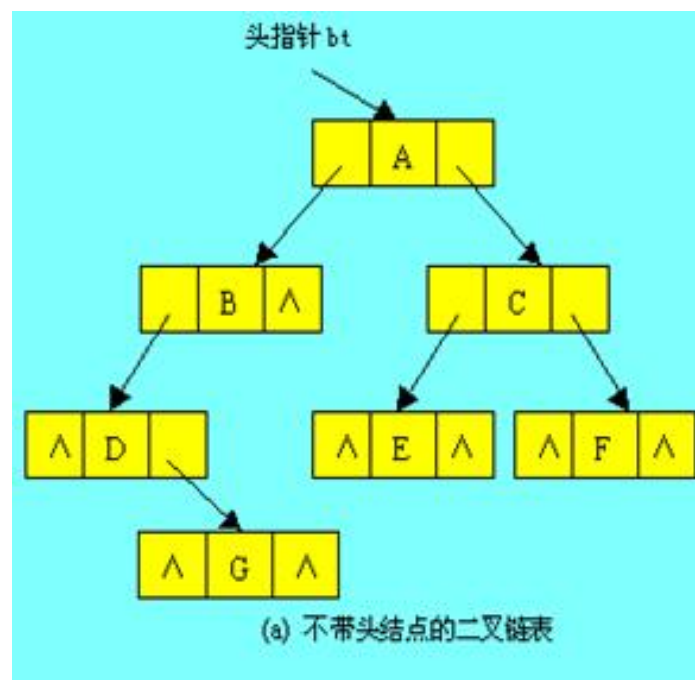


DLR: R D F A H S

- 上述做法存储效率较低.
- 一个具有 n 个结点的二叉树若采用二叉链表存储结构, 在 $2n$ 个指针域中只有 $n-1$ 个指针域是用来存储孩子结点的地址, 而另外 $n+1$ 个指针域存放的都是NULL。
- 例如下图: 7个结点, 14个指针域, 6个指针域存储孩子结点地址, 8个存放的是NULL。

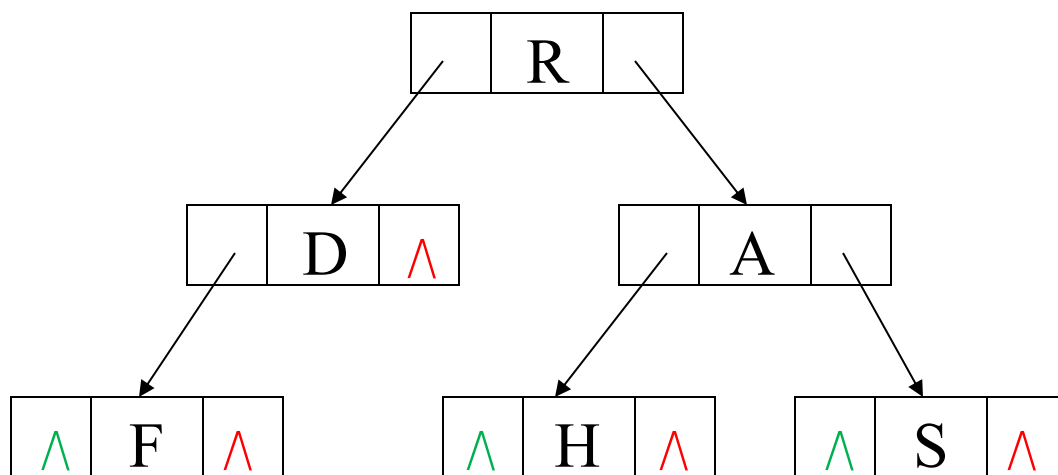
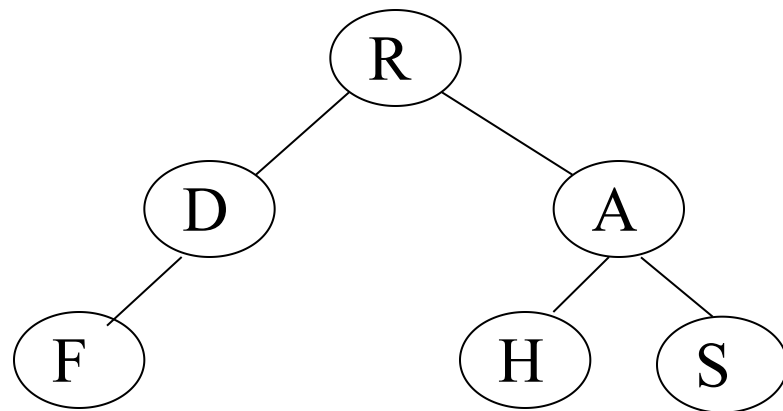
一个新的思路:

是利用二叉树的二叉链表存储结构中的那些空指针域来指示前趋和后继。



- 利用二叉链表的空指针域，存储指向该结点的前驱/后继(某种遍历方式下)结点的指针，方便二叉树的线性化使用。(n个结点的二叉树含有n+1空指针域)
- 例如：可以利用某结点空的左指针域 (lchild) 指出该结点在某种遍历序列中的直接前驱结点的存储地址，利用结点空的右指针域 (rchild) 指出该结点在某种遍历序列中的直接后继结点的存储地址。对于那些非空的指针域，则仍然存放指向该结点左、右孩子的指针。这样，就得到了一棵线索二叉树。

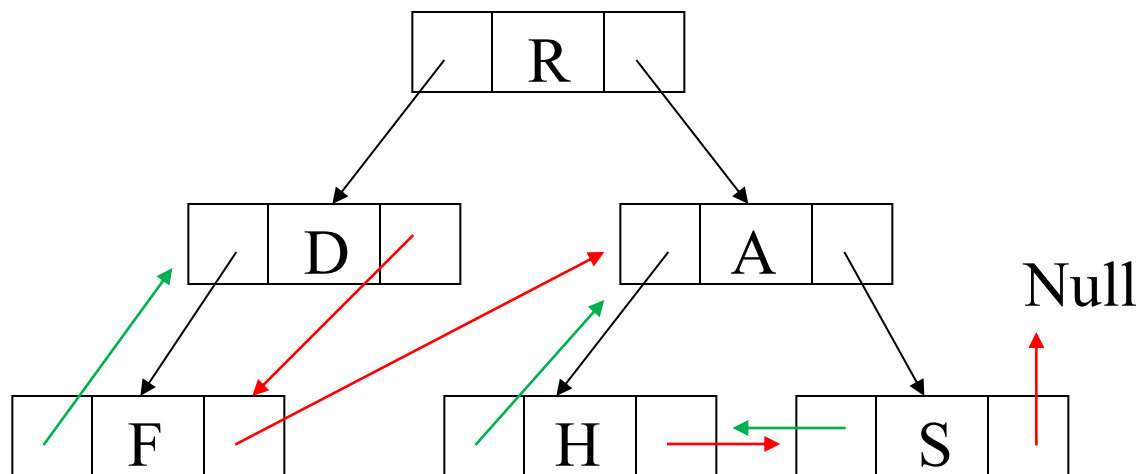
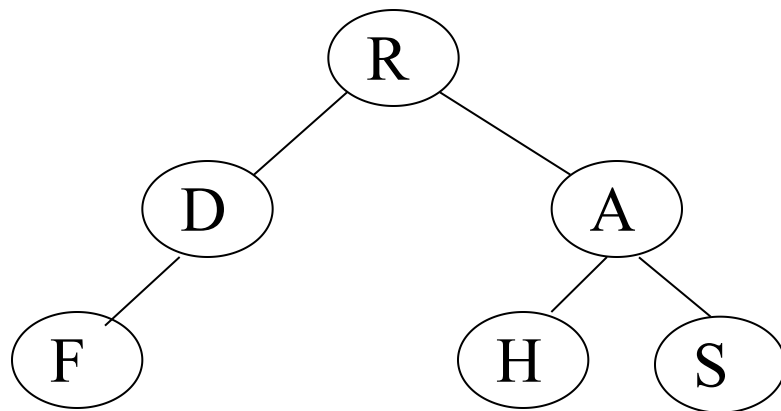
线索二叉树:



图中有7个空指针可以被利用

线索二叉树:

红箭头指向后继
绿箭头指向前驱



DLR: R D F A H S

图中所有结点的左右指针都被利用上了。

指向孩子的指针和线索都是地址，如何区别某结点的指针域内存放的是指针还是线索？方法是为每个结点增设两个标志位域ltag和rtag，令：

ltag = $\begin{cases} 0 & \text{lchild 指向结点的左孩子} \\ 1 & \text{lchild 指向结点的前驱结点} \end{cases}$

rtag = $\begin{cases} 0 & \text{rchild 指向结点的右孩子} \\ 1 & \text{rchild 指向结点的后继结点} \end{cases}$

结点结构为：



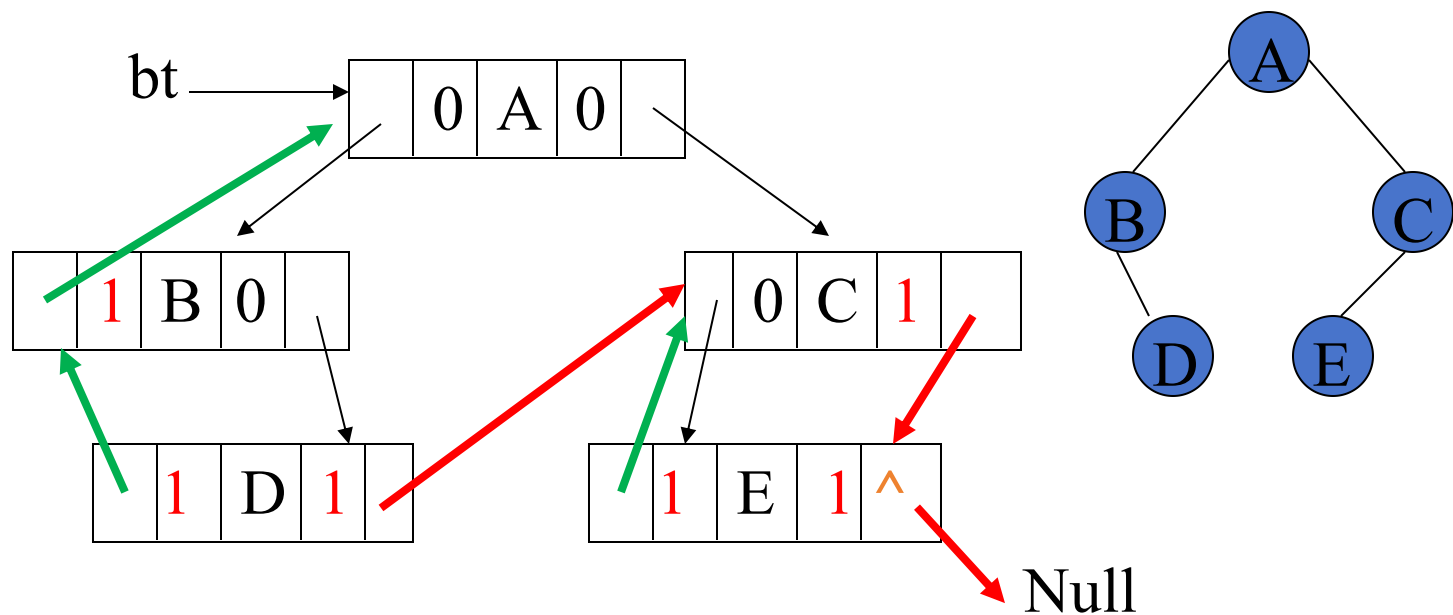
线索二叉树的结点定义如下：

```
typedef struct BiThrNode
{
    datatype data;
    struct BiThrNode *lchild;
    struct BiThrNode *rchild;
    PointerTag ltag;
    PointerTag rtag;
}BiThrNodeType, *BiThrTree;
```

【例】

先序线索二叉树

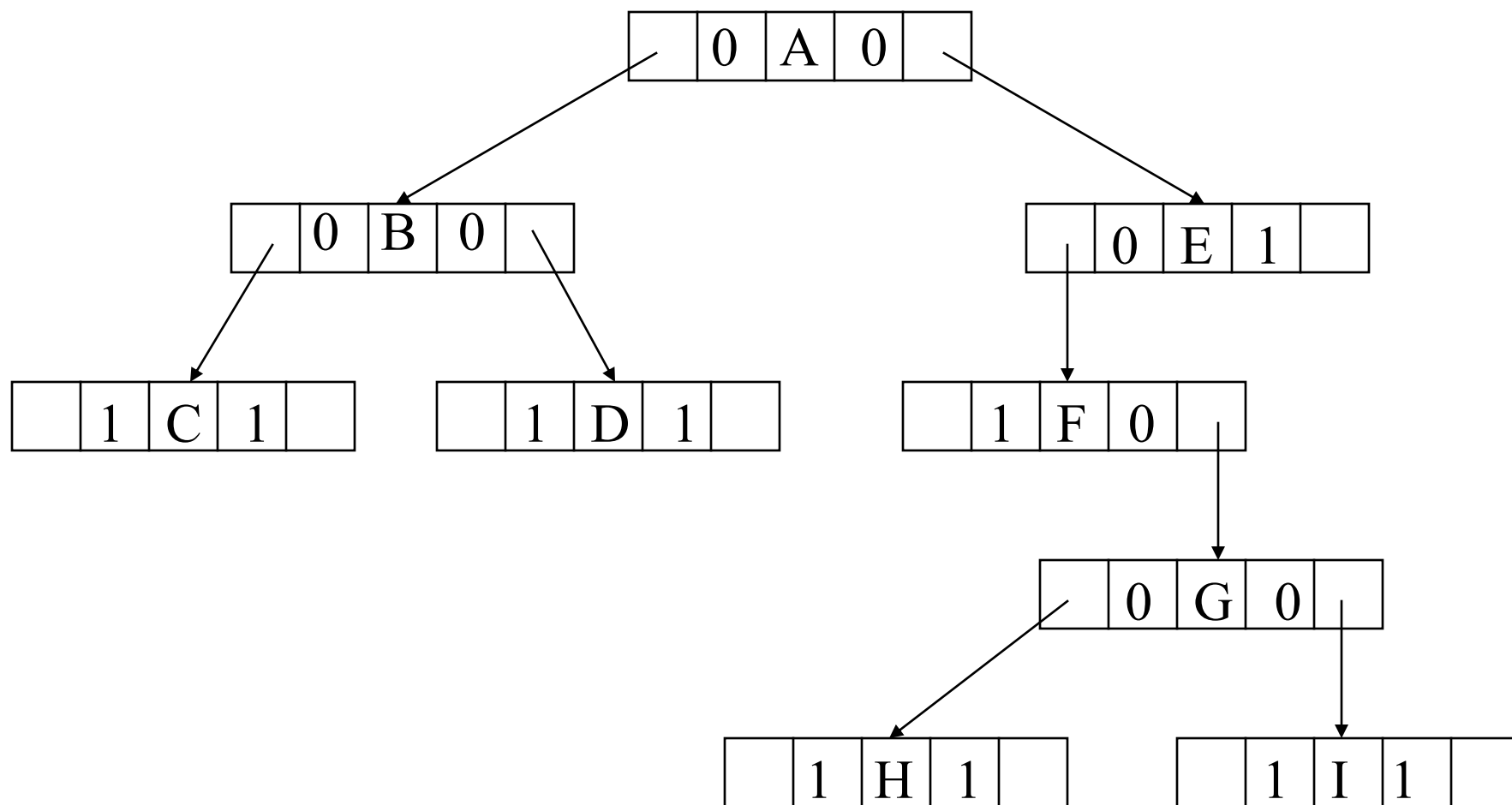
ABDCE



0表示为孩子指针，1表示为线索指针

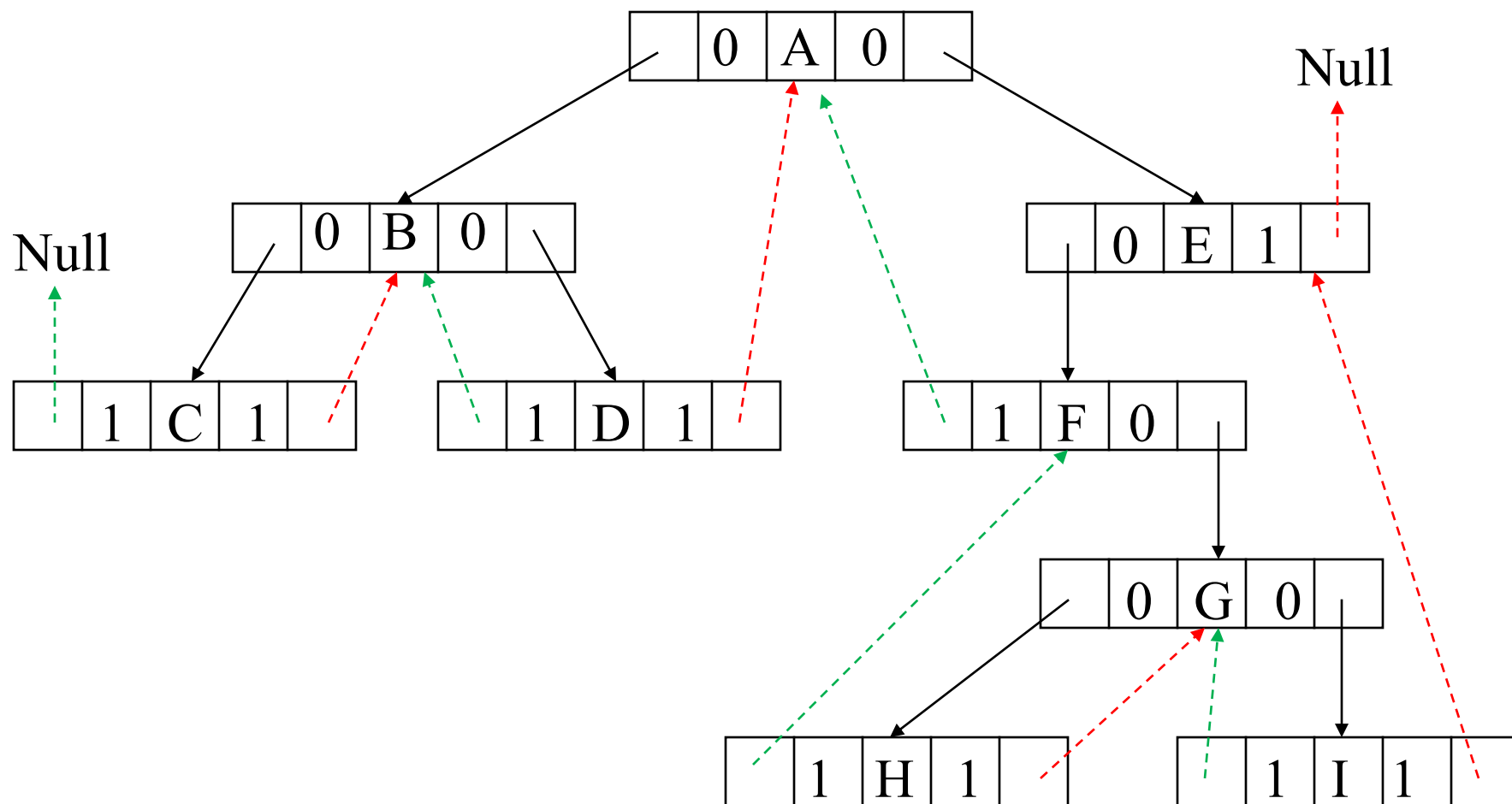
红箭头指向后继
绿箭头指向前驱

线索二叉树（中序）：



LDR: C B D A F H G I E

线索二叉树（**中序**）：

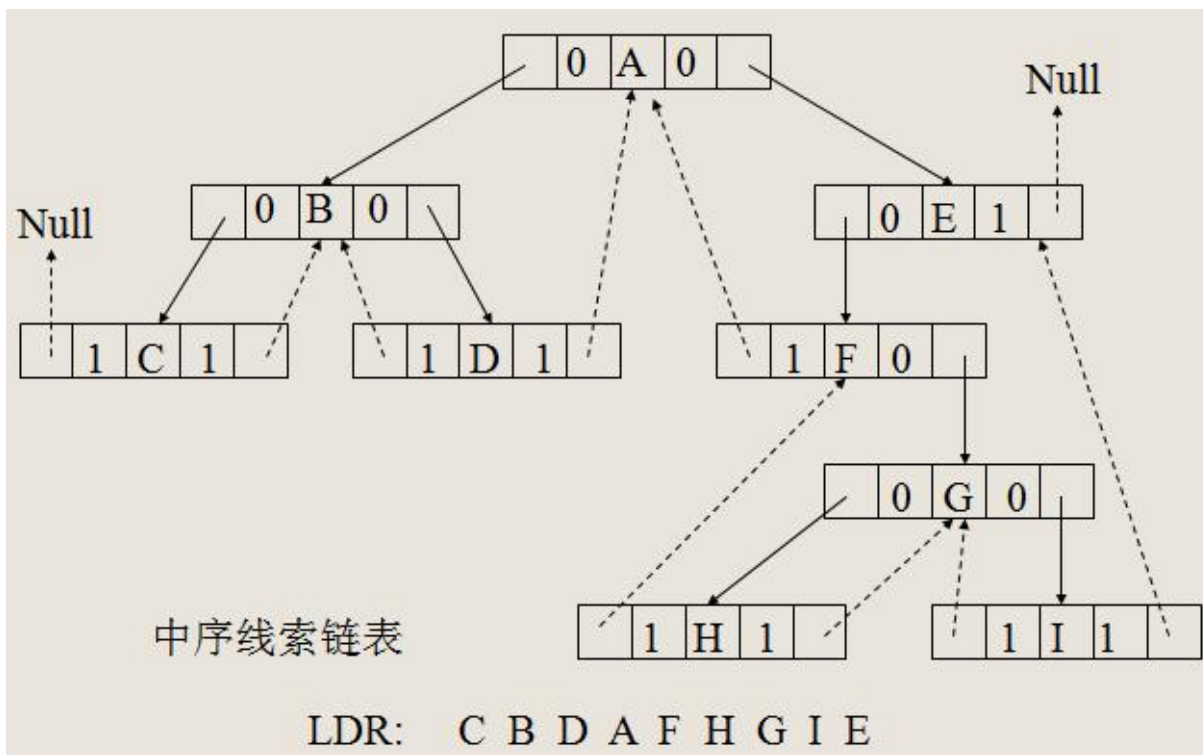


红箭头指向后继
绿箭头指向前驱

LDR: C B D A F H G I E

有些结点的Lchild或Rchild指针域指向其左孩子或右孩子，而**没有记录其前趋或后继结点**信息，要找到该结点的前趋或后继，还要进行一定的搜索。

例如，图A结点的**前趋**是D（即A结点的左子树的**最右下**结点）；A结点的**后继**是F（即A结点的右子树的**最左下**结点）。



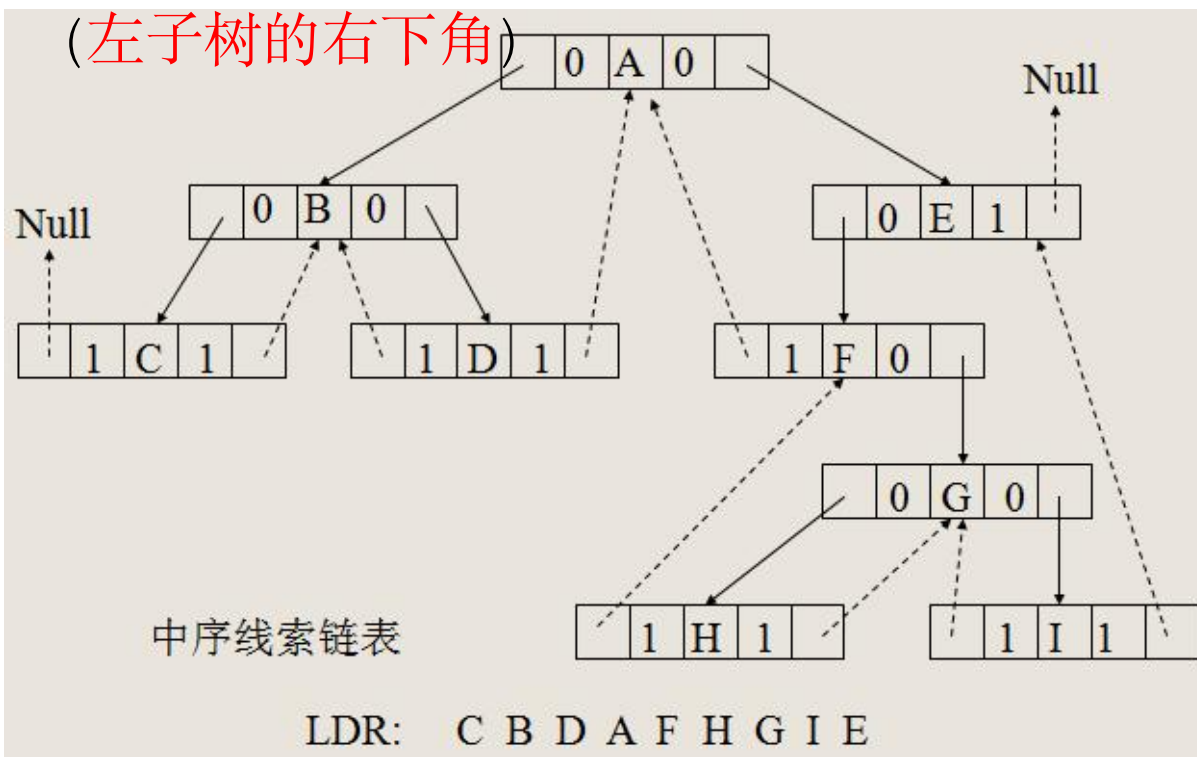
查找结点p在指定次序下的前驱 / 后继结点

- 中序线索二叉树

- p的前驱

若 $p \rightarrow ltag = 1$, 则 $p \rightarrow lchild$ 即为所求;

若 $p \rightarrow ltag = 0$, 则从其左子沿着右链走到 $rtag = 1$ 的那个结点就是。
(左子树的右下角)



查找结点p在指定次序下的前驱 / 后继结点

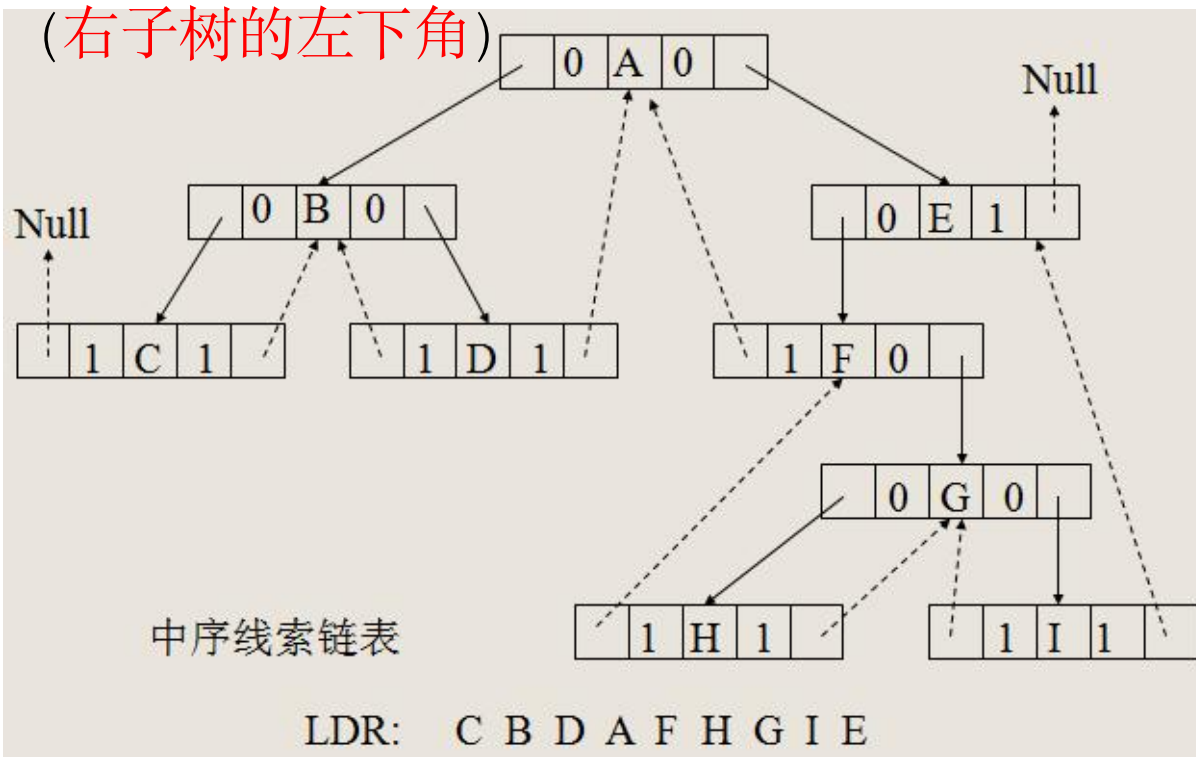
- 中序线索二叉树

- p的后继

若 $p \rightarrow rtag = 1$ ，则 $p \rightarrow rchild$ 即为所求；

若 $p \rightarrow rtag = 0$ ，则从其右子沿着左链走到 $ltag = 1$ 的那个结点就是。

(右子树的左下角)

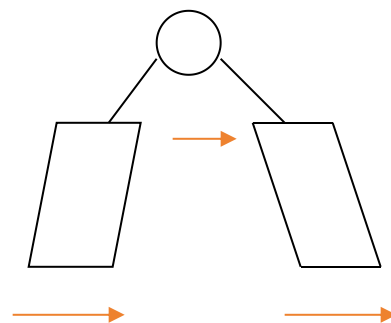


二叉线索树的遍历不需要递归，在先序/中序线索二叉树中，利用线索可方便地找到当前结点的后继结点。因此，对有 n 个结点的二叉线索树，可以在 $O(n)$ 时间内遍历完该树。

[中序线索二叉树遍历算法步骤]

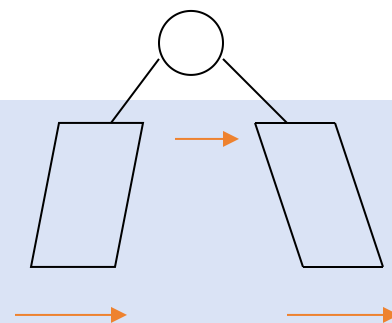
若二叉树 p 非空，

- 1) 找到中序序列的第一个结点；
- 2) 访问当前结点；
- 3) 将当前结点的后继作为新的当前结点；
- 4) 当前结点存在，则转2)



[算法描述]

```
Status InOrderTraverse_Thr(BiThrTree T,  
                           Status (* Visit)(TElemType e) )  
//T指向头结点，头结点的左指针指向根节点  
//中序遍历二叉线索树的非递归算法  
{ p=T->lchild; //p指向根结点  
  while ( p!=T ) { //结束判断条件，因为最后一个结点的next回指T  
    while ( p->LTag==Link) p=p->lchild;  
    //找到最左下角结点，即接下来应该访问的第一个结点  
    if ( !Visit(p->data) ) return ERROR; //访问该结点  
    while (p->Rtag==Thread && p->rchild!=T) { //如果右指针为线索且  
      不为空，则p->rchild即为后继  
      p=p->rchild; Visit(p->data); //指向后继并访问  
    } // while  
    p=p->rchild; //p指向右子树根结点，下一步再去找左下角  
  }  
  return OK;  
} // InOrderTraverse_Thr
```



- 后序线索二叉树

- p的前驱

若 $p \rightarrow ltag = 1$, 则 $p \rightarrow lchild$ 即为所求;

若 $p \rightarrow ltag = 0$ (p有左孩子),

则若 $p \rightarrow rtag = 0$, 则 $p \rightarrow rchild$ 即为所求 (有右子树时, 右子树根结点)

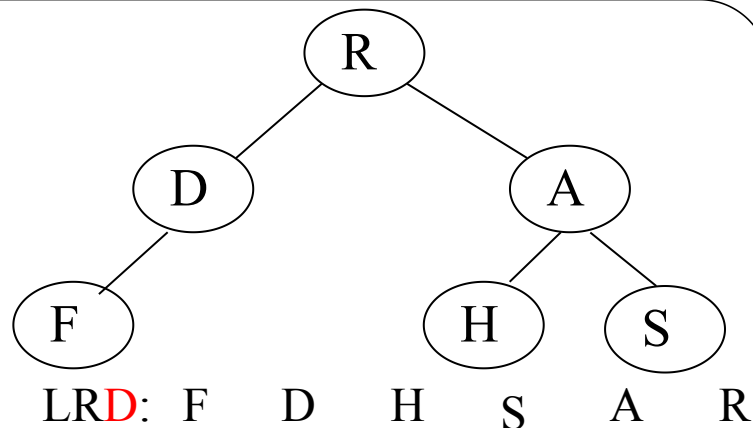
若 $p \rightarrow rtag = 1$, 则 $p \rightarrow lchild$ 即为所求 (没有右子树时, 左子树根结点)

- p的后继

与双亲结点有关, 因二叉链表中没有指向双亲结点的指针, 就可能需通过二叉树的后序遍历才可确定, 因而后序线索二叉树在此问题上并不比普通二叉树有效。

- 先序线索二叉树

与后序线索二叉树对偶



线索二叉树的构建

由于线索化的实质是将二叉链表中的空指针改为指向前驱或后继的线索，而前驱或后继的信息只有在遍历时才能得到，因此线索化的过程即为在遍历过程中，访问结点的操作是检查当前结点的左、右指针域是否为空，如果为空，将它们改完指向前驱结点或后继结点的线索。

为实现这一过程，设指针pre始终指向刚刚访问过的结点，即若指针p指向当前结点，则pre指向它的前驱，以便增设线索。

二叉树中序线索化

[算法步骤]

若二叉树p非空,

1) 左子树线索化;

2) 访问当前结点p:

2.1) 置 LTag和RTag;

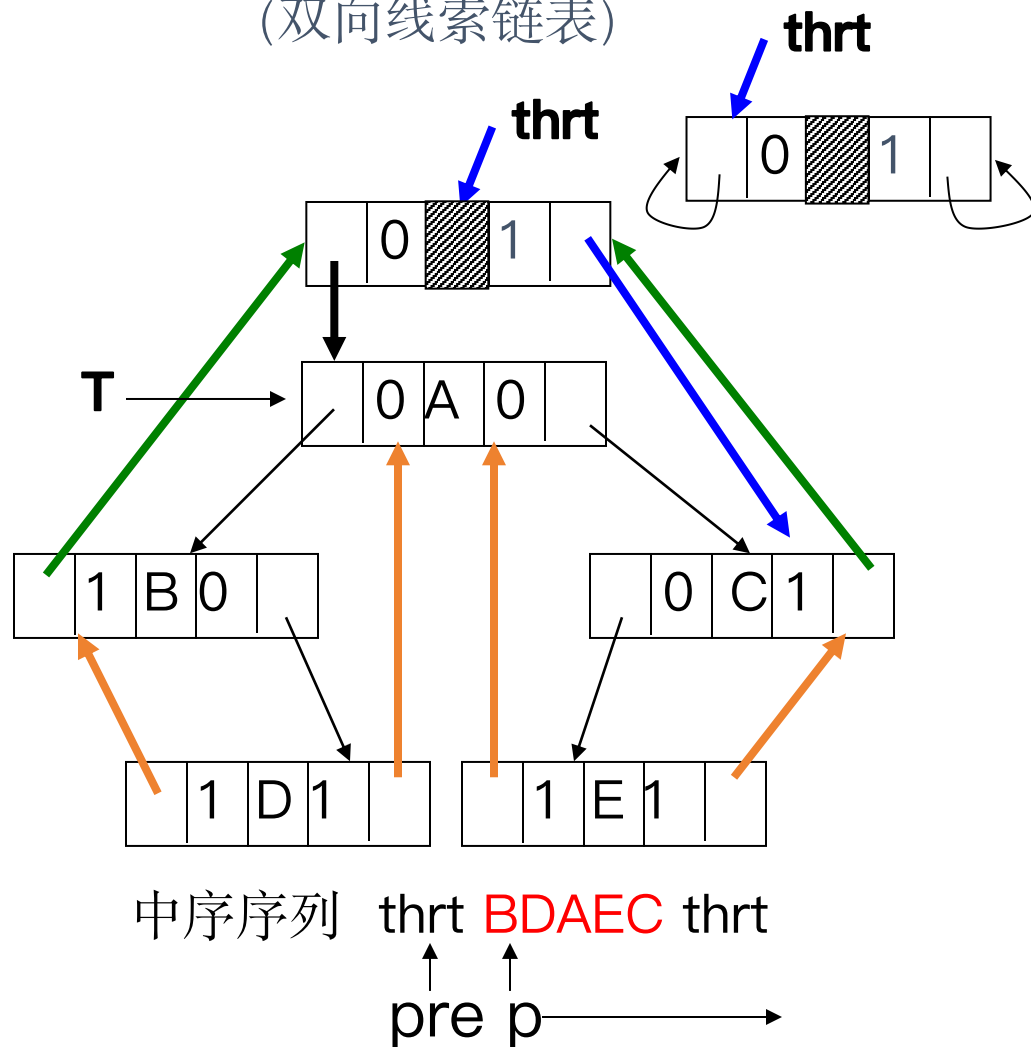
2.2) 建立p的前趋线索;

2.3) 建立p的前趋pre的后继线索;

2.4) 将p作为下一个被访问结点的前趋结点

3) 右子树线索化;

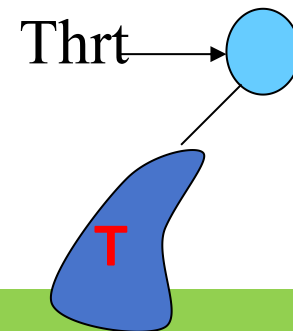
带头结点的中序线索二叉树 (双向线索链表)



算法描述参教材P134-135 算法6.6、6.7

[算法描述]

附设一个指针`pre`（全局变量）始终指向刚访问过的结点

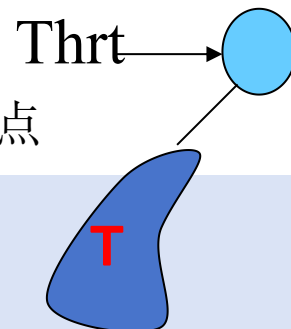


```
void InThreading( BiThrTree p)
{  if (p) {
    InThreading(p->lchild);    //左子树线索化
    if ( ! p->lchild) //如果p没有左孩子
        { p->LTag=Thread; p->lchild=pre; } //p结点存储前驱线索
    if ( ! pre->rchild) //如果pre没有右孩子
        { pre->RTag=Thread; pre->rchild=p; } //pre结点存储后继线索
    pre=p; //保持pre指向p的前驱
    InThreading(p->rchild); //右子树线索化
  }
} // InThreading  P135-6.7
```

[算法描述]

附设一个指针pre（全局变量）始终指向刚访问过的结点

```
Status InOrderThreading( BiThrTree &Thrt, BiThrTree T)
// 中序遍历二叉树T并将其线索化, Thrt指向头结点
{ if (!(Thrt=(BiThrTree)malloc(sizeof(BiThrNode))))
    exit(OVERFLOW); //构造头结点
    Thrt->LTag=Link; Thrt->Rtag=Thread; //构造头结点
    Thrt->rchild=Thrt; //右指针回指
    if (!T) Thrt->lchild= Thrt; //若二叉树空, 左指针回指
    else { Thrt->lchild=T; //头结点的左指针是根结点
        pre=Thrt //将头结点作为初始的pre
        InThreading(T);
        pre->RTag=Thread; //最后一个结点线索化
        pre->rchild=Thrt; //最后一个结点的后续是头结点
        Thrt->rchild=pre; // 头结点的右指针指向最后一个结点
    }
    return OK;
}
} // InOrderThreading P134-6.6
```



6.6 哈夫曼树及其应用

6.6 哈夫曼树及其应用

6.6.1 问题的引入

一个将百分制转换为五级分制的判定程序可以用以下条件语句完成：

if (a<60) b="bad";

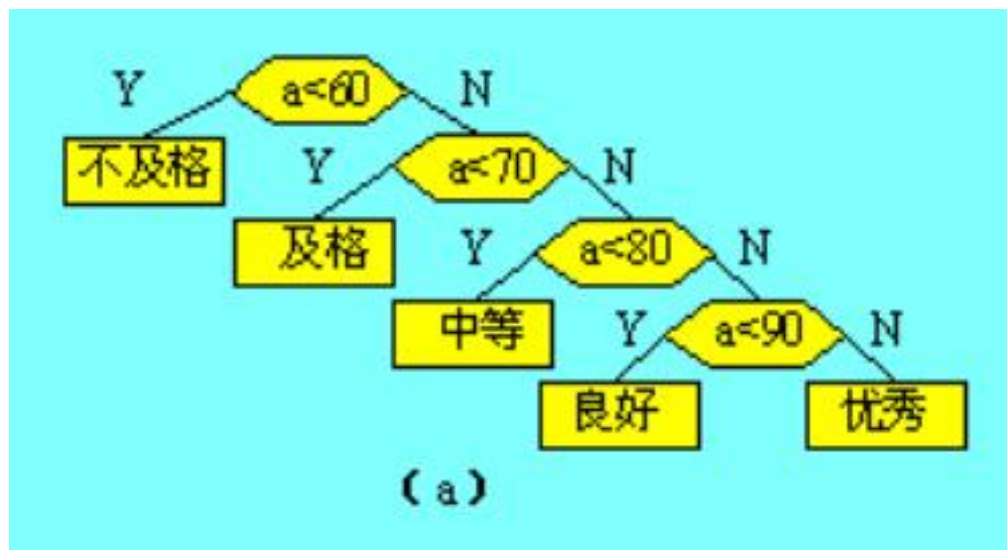
else if (a<70) b="pass"

else if (a<80) b="general"

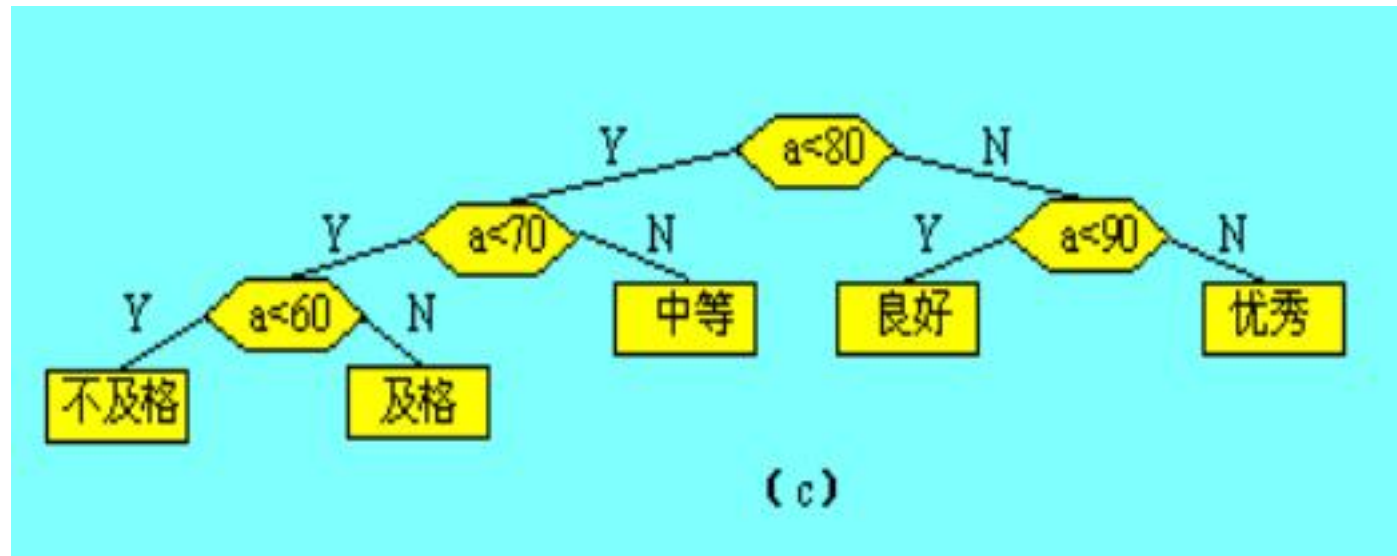
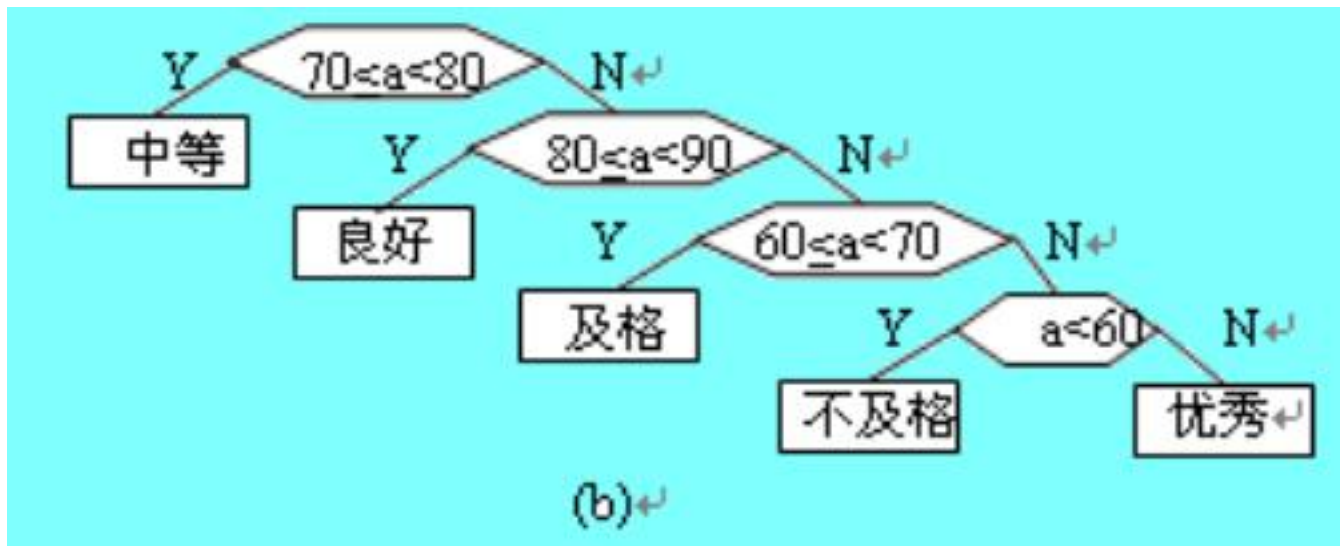
else if (a<90) b="good"

else b="excellent";

判定过程如下图所示。



判定方法是多样的，也可以采用下面的判定过程来完成。



如果输入量很大，则应考虑上述程序的质量问题，即其操作所需要的时间。因为在实际中，学生的成绩在五个等级上的分布是不均匀的，假设其分布规律如下表所示：

分数	0 – 59	60 – 69	70 – 79	80 – 89	90 – 100
比例数	0.05	0.15	0.40	0.30	0.10

假设有10000个输入数据，若按图(a)的判定过程进行操作，则总共需进行31500次比较；而若按图(c)的判定过程进行操作，则总共仅需进行22000次比较。

[术语]

结点权值: 和叶子结点对应的一个有某种意义的实数(w_i)

结点带权路径长度 叶子结点的路径长度与该结点的权之积。

树的路径长度 从树根到每一个叶子结点的路径长度之和。

树的带权路径长度 树中所有叶子结点的带权路径长度之和。

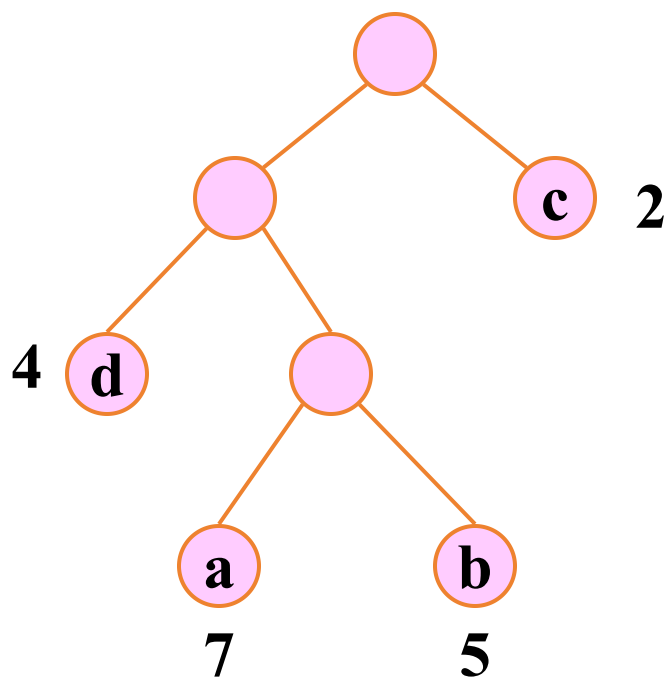
记作
$$wpl = \sum_{k=1}^n w_k l_k$$

其中 w_k — 权值

l_k — 结点到根的路径长度

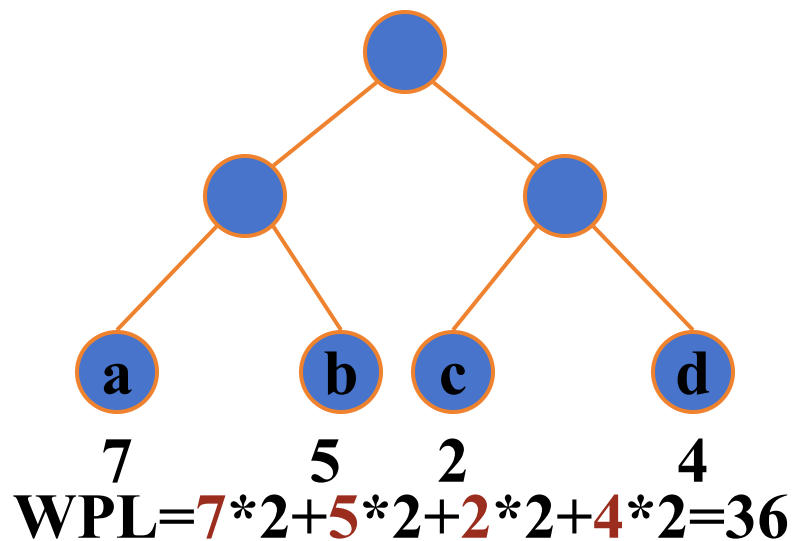
最优二叉树(哈夫曼树) 带权路径长度WPL最小的 二叉树。

【例】 有4个结点，权值分别为7，5，2，4，构造有4个叶子结点的二叉树。

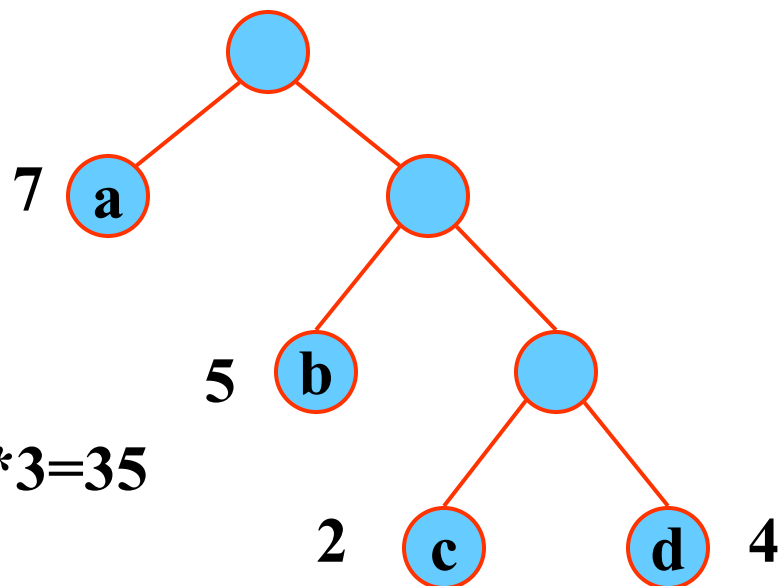


$$WPL = 7 \times 3 + 5 \times 3 + 2 \times 1 + 4 \times 2 = 46$$

$$WPL = 7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 3 = 35$$



$$WPL = 7 \times 2 + 5 \times 2 + 2 \times 2 + 4 \times 2 = 36$$



最优二叉树

在权值为 $w_1, w_2, \dots, w_n$ 的 n 个叶子所构成的所有二叉树中，带权路径长度最小的二叉树，称为最优二叉树(哈夫曼树)。

那么如何对于给定的叶子数目及其权值来构造一棵最优二叉树呢？从前图可以看出，最优二叉树中，**权值越大的叶子距离根越近**。哈夫曼首先给出了构造最优二叉树的方法，故我们称其为**哈夫曼算法**。

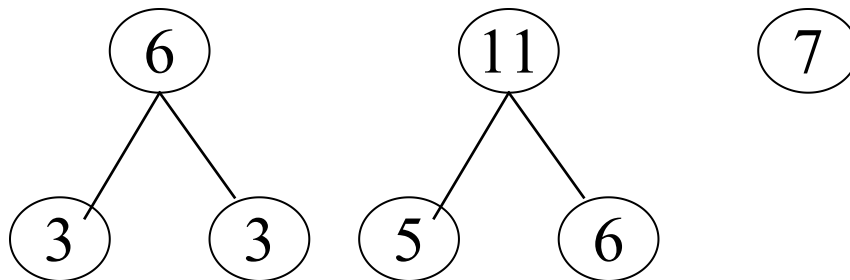
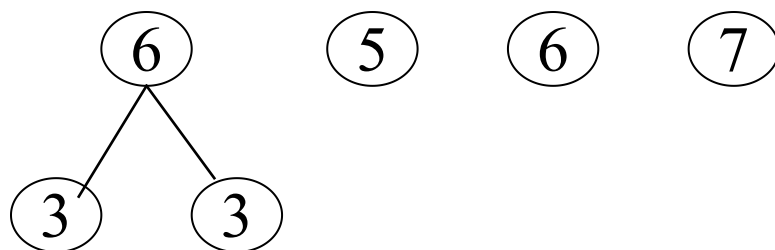
建立哈夫曼树（最优二叉树）的方法

【基本思想】

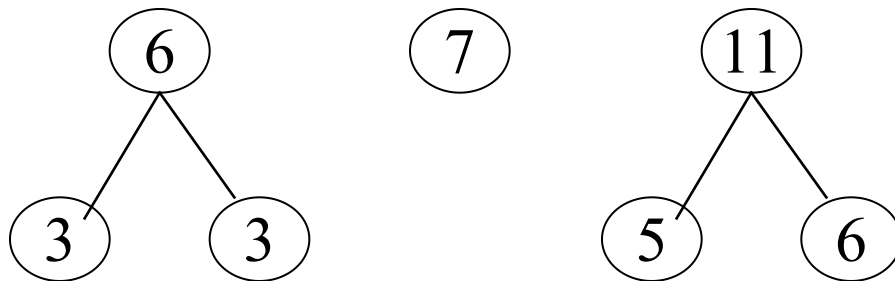
使权大的结点靠近根。

- (1)将给定权值从小到大排序成 $\{w_1, w_2, \dots, w_m\}$ ，生成一个森林
 $F = \{T_1, T_2, \dots, T_m\}$ ，其中 T_i 是一个带权 w_i 的根结点，它的左右子树均空。
- (2)把 F 中根的权值最小的两棵二叉树 T_1 和 T_2 合并成一棵新的二叉树
 T ： T 的左子树是 T_1 ，右子树是 T_2 ， T 的根的权值是 T_1 、 T_2 树根
结点权值之和。
- (3)将 T 按权值大小加入 F 中，同时从 F 中删去 T_1 和 T_2
- (4)重复(2)和(3)，直到 F 中只含一棵树为止，该树即为所求。

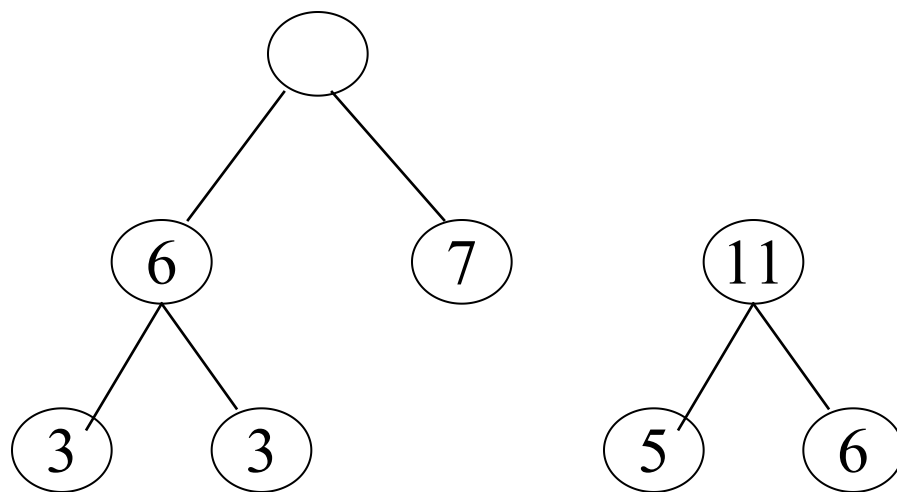
例1: 3, 3, 5, 6, 7



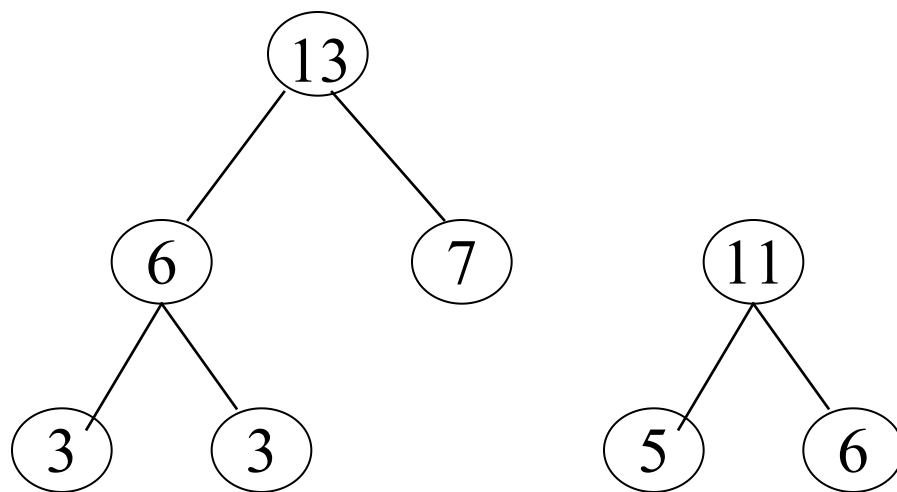
例1: 3, 3, 5, 6, 7



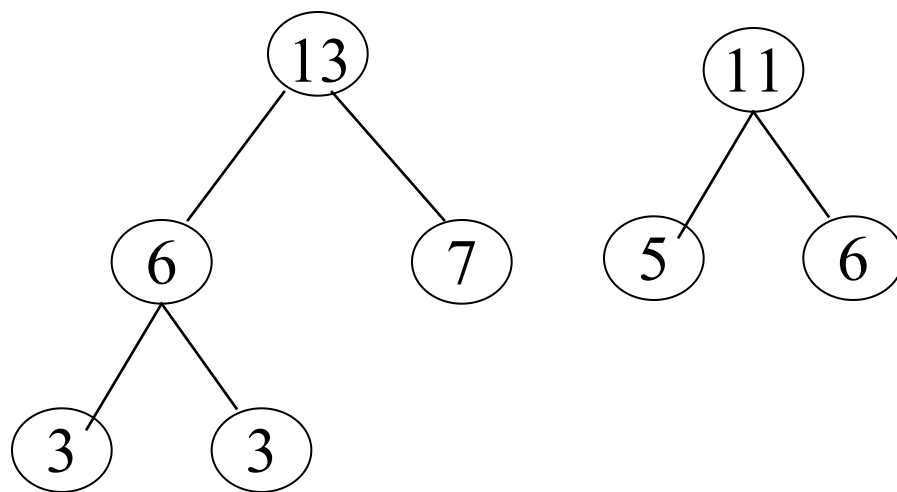
例1: 3, 3, 5, 6, 7



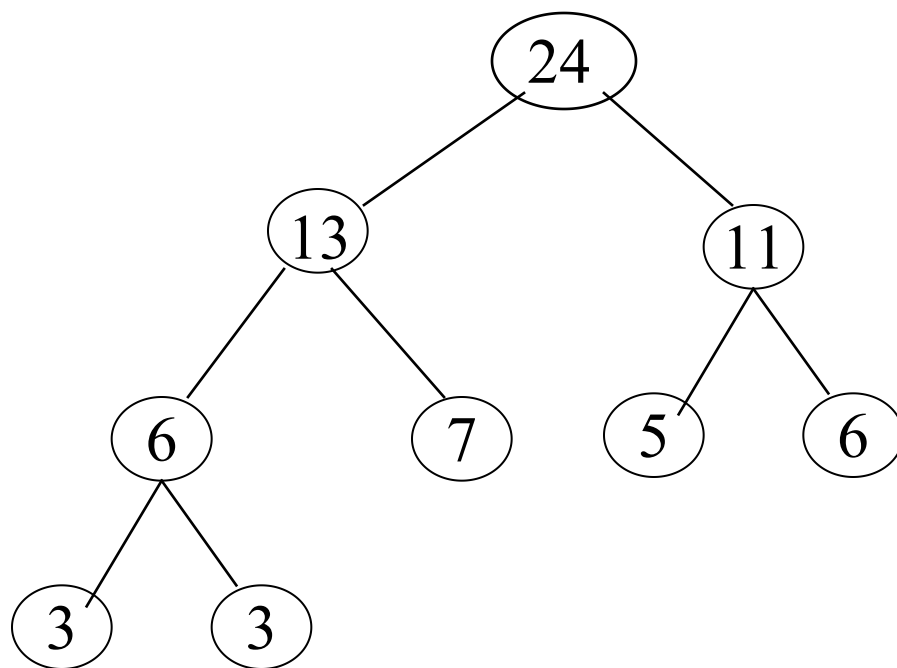
例1: 3, 3, 5, 6, 7



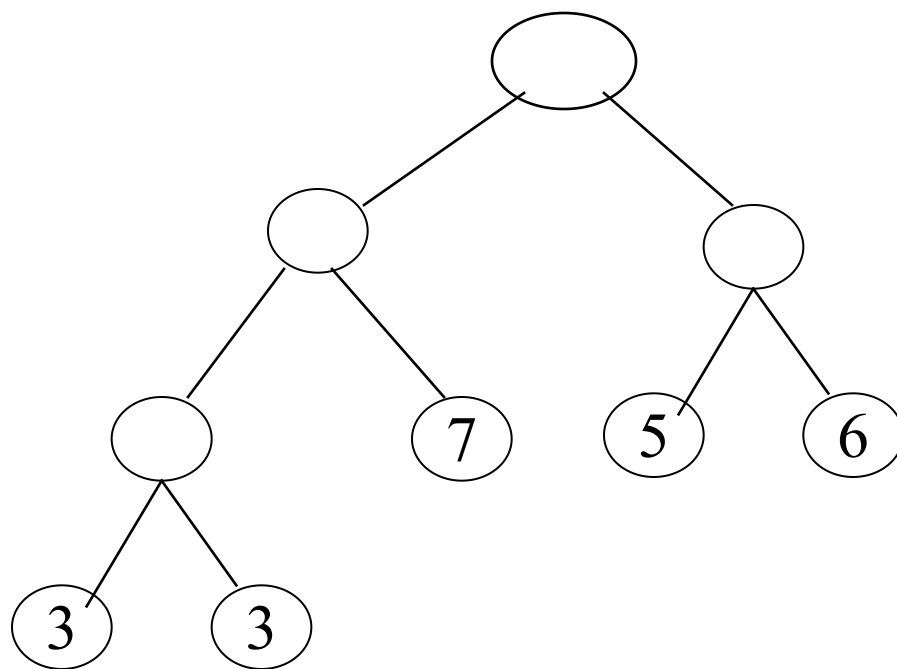
例1: 3, 3, 5, 6, 7



例1: 3, 3, 5, 6, 7



例1: 3, 3, 5, 6, 7

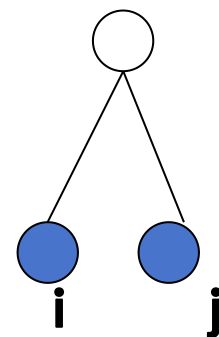


[哈夫曼树的存储结构]

- 初始有 n 个叶子结点，则构造出的哈夫曼树(只有度为0和2的结点)共有 $2n-1$ 个结点。
- 用大小为 $2n$ 的向量存储哈夫曼树—顺序存储。

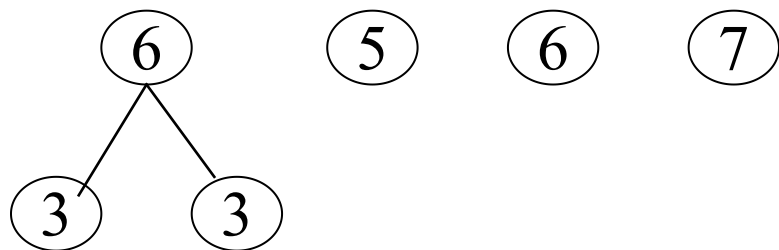
	0	1	2	i	j	n	$n+1$			$2n-1$
weight										
lchild		0	0				0			
rchild		0	0				0			
parent										

注：用0表示空指针

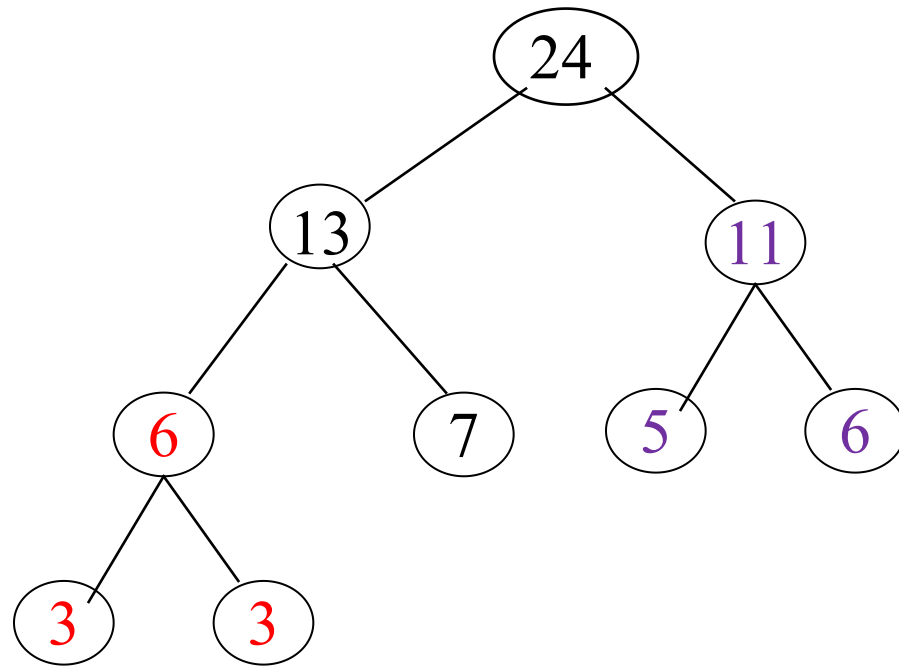


```
typedef struct {
    unsigned int    weight;
    unsigned int    parent, lchild, rchild;
}HTNode, *HuffmanTree;
//动态分配存储空间
```


1. 在HT[1..i-1]中parent = 0的结点中选weight最小的两个结点HT[s1]和HT[s2]
2. 修改HT[s1]和HT[s2]的parent值: parent=i
3. 置HT[i]: weight=HT[s1].weight + HT[s2].weight ,lchild=s1, rchild=s2



0	1	2	3	4	5	6	7	8	9
w	3	3	5	6	7	6	0	0	0
l	0	0	0	0	0	1	0	0	0
r	0	0	0	0	0	2	0	0	0
p	6	6	0	0	0	0	0	0	0



0	1	2	3	4	5	6	7	8	9
w	3	3	5	6	7	6	11	13	24
l	0	0	0	0	0	1	3	6	8
r	0	0	0	0	0	2	4	5	7
p	6	6	7	7	8	8	9	9	0

[构造哈夫曼树的算法步骤]

HuffmanTree HT;

1) 申请空间并初始化HT[1..2n-1]:

weight=0, lchild=rchild=parent=0

2) 设置HT[1..n]的weight值

3) 进行以下n-1次合并, 依次产生HT[i], $i=n+1..2n-1$:

3.1) 在HT[1..i-1]中parent = 0的结点中选weight最小的两个结点HT[s1]和HT[s2]

3.2) 修改HT[s1]和HT[s2]的parent值: parent=i

3.3) 置HT[i]: weight=HT[s1].weight + HT[s2].weight ,
lchild=s1, rchild=s2

[构造哈夫曼树的算法]

```
Void BuildHT(HuffmanTree &HT, int *w, int n)
// w存放n个叶结点的权值(均>0), 构造哈夫曼树HT
{   if ( n<=1 ) { HT=NULL; return; }
    m=2*n-1;    //计算总结点个数
    HT=(HuffmanTree)malloc((m+1)sizeof(HTNode));
    for (p=HT+1, i=1; i<=n; ++i, ++p, ++w) *p={*w,0,0,0}; //初始化
    for (; i<=m; ++i, ++p) *p={0,0,0,0}; //后续结点初始化为0
    for (i=n+1; i<=m; ++i) { //建哈夫曼树
        Select(HT, i-1, s1, s2);
        //在HT[1..i-1]中选择parent为0且权值最小的两个结点
        HT[s1].parent=i; HT[s2].parent=i;
        HT[i].lchild=s1; HT[i].rchild=s2;
        HT[i].weight=HT[s1].weight+HT[s2].weight;
    }
}
} //BuildHT
```

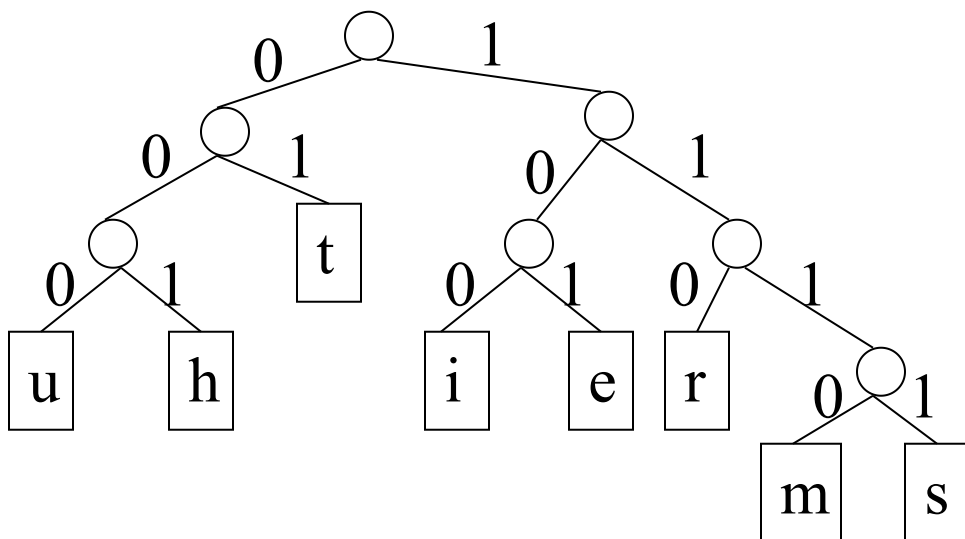
6.6.3 哈夫曼树的应用

- 最佳判定树
- 哈夫曼编码：用于通信和数据传送中字符的二进制编码，可以使电文编码总长度最短。

【例】 对time tries truth哈夫曼编码

字符集 $D=\{t, i, m, e, r, s, u, h\}$

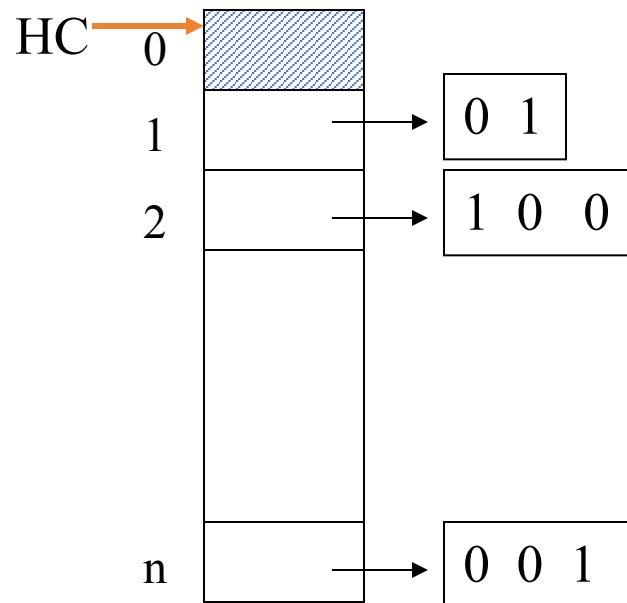
出现频次 $W=\{4, 2, 1, 2, 2, 1, 1, 1\}$



t	01
i	100
m	1110
e	101
r	110
s	1111
u	000
h	001

- 设需要编码的字符集合为 $\{d_1, d_2, \dots, d_n\}$ ，它们在电文中出现的次数或频率集合为 $\{w_1, w_2, \dots, w_n\}$ ，以 $d_1, d_2, \dots, d_n$ 作为叶结点， $w_1, w_2, \dots, w_n$ 作为它们的权值，构造一棵哈夫曼树。
- 哈夫曼编码是不等长编码
- 哈夫曼编码是前缀编码，即任一字符的编码都不是另一字符编码的前缀
- 哈夫曼编码树中没有度为1的结点。若叶子结点的个数为 n ，则哈夫曼编码树的结点总数为 $2n-1$
- 发送过程：根据由哈夫曼树得到的编码表送出字符数据
- 接收过程：按左0、右1的规定，从根结点走到一个叶结点，完成一个字符的译码。反复此过程，直到接收数据结束

[哈夫曼编码表的存储结构]



```
typedef char * * HuffmanCode;
```

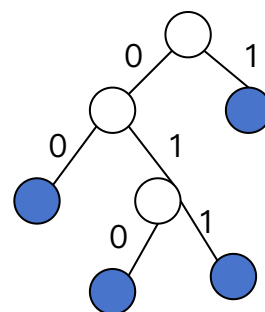
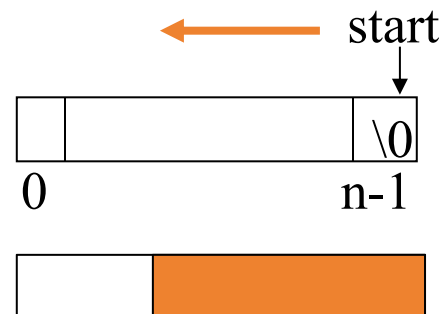
```
HuffmanCode HC;
```


[哈夫曼编码实现的算法步骤]

主数据结构：哈夫曼树HT 

哈夫曼编码表HC

辅助数据结构：cd {编码工作空间}



- 1) 分配HC空间;
- 2) 分配cd空间, 置 $cd[n-1] = "\0"$;
- 3) 依次求叶子HT[i]的编码HC[i], $i=1..n$
 - 3.1) 置cd中记录编码位置的初值: $start = n-1$;
置上溯的起点: $c = i$;
取c的父结点: $f = HT[c].parent$;
 - 3.2) 根据HT, c、f上溯直到根结点($f=0$), 依次产生的编码(c是f的左孩子编码为0; 否则为1)置入cd的相应编码空间 $cd[start-1]$ 并调整start(减1)
 - 3.3) 为第i个字符编码申请 $(n-start)$ 大小的字符空间HC[i], cd复制给HC[i]
- 4) 释放cd空间

```
void HuffmanCoding(HuffmanTree HT, int n, HuffmanCode &HC)
```

```
// 求哈夫曼树HT的n个字符的哈夫曼编码HC
```

```
{ HC=(HuffmanCode)malloc((n+1)sizeof(char *));
```

```
cd=(char *) malloc(n*sizeof(char)); //分配编码的工作空间
```

```
cd[n-1]='\0';
```

```
for (i=1; i<=n; ++i) { //为每个字符求编码
```

```
start=n-1;
```

```
for (c=i, f=HT[i].parent; f!=0; c=f, f=HT[f].parent )
```

```
//f是c的父节点
```

```
if (HT[f].lchild==c) cd[--start]="0"; //若c是f的左孩子, 记录编码0
```

```
else cd[--start]="1"; //否则为右孩子, 记录编码1
```

```
HC[i]=(char *)malloc((n-start)*size(char)); //申请存储第i个字符编码  
的空间
```

```
strcpy(HC[i], &cd[start]); //从临时工作空间cd拷贝到HC[i]
```

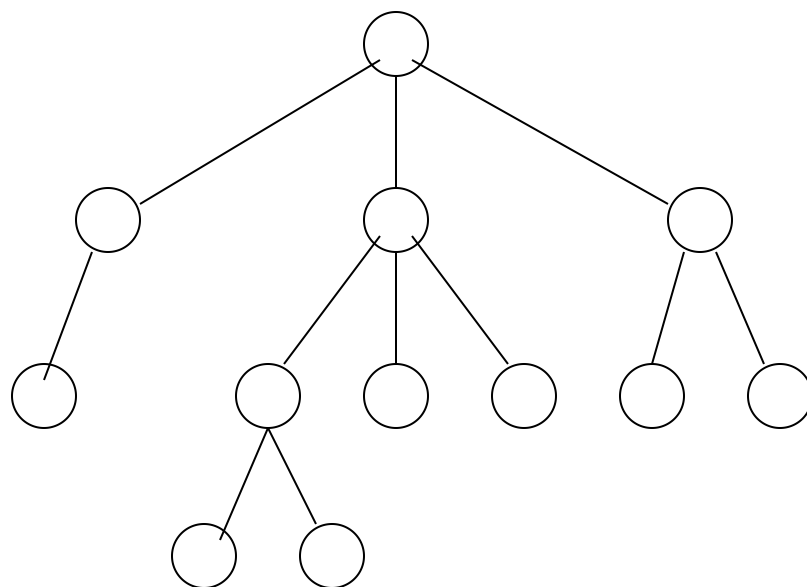
```
}
```

```
free(cd);
```

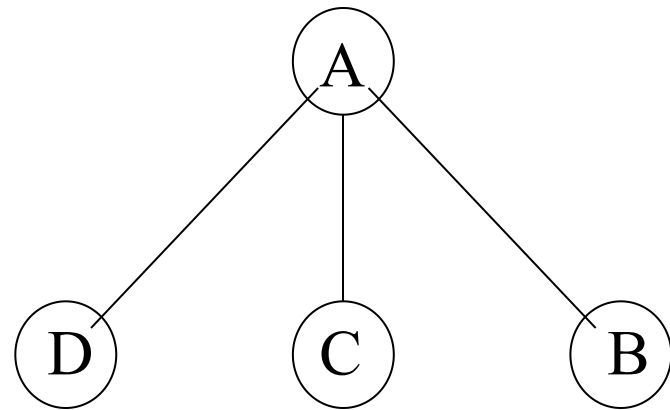
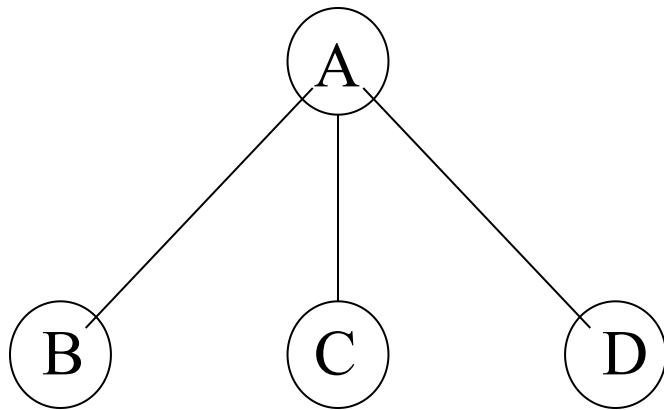
```
//HuffmanCoding
```

6.4 树和森林

树的概念



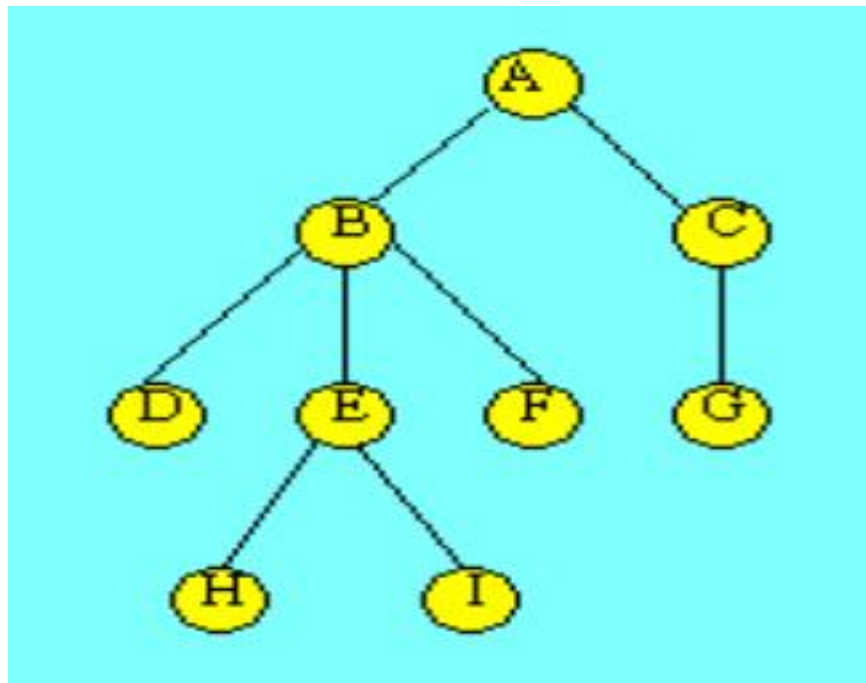
有序树和无序树



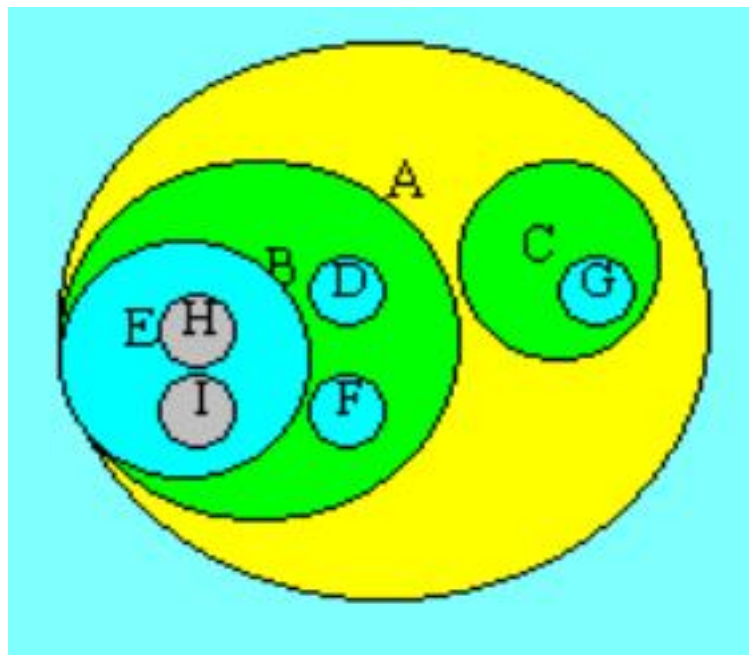
2. 树的四种表示方法

(1) 直观表示法

树的直观表示法就是以倒着的分支树的形式表示，下图就是一棵树的直观表示。其特点就是对树的逻辑结构的描述非常直观。是数据结构中最常用的树的描述方法。



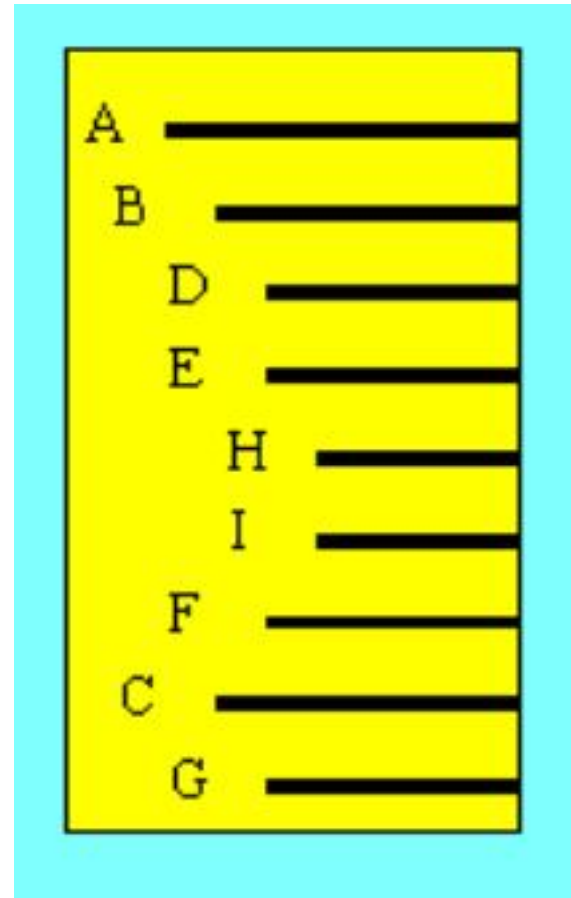
(2) 嵌套集合表示法



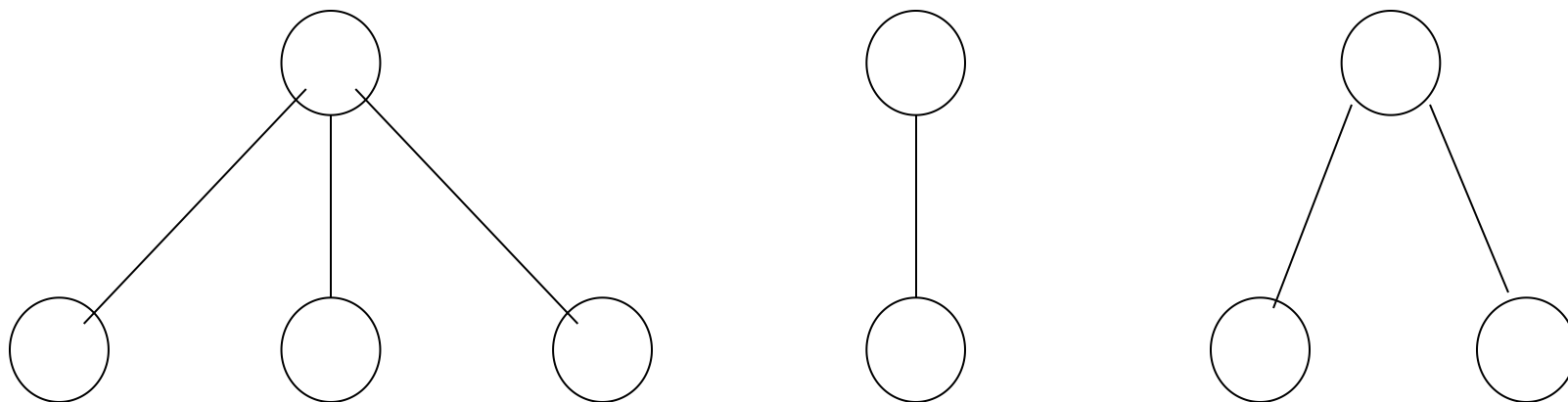
(3) 凹入表示法

(4) 广义表表示法

$(A(B(D, E(H, I), F), C(G)))$



森林



是 $m(m \geq 0)$ 棵互不相交的树的集合

6.4.2 树的存储

1. 双亲链表存储方法

```
#define MAXNODE    1024
```

```
//树中结点的最大个数，可根据实际情况进行修改
```

```
typedef struct
```

```
{
```

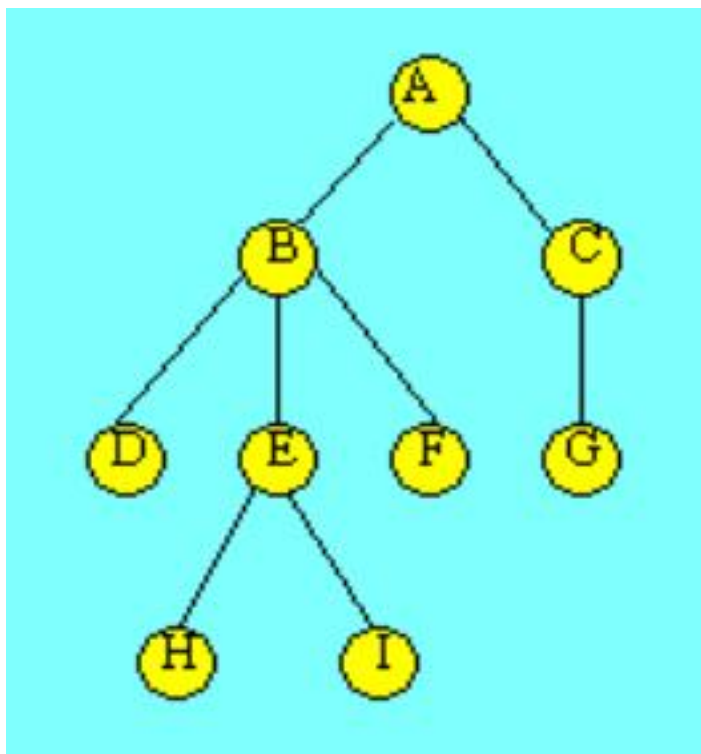
```
    datatype data;
```

```
    int parent;
```

```
} PNode;
```

```
PNode t[MAXNODE];
```

下图所示为树的双亲表示。图中用parent域的值为一1表示该结点无双亲结点，即该结点是一个根结点。



序号	data	parent
0	A	-1
1	B	0
2	C	0
3	D	1
4	E	1
5	F	1
6	G	2
7	H	4
8	I	4

对于实现Parent(t, x)操作和Root(x)操作很方便，但若求某结点的孩子结点时，则需查询整个数组。

2. 指向孩子的链表存储方法

(1) 多重链表法

令每个结点包括一个结点信息域和多个指针域，每个指针域指向该结点的一个孩子结点，通过各个指针域值反映出树中各结点之间的逻辑关系。常称为多重链表法。

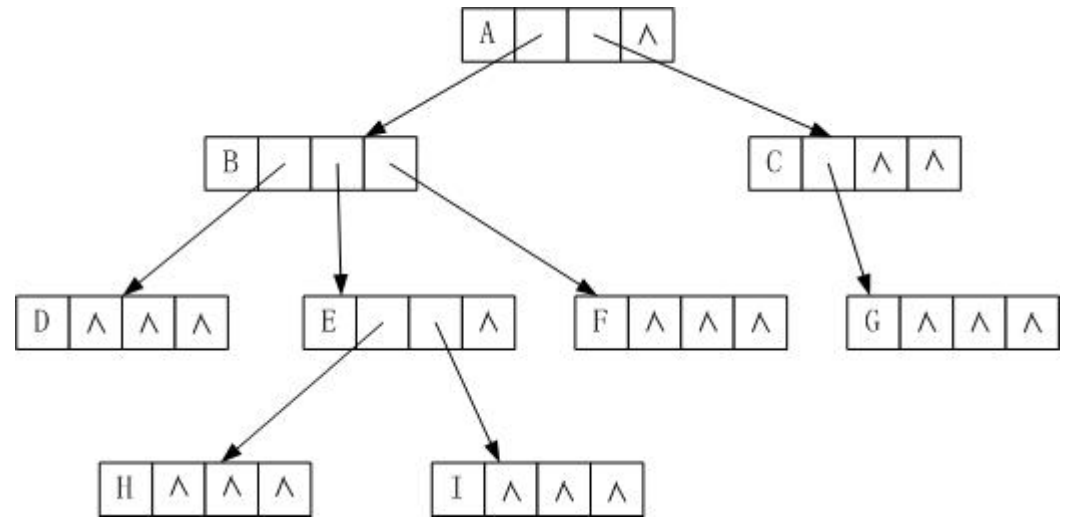
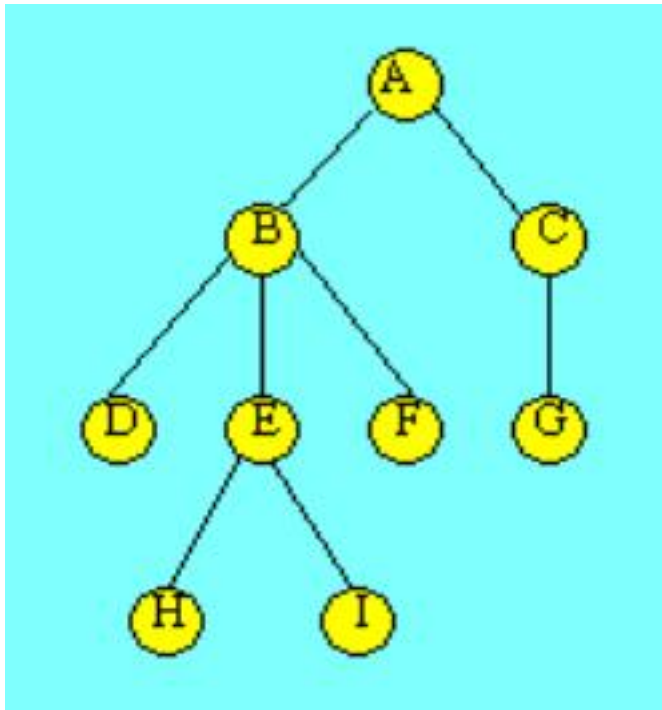
在一棵树中，各结点的度数各异，因此结点的指针域个数的设置有两种方法

- 1) 每个结点指针域的个数等于该结点的度数
- 2) 每个结点指针域的个数等于树的度数

方法1)虽然在一定程度上节约存储空间，但由于树中各结点不是同构的，操作不易实现，所以很少采用。

方法2) 各结点是同构的操作方便，但浪费存储空间。

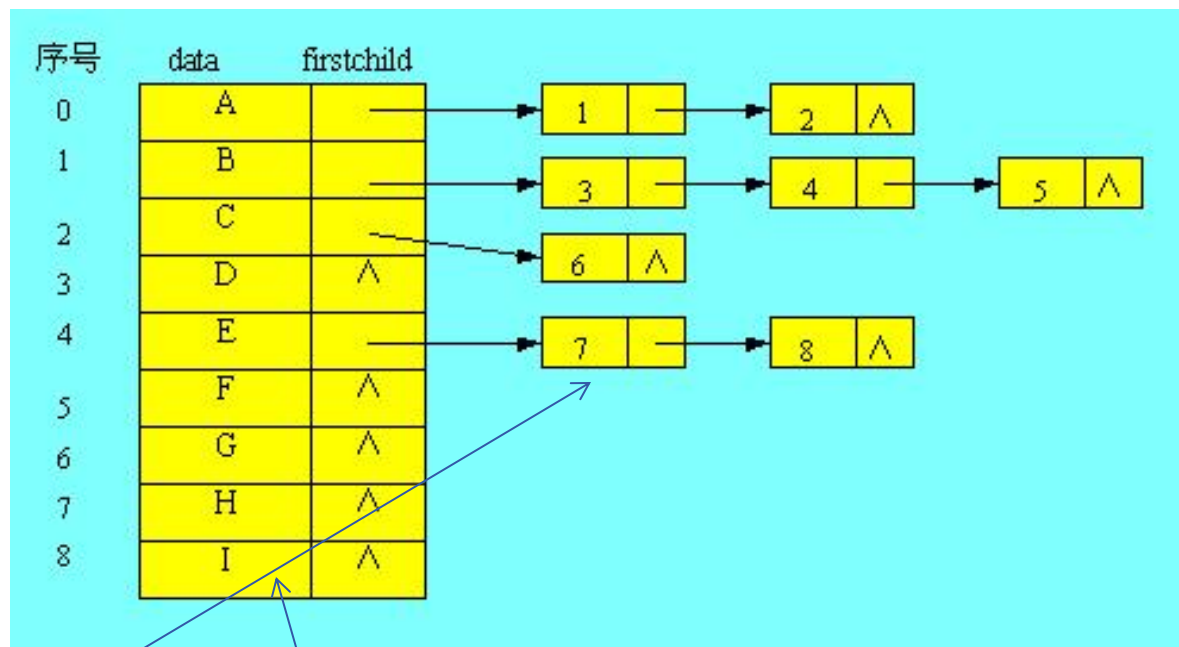
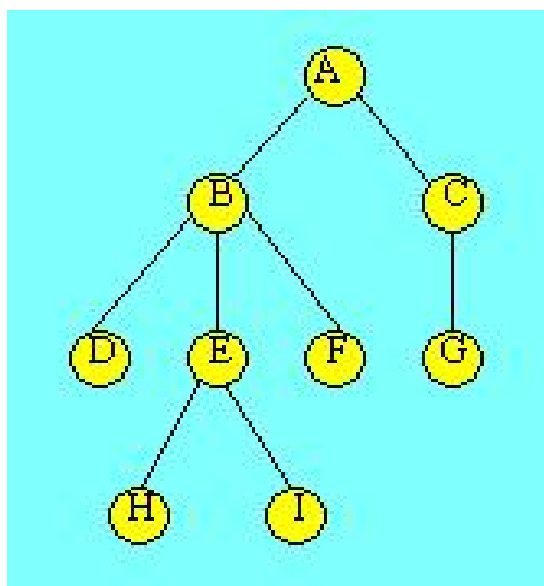
(1) 多重链表表法



```
typedef struct TreeNode
{
    datatype data;
    struct TreeNode * son[MAXSON];
}MSNode
```

(2) 孩子链表表示法 (查找孩子方便, 查找双亲比较困难)

树中每一个元素对应一个结点, 结点中有两部分信息, 一是其本身的数据信息, 二是其孩子链表的头指针, 即将每个数据元素的孩子链接成一个孩子链表。

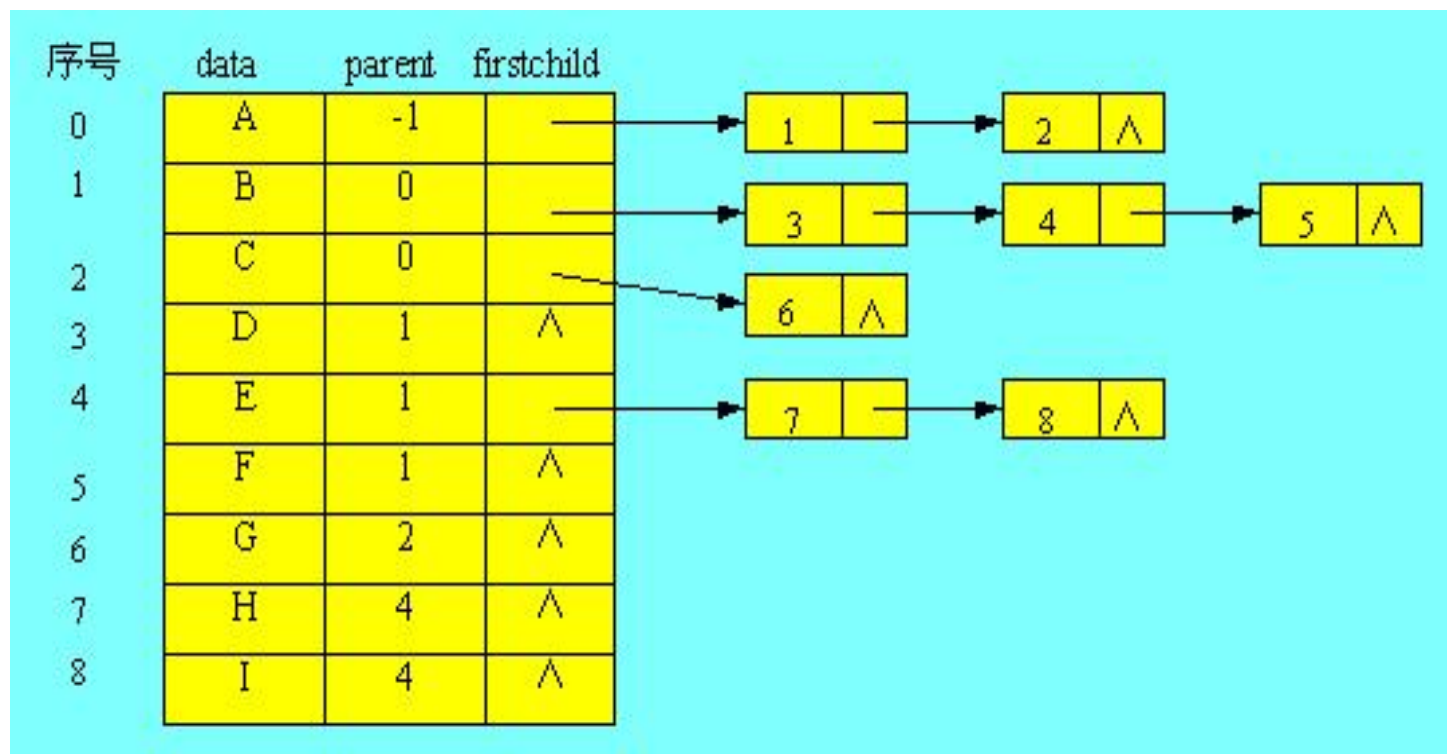


```
typedef struct childnode  
{  
    int childcode;  
    struct childnode * nextchild;  
}ChildNode;
```

```
typedef struct  
{  
    datatype data;  
    ChildNode * firstchild;  
}SNode;
```

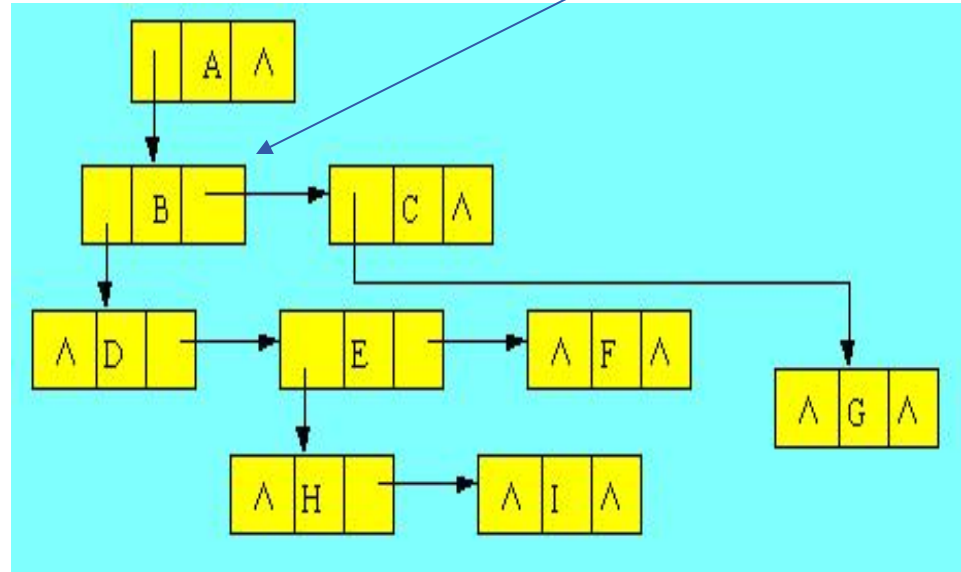
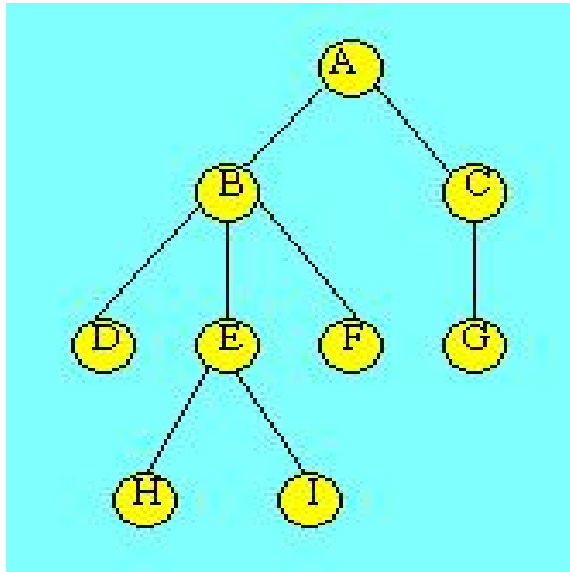
(1) 双亲孩子链表存储方法

将双亲表示法和孩子链表法想结合，在孩子链表存储的基础上，其主结点又加上了其双亲的静态指针。



(4) 孩子兄弟链表存储方法

左孩子，右兄弟



树中每个元素对应一个结点，每个结点除其信息域外，有两个指针域，分别指向该结点的第一个孩子结点和下一个兄弟结点。

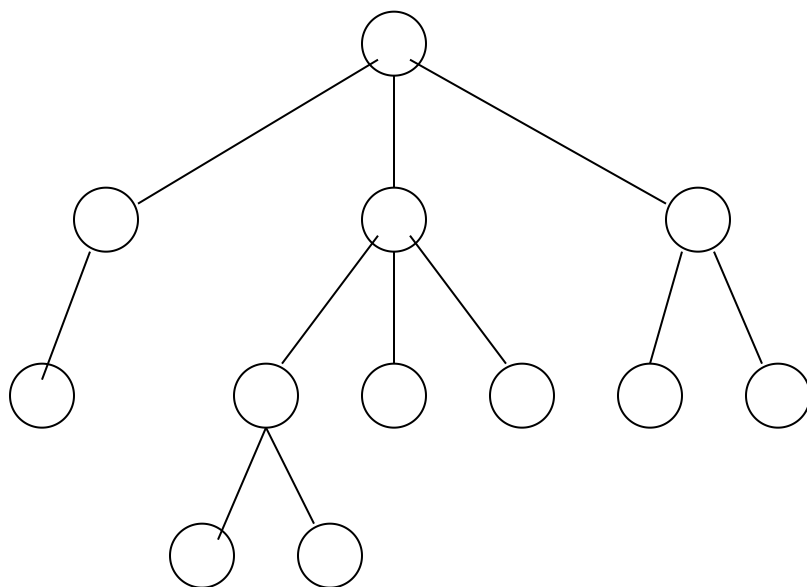
```
typedef struct csnode
{
    datatype data;
    struct csnode* lchild;
    struct csnode* nextsibling;
} SNode;
```


6.4.3.1 树与二叉树的转换

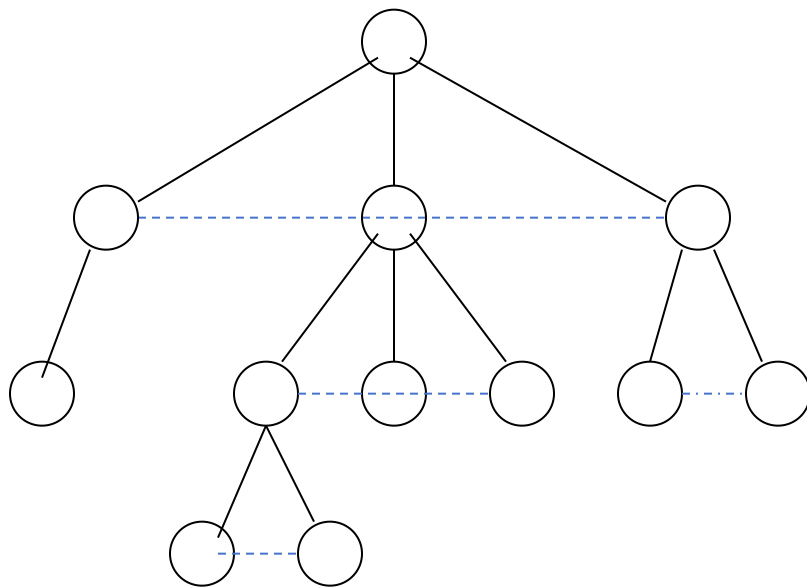
从树的孩子兄弟表示法可以看到，如果设定一定规则，就可用二叉树结构表示树和森林，这样，对树的操作实现就可以借助二叉树存储，利用二叉树上的操作来实现。

在树或森林与二叉树之间有一个一一对应关系。任何一个树或森林可唯一对应到一棵二叉树；反之，任何一棵二叉树也能唯一地对应到一个树或森林。

树与二叉树的转换

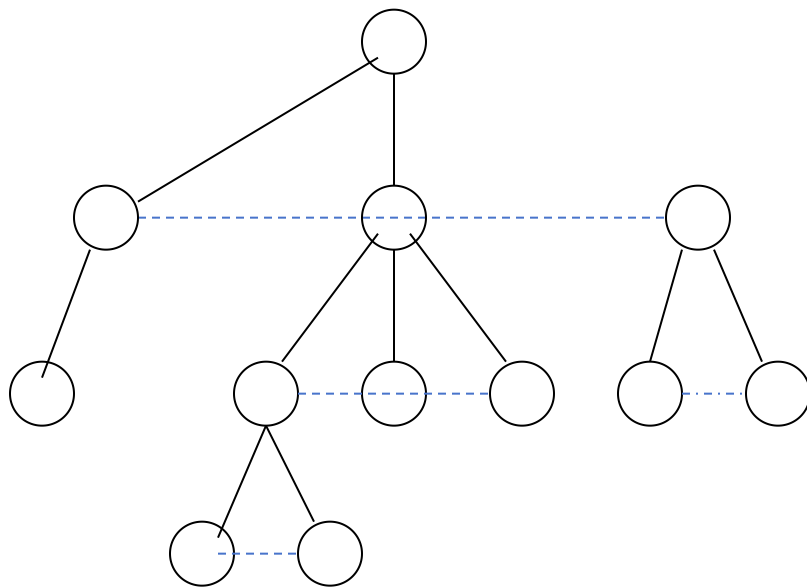


树与二叉树的转换



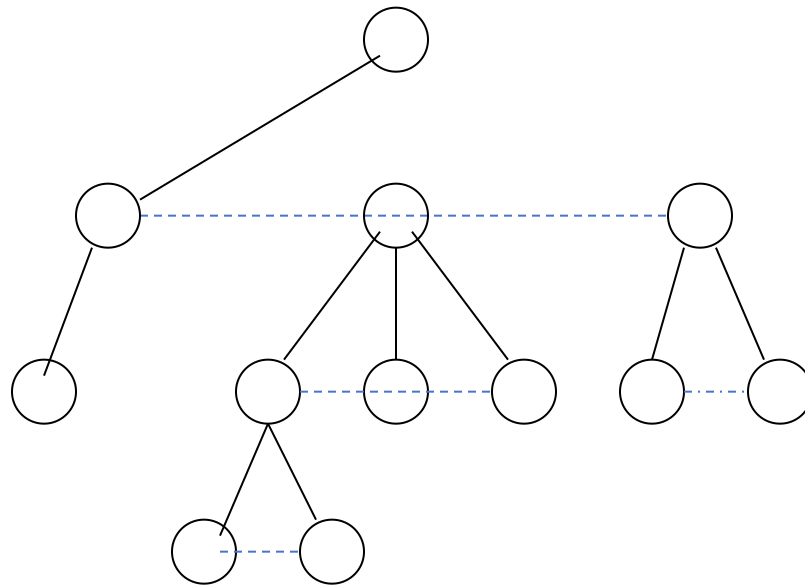
- 1 在所有兄弟结点之间加一条连线
- 2 对每个结点，除了保留与其长子的连线外，去掉该结点 与其他孩子的连线。

树与二叉树的转换



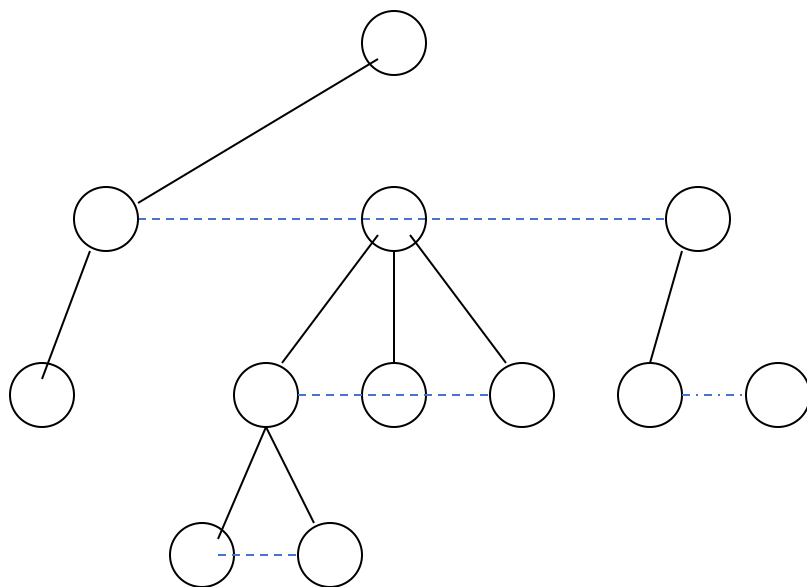
- 1 在所有兄弟结点之间加一条连线
- 2 对每个结点，除了保留与其长子的连线外，去掉该结点与其他孩子的连线。

树与二叉树的转换

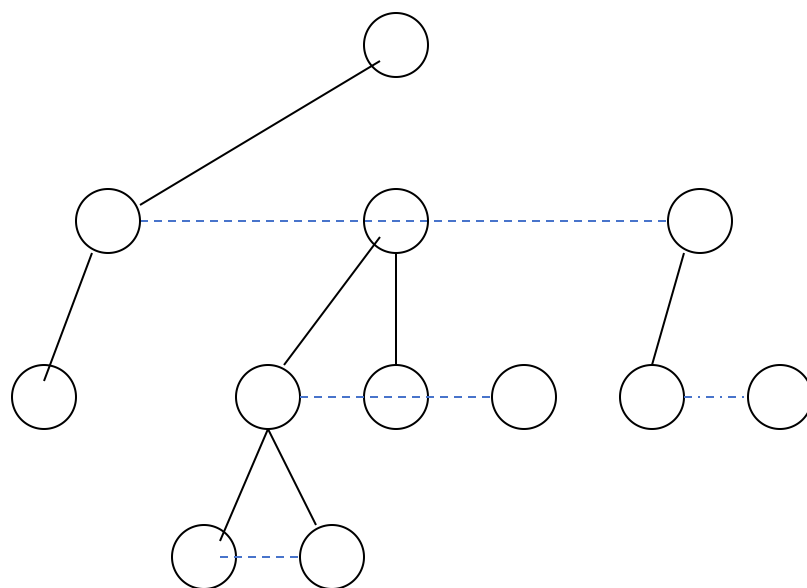


- 1 在所有兄弟结点之间加一条连线
- 2 对每个结点，除了保留与其长子的连线外，去掉该结点与其他孩子的连线。

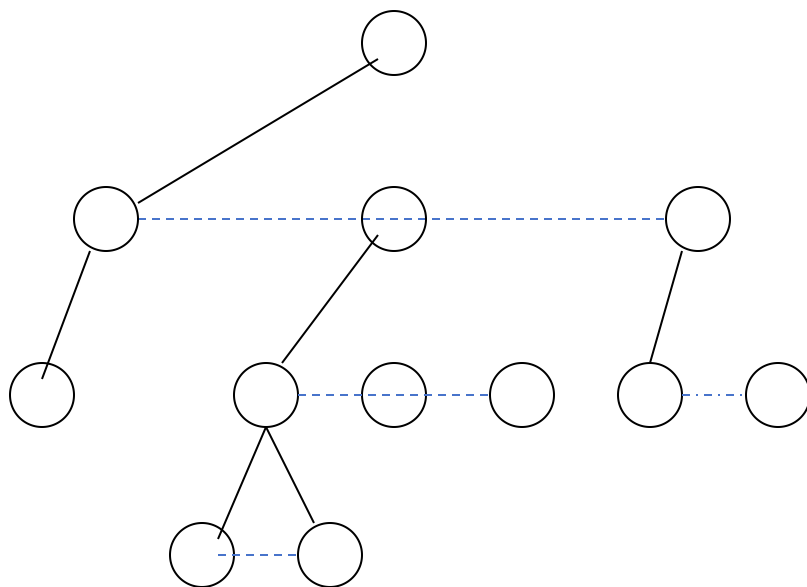
树与二叉树的转换



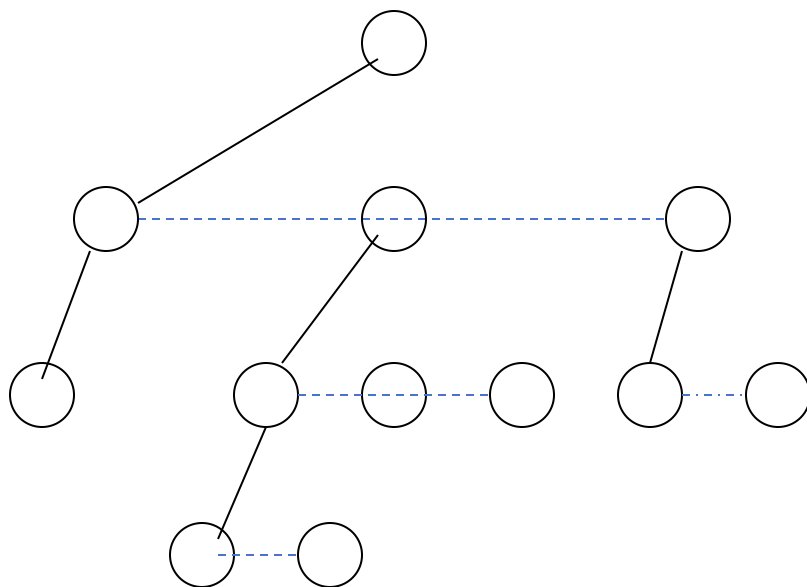
树与二叉树的转换



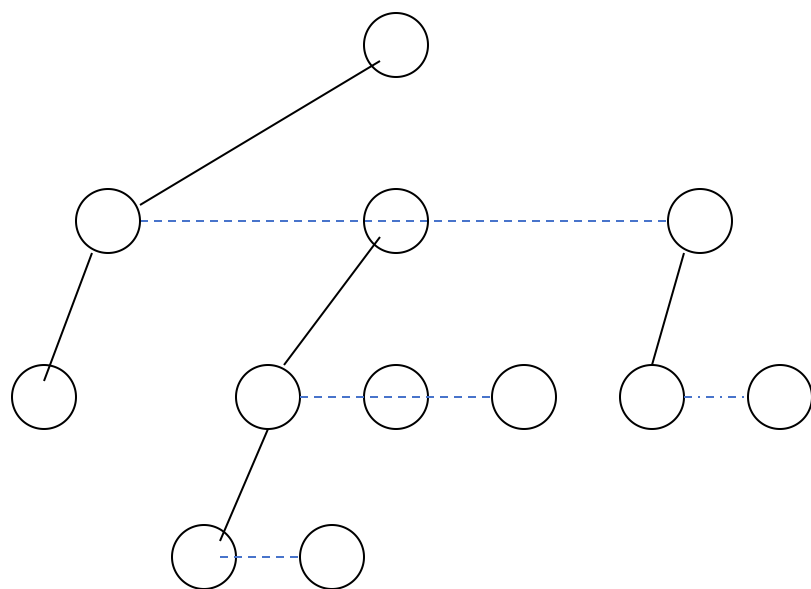
树与二叉树的转换



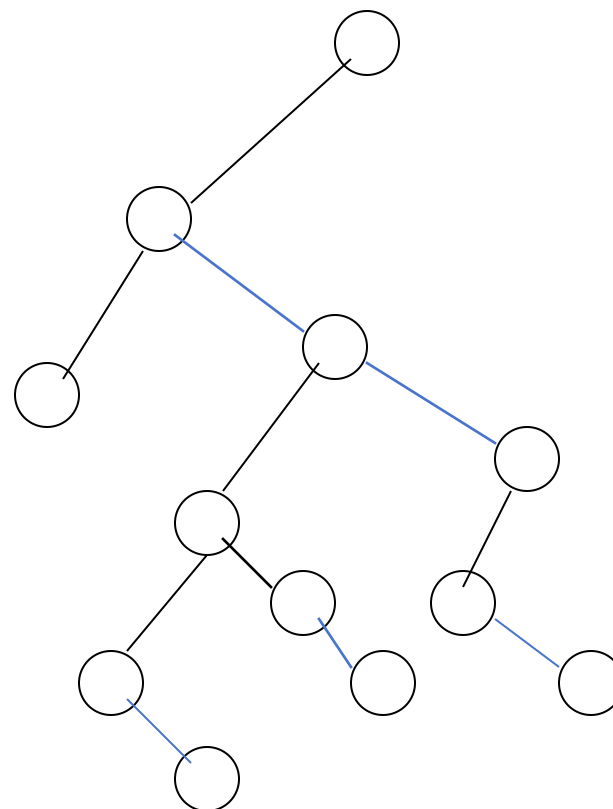
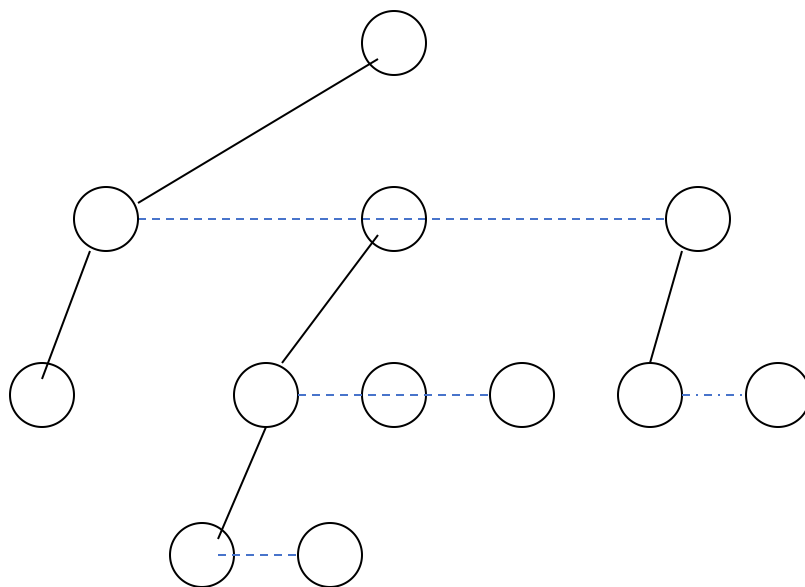
树与二叉树的转换



树与二叉树的转换

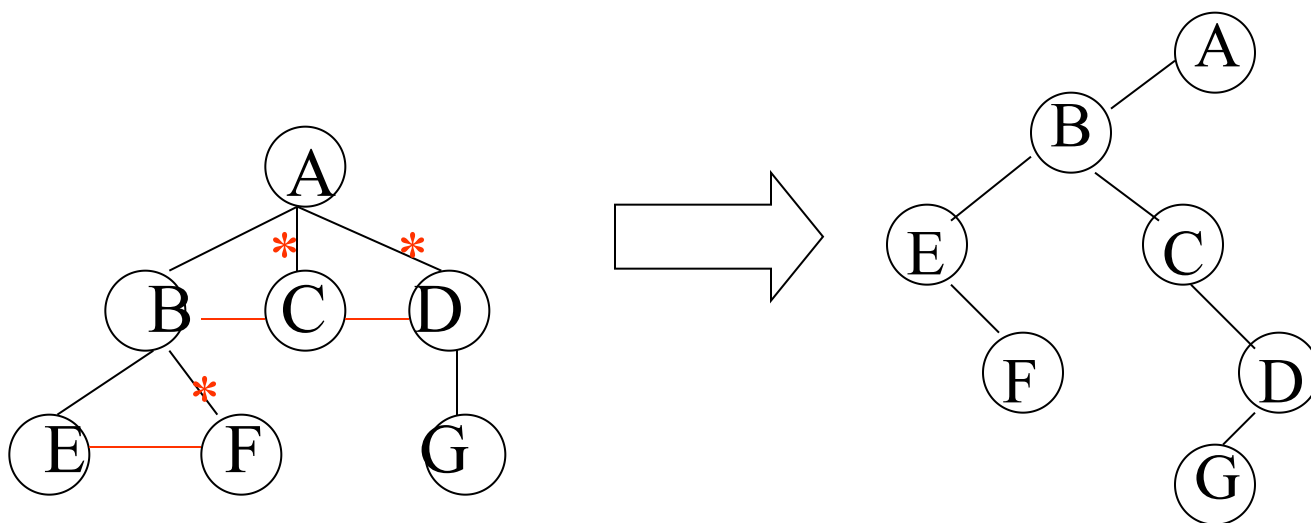


树与二叉树的转换



[树转换为二叉树]

- 1)在兄弟间加一连线
- 2)对每一结点，去掉它与孩子的连线(最左子除外)
- 3)以根为轴心将整棵树顺时针转 45°



特点:

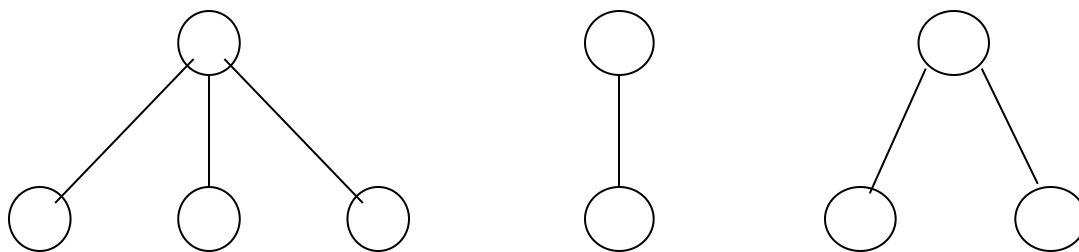
无右子树

左支是孩子

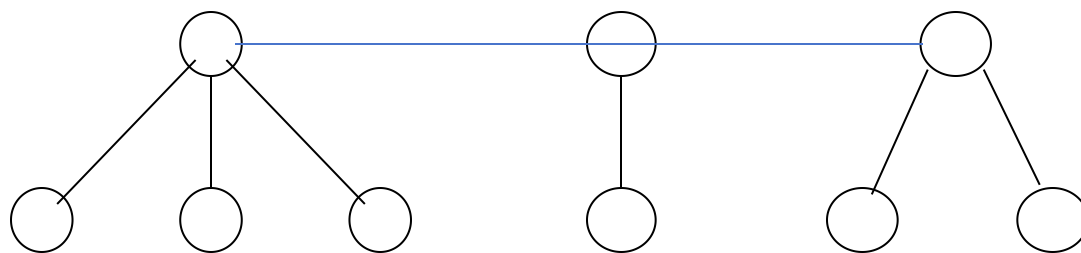
右支是兄弟

6.4.3.2 森林与二叉树的转换

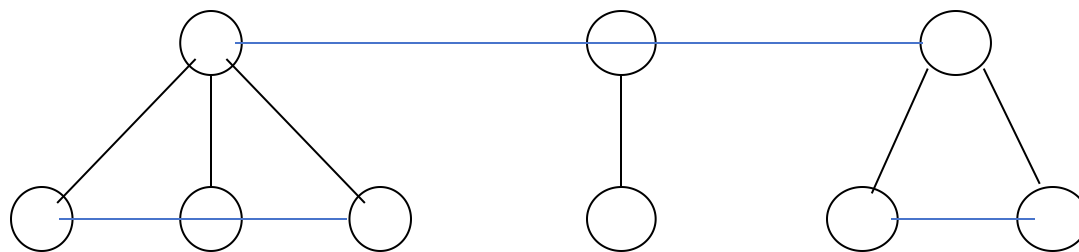
森林与二叉树的转换



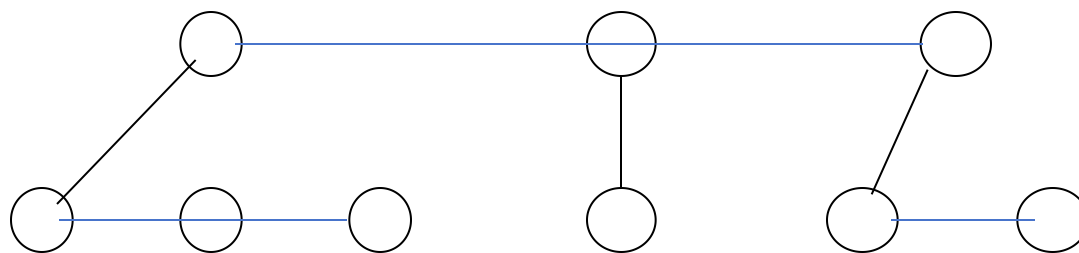
森林与二叉树的转换



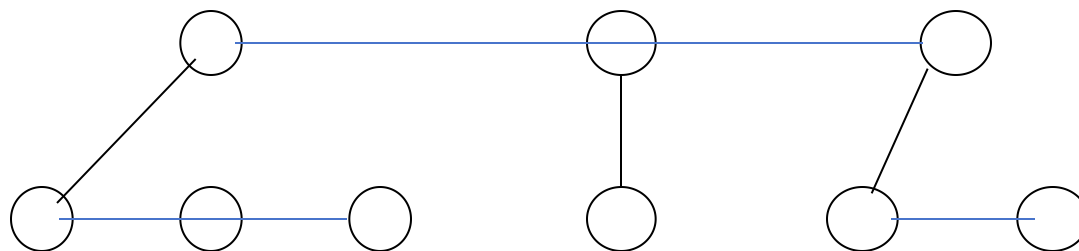
森林与二叉树的转换



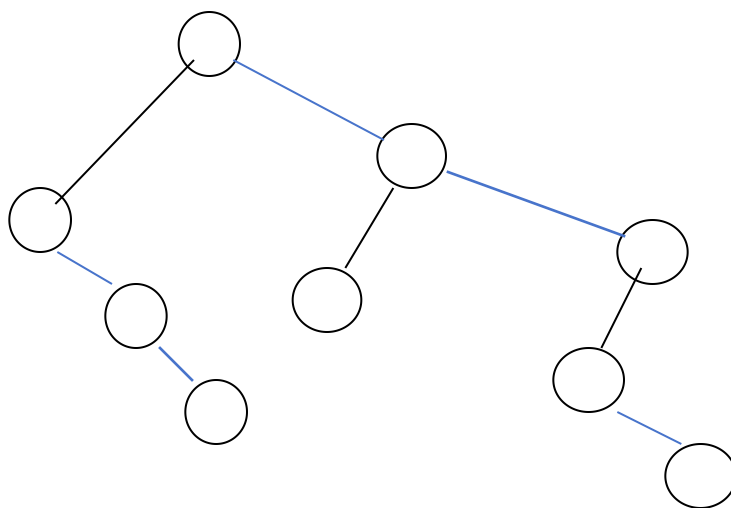
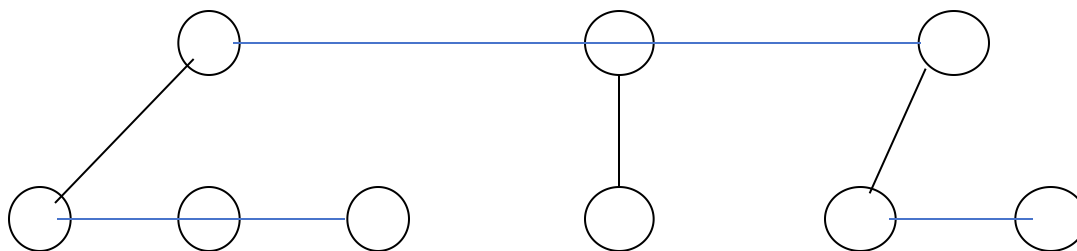
森林与二叉树的转换



森林与二叉树的转换

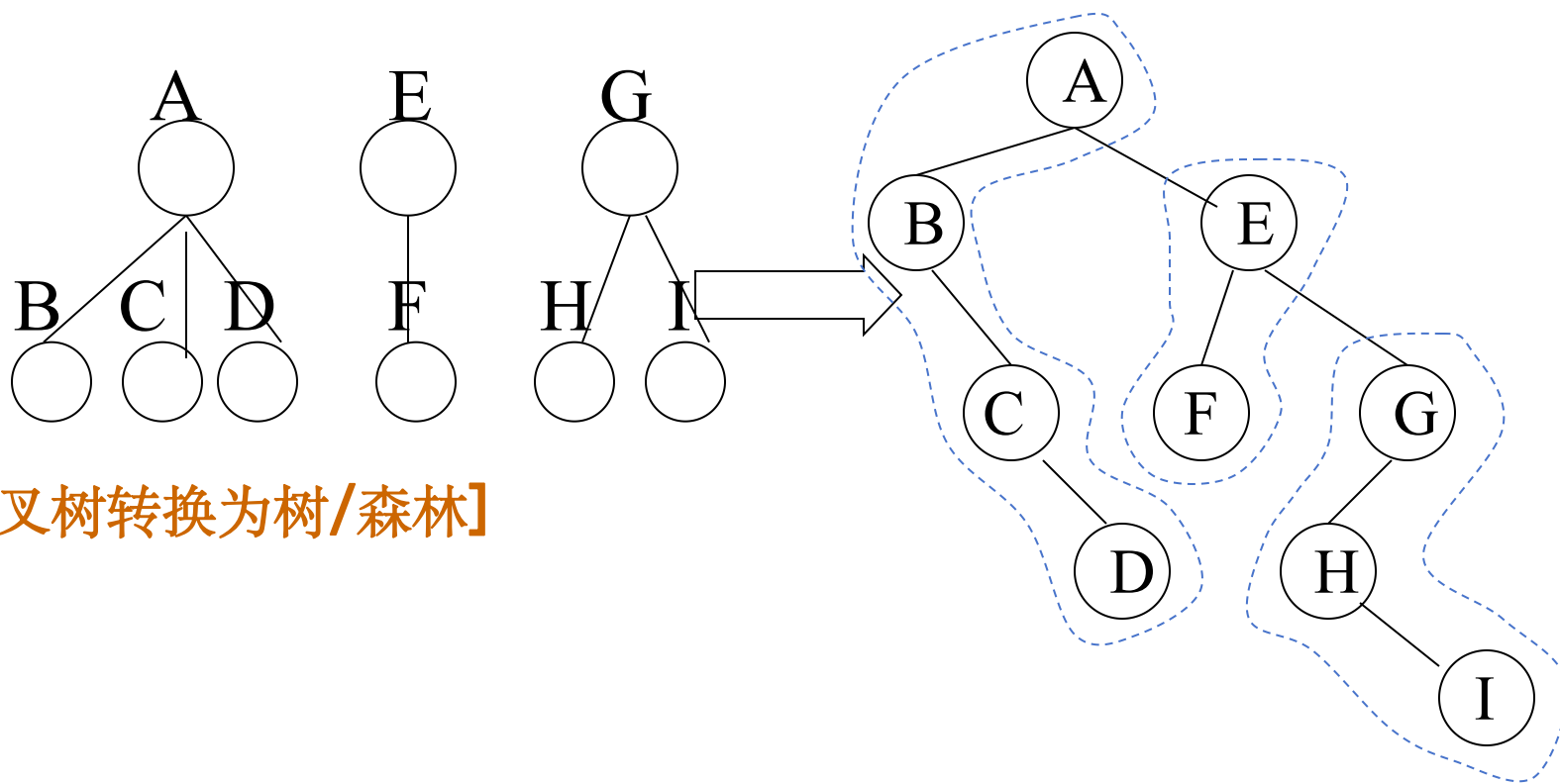


森林与二叉树的转换



[森林转换为二叉树]

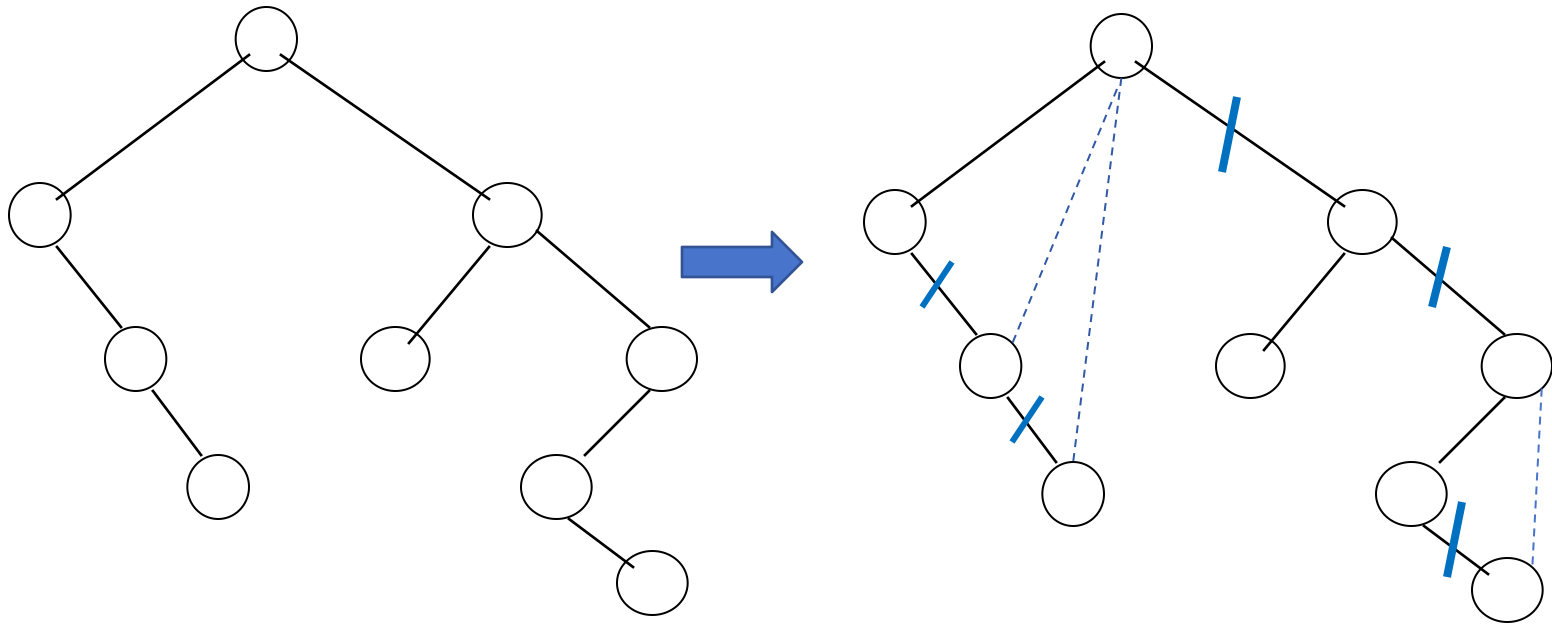
- 1) 先将森林里的每一棵树转换成一棵二叉树
- 2) 从最后一棵树开始，把后一棵树的作为前一棵树的根的右子



[二叉树转换为树/森林]

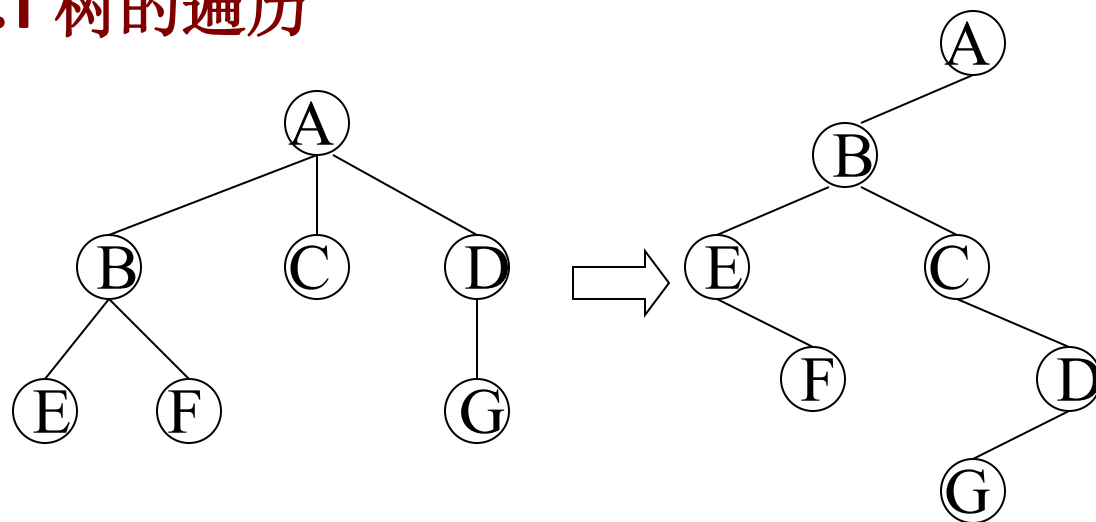
二叉树到树、森林的转换

同样，也有方法把二叉树转换到树或森林：若结点 x 是其双亲 y 的左孩子，则把 x 的右孩子、右孩子的右孩子、...，都与 y 用连线连起来，最后去掉所有双亲到右孩子的连线。



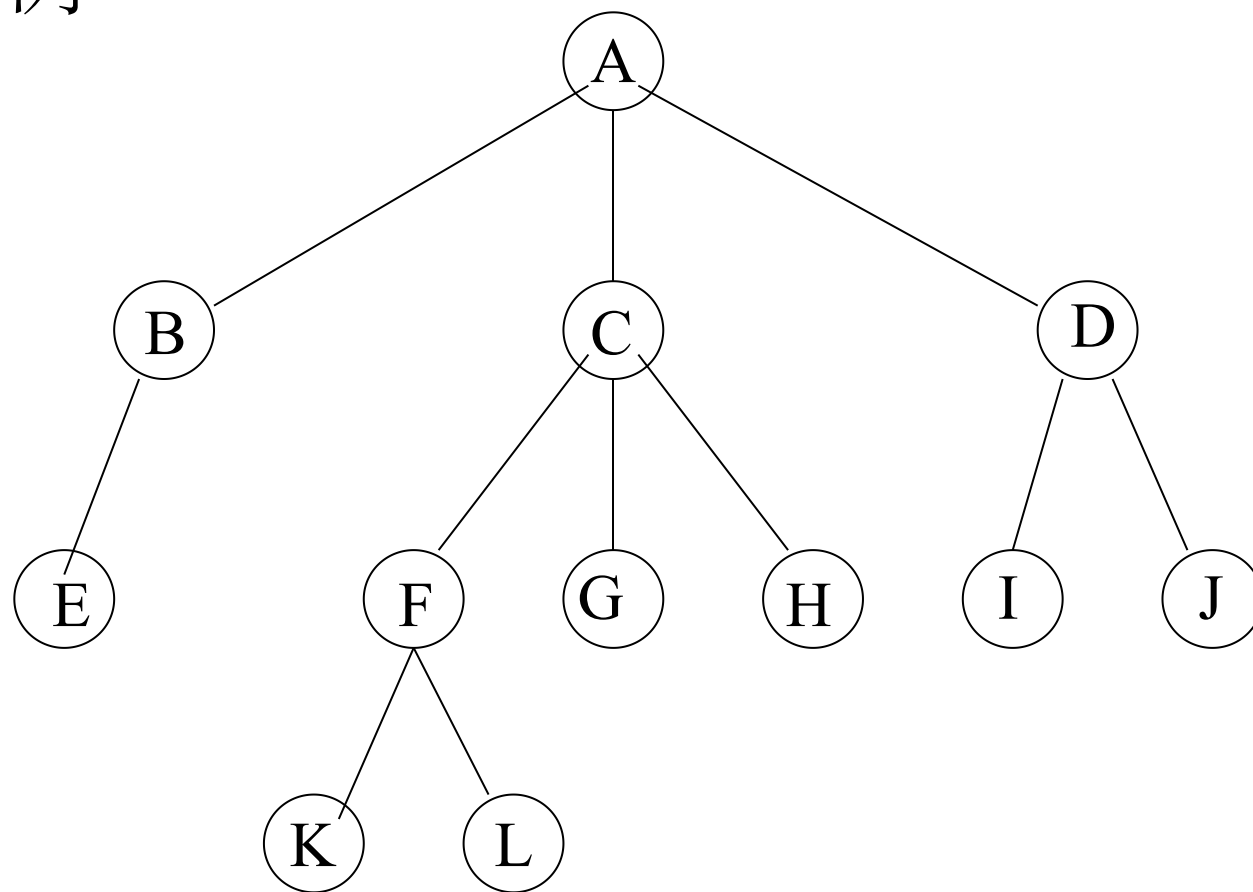
6.4.4 树和森林的遍历

6.4.4.1 树的遍历



- **先序遍历** 先访问树的根结点，然后依次先根遍历根的**每棵子树**
ABEFCDBG(等效于相应二叉树先序)
- **后序遍历** 先依次后根遍历根的**每棵子树**，然后访问树的根结点
EFBCGDA(等效于相应二叉树中序)

例



先根遍历:

A B E C F K L G H D I J

后根遍历:

E B K L F G H C I J D A

6.4.4.2 森林的遍历

- 先序遍历

访问第一棵树的根结点;

先序遍历第一棵树的根
的子树森林;

先序遍历除第一棵树外
剩余的树构成的森林

(逐棵先序遍历每棵子树/ 对应
二叉树的先序遍历)

- 中序遍历

中序遍历第一棵树的根
的子树森林;

访问第一棵树的根结点;
中序遍历除第一棵树外
剩余的树构成的森林

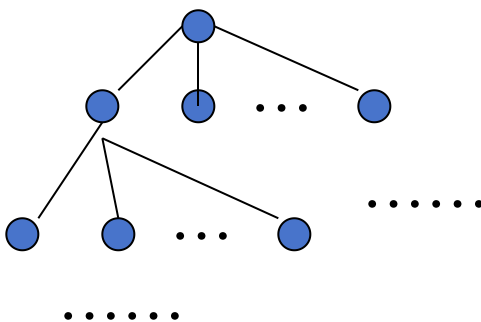
(逐棵中序遍历每棵子树/
对应二叉树的中序遍历)

6.7 回溯法与树的遍历

回溯求解问题

[回溯法求解步骤]

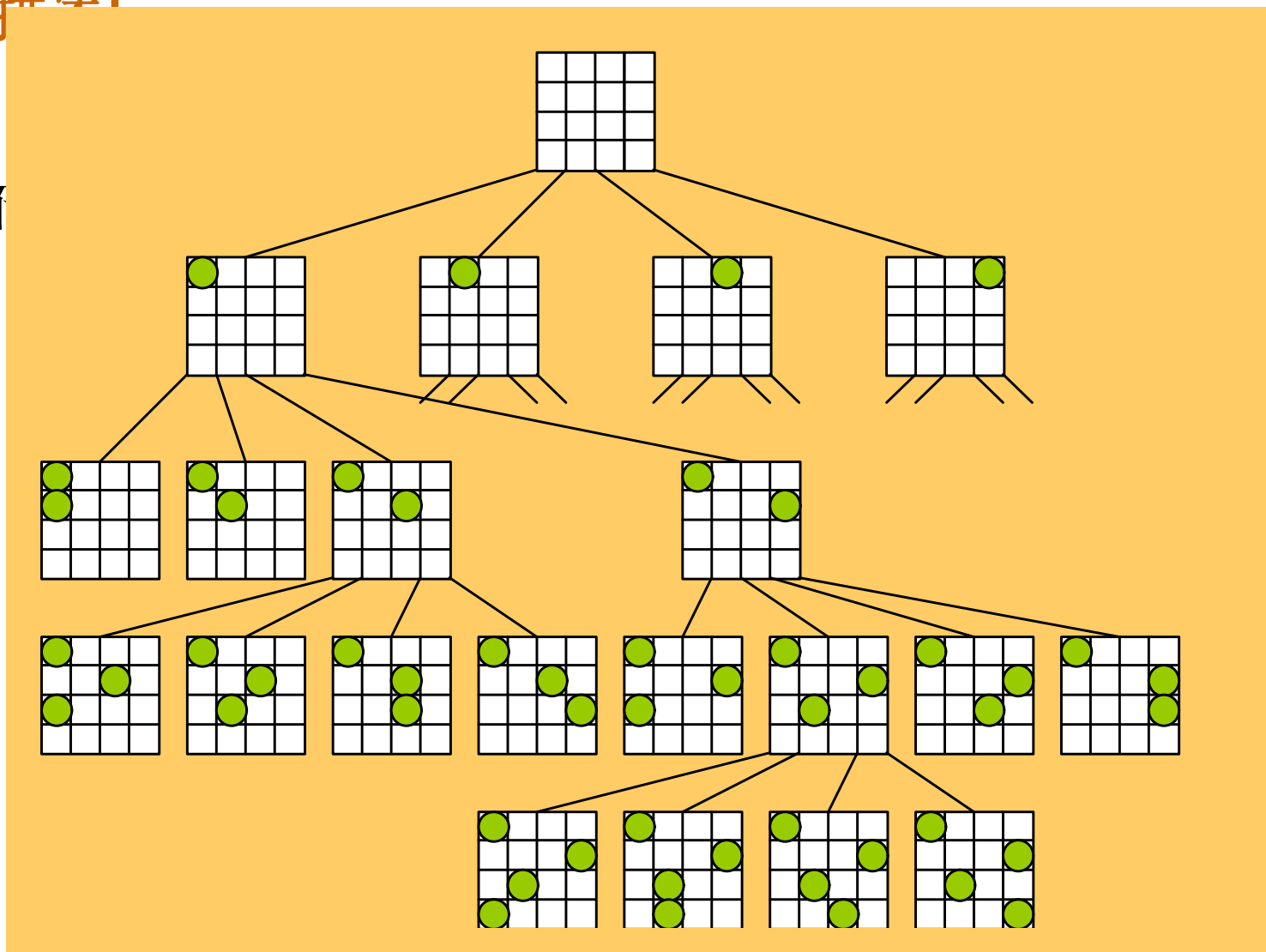
- 1) 定义一个解空间（**搜索树**），它包含问题的解
- 2) 用适合于搜索的方式组织该空间
- 3) 在约束条件下先序遍历（深度优先）搜索该解空间，并在遍历过程中剪去那些不满足条件的分支。



四皇后问题

[问题描述]

子都不



[算法描述]

```
void trial(int i, int n)
{ // 进入本函数时, 前i-1已放置好, 现在从i行选择合适的
  位置
    if ( i>n )  输出棋盘的当前布局; //已经摆满n个棋子
    else for ( j=1; j<=n; ++j ) {
        在第i行第j列放一个棋子;
        if (当前布局合法) trial(i+1, n); //放下一行
        移走第i行第j列的棋子;
    }
} //trial
```

八皇后问题调用: **trial(1,8)**



回溯求解问题

【回溯法的一般形式】

```
void trial(ElemType r, ...)
{
    if (当前所得解r为所求) printf( r );
    else for (i=1; i<=n; ++i) {
        将当前解r在第i维修改;
        if (修改后的r合法) then trial(r);
        将r恢复为未修改;
    }
}

//trial
```

